

Dossier De Conception (DDC)

du projet

Robot Mini-Sumo

Responsabilité documentaire

Action	NOM Prénom	Fonction	Date	Signature
Rédigé par	PETRY Elou Anne TUCCIO Chiara VULLIEZ Mattéo LEROY Gabin DUPUCH Gabriel	Technicien	07/10/2025	
Approuvé par	M.Champion M.Combastel (IUT GEII Bdx)	Chef de projet	07/10/2025	
Approuvé par	M.Champion M.Combastel (IUT GEII Bdx)	Client	07/10/2025	

Suivi des révisions documentaires

Indice	Date	Nature de la révision
1	01/09/2022	Publication préliminaire du DDC document à compléter par le Technicien.
2	07/10/2025	Première publication
3	18/11/2025	Deuxième publication

Documents de références

Sigle	Référence	Titre	Rév.	Origine
[CDC]	RMS_CDC	Cahier des charges	1	IUT GEii Bdx

Table des matières

1. Nature du document	6
2. Conception préliminaire du produit	6
2.1 Architecture Électronique	6
2.1.1 Choix des capteurs adversaire avant	10
2.1.2 Choix des capteurs adversaire latéraux	12
2.1.3 Choix des capteurs de sol	13
2.1.4 Choix du capteur infrarouge	17
2.1.5 Choix du pont en H	18
2.1.6 Choix du régulateur de tension	24
2.1.7 Choix de la batterie	24
2.2 Architecture Informatique	27
2.2.1 Fonctions d'acquisition	27
2.2.2 Fonctions d'action	28
2.2.3 Fonctions d'énergie	30
2.2.4 Fonctions de traitement	31
2.3 Conclusion de la conception préliminaire du produit	44
3. Conception détaillée du produit	44
3.1 Conception détaillée du robot mini-sumo	44
3.2 Conception détaillée de la partie acquisition	45
3.2.1 Capteurs adversaires avant	47
3.2.1.1 Schéma électrique des capteurs adversaires avant	47
3.2.1.2 Code informatique des capteurs adversaires avant	48
3.2.2 Capteurs de sol (ligne)	52
3.2.2.1 Schéma électrique des capteurs de sol	52
3.2.2.2 Code informatique des capteurs de sol	55
3.2.3 Capteur infrarouge	58
3.2.3.1 Schéma électrique du capteur infrarouge	58
3.2.3.2 Code informatique du capteur infrarouge	60
3.2.4 Capteurs adversaires latéraux	62
3.2.4.1 Schéma électrique des capteurs adversaires latéraux	62
3.2.4.2 Code informatique des capteurs adversaires latéraux	63
3.3 Conception détaillée de la partie action	65
3.3.1 Pont en H	66
3.3.1.1 Schéma électrique du pont en H	66
3.3.1.2 Code informatique du pont en H	68
3.3.2 LED	73
3.3.2.1 Schéma électrique de la LED	73

3.3.3 Buzzer	75
3.3.3.1 Schéma électrique du buzzer	75
3.4 Conception détaillée de la partie énergie	77
3.4.1 Pont diviseur de tension pour la génération de la tension de référence	77
3.4.1.1 Schéma électrique du pont diviseur de tension pour la tension de référence	77
3.4.2 Pont diviseur de tension pour la mesure de la tension de la batterie	79
3.4.2.1 Schéma électrique du pont diviseur de tension de la batterie	80
3.4.3 Régulateur de tension	81
3.4.3.1 Schéma électrique du régulateur de tension	81
3.4.3.2 Condensateurs de découplage	82
3.4.4 Comparateur	83
3.4.4.1 Schéma électrique du comparateur	83
3.4.4.2 Code informatique de l'étage comparateur	85
3.4.5 Batterie	85
3.4.5.1 Schéma électrique de la batterie	85
3.4.6 Diode	87
3.4.6.1 Schéma électrique diode de protection	87
3.4.7 Interrupteur	87
3.4.7.1 Schéma électrique de l'interrupteur	88
3.4.8 Connecteur extension	88
3.4.8.1 Schéma électrique du connecteur extension	88
3.5 Conception détaillée de la partie traitement	89
3.5.1 Le microcontrôleur ATMEGA328P-PU	89
3.5.1.1 Schéma électrique du microcontrôleur	89
3.5.1.2 Code informatique du microcontrôleur	90
Setup	90
Main	91
3.6 Conception détaillée de coût et de délai	98
3.6.1 Le coût	98
3.6.2 Le délai	98
4. Dérivage des solutions techniques retenues	99
Acquisition	99
4.1 Simulation du Capteur infrarouge	99
4.1.2 Simulation des capteurs de sol	107
4.1.3 Simulation des capteurs télémétriques	110
4.1.4 Simulation des capteurs ultrasons	112
Energie	116
4.2.1 Prototypage rapide des ponts diviseur de tension de référence	116
4.2.2 Prototypage rapide des ponts diviseur de tension Batterie	118

4.2.3 Simulation / prototypage Comparateur	121
4.2.4 Prototypage rapide Diode et Interrupteur	123
Action	125
4.3 Prototypage rapide pont en H	125
4.3 Conclusion de la simulation / prototypage rapide du produit	130
5. Conclusion de la conception du produit	131
6. Matrice de conformité du produit	131

1. Nature du document

Ce document est un dossier de conception et a pour but de détailler la conception du produit développé. Il apporte ainsi des preuves de la conformité du produit par rapport à l'ensemble des exigences client. Le paragraphe 3 du [CDC] décrit de façon plus détaillée la nature et le positionnement de ce document dans l'arborescence documentaire du projet.

2. Conception préliminaire du produit

Ce chapitre décrit l'architecture fonctionnelle du produit. Il apporte les premiers éléments de preuve de la faisabilité du produit vis-à-vis des exigences client.

2.1 Architecture Électronique

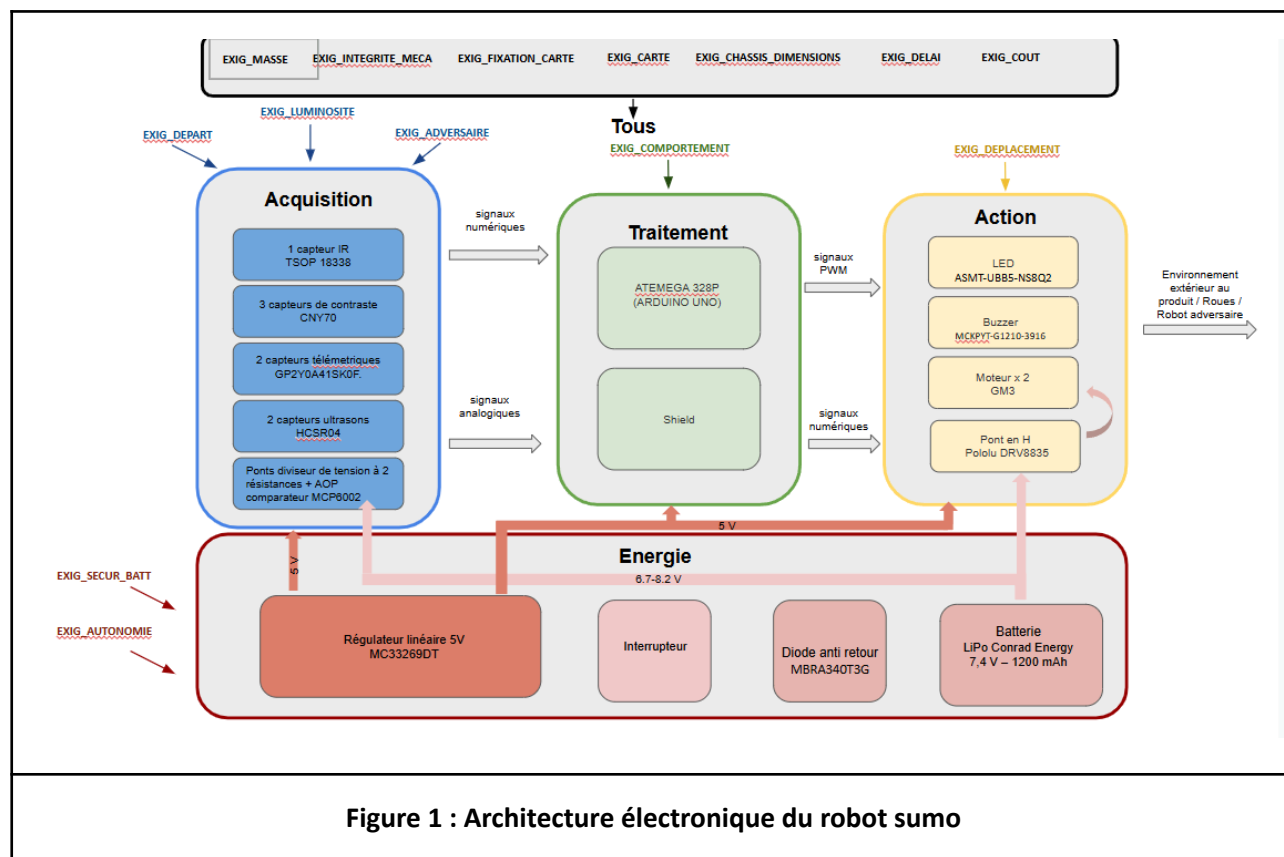
Référence de pré-conception : CPR01

Compétences GEII : C1a3 C1a4 C1a9 C1a10 C1a11

Exigences client vérifiées par préconception : EXIG_COMPORTEMENT EXIG_AUTONOMIE
EXIG_SECUR_BATT EXIG_DEPLACEMENT EXIG_ADVERSAIRE EXIG_LUMINOSITE EXIG_DEPART
EXIG_CARTE EXIG_COUT EXIG_CHASSIS_DIMENSIONS EXIG_MASSE EXIG_INTEGRITE_MECA
EXIG_FIXATION_CARTE

Afin de répondre au cahier des charges, une analyse globale des exigences a conduit à l'architecture fonctionnelle présentée ci-dessous.

Robot Mini-Sumo (RMS)



Pour le bloc “Acquisition” le cahier des charges nous impose trois exigences :

- **EXIG_DEPART** : Pour répondre à cette exigence, un capteur infrarouge est intégré au robot afin de détecter et lire les trames NEC émises par la télécommande TSOP18338. Ces trames correspondent aux appuis sur les boutons spécifiques (par exemple, Start, Attention, Stop).

Pour respecter les contraintes du règlement, qui stipule qu'un dépassement de la ligne blanche entraîne une défaite, nous avons également intégré un capteur de contraste. Ce capteur permet de détecter la ligne blanche sur la surface du dohyo et de déclencher une action appropriée pour éviter que le robot ne sorte de cette zone autorisée.

- **EXIG_LUMINOSITE** : Afin de répondre à cette exigence, nous avons réalisé des tests sur les capteurs de contraste, susceptibles d'être influencés par un changement brutal de luminosité et donc de fausser la détection (blanc ou noir). Pour simuler ce cas, nous avons utilisé la lampe de nos téléphones, pouvant s'apparenter à des flashes d'appareil photo, afin de reproduire une situation de luminosité extrême. Nous avons ensuite développé un

code pour faire fonctionner le capteur et avons constaté qu'il ne présentait aucun problème de détection du contraste.

- **EXIG_ADVERSAIRE** : Pour répondre à cette exigence, nous avons intégré plusieurs capteurs sur le robot. Pour détecter un adversaire positionné à l'avant, deux capteurs à ultrasons sont installés à l'avant du robot. Pour les détections latérales, deux capteurs télémétriques sont placés respectivement sur les côtés gauche et droit du robot. Néanmoins, seuls les capteurs à ultrasons répondent à l'exigence stipulant que le robot doit être capable de détecter la présence de son adversaire à une distance minimale de 40 cm. Les autres capteurs, quant à eux, ne permettent qu'une détection jusqu'à 30 cm. Cependant, cette exigence ne précise pas que tous les capteurs doivent atteindre une portée de 40 cm.

Pour le bloc "Énergie" le cahier des charges nous impose deux exigences :

- **EXIG_SECUR_BATT** : Pour répondre à cette exigence, nous devons connaître la tension de la batterie afin de couper l'alimentation avant que celle-ci ne descende en dessous de 6,7V. Pour cela, nous avons choisi d'utiliser un pont diviseur de tension composé de deux résistances, permettant d'obtenir une tension comprise entre 0V et 5V, compatible avec une entrée ADC de l'Arduino et avec un comparateur (type alimenté en 5V).

La sortie du pont diviseur est envoyée à la fois sur une entrée ADC de l'Arduino et sur l'entrée d'un comparateur.

Le comparateur compare la tension issue du pont diviseur de la batterie avec une tension de référence générée par un autre pont diviseur alimenté par une source régulée à 5 V.

La sortie du comparateur est reliée à une entrée GPIO de l'Arduino :

- niveau logique haut si la tension de la batterie est supérieure à 6,7 V,
- niveau logique bas si la tension de la batterie est inférieure à 6,7 V.

L'intérêt d'utiliser un comparateur est d'éviter de solliciter l'ADC de l'Arduino pour cette tâche, ce qui permet de gagner du temps de traitement et d'alléger le programme.

Le fait d'avoir également relié la sortie du pont diviseur à un port ADC de l'Arduino permet, pendant la phase de test, de connaître l'état de la batterie sans utiliser de voltmètre mais aussi d'ajuster les mouvements du robot liés au moteur, en fonction de l'état de la batterie.

- **EXIG_AUTONOMIE** : Afin de répondre à cette exigence, nous devons obtenir une autonomie d'au moins 15 minutes en match et 65 minutes sans adversaire.

Robot Mini-Sumo (RMS)

La technologie retenue est une batterie LiPo 2S, dont la tension varie entre 8,4 V et 6,7 V. Cette plage est compatible avec les moteurs, le module Pololu et le régulateur linéaire utilisés.

Dans le bloc énergie, pour assurer le bon fonctionnement de la carte, nous avons ajouté un régulateur linéaire fournissant une tension régulée de 5 V destinée à alimenter l'Arduino, les capteurs et les autres composants nécessitant cette tension. Le régulateur intégré à l'Arduino étant légèrement sous-dimensionné, il a été jugé préférable d'utiliser un régulateur linéaire externe.

Nous avons également ajouté une diode de protection afin d'éviter toute détérioration de la carte en cas d'inversion de polarité de la batterie.

Pour le bloc "Traitement" le cahier des charges nous impose une exigences :

- **EXIG_COMPORTEMENT** : Pour répondre à cette exigence, qui stipule que le robot doit être capable de changer de direction de façon autonome en utilisant les données des capteurs et en agissant sur les actionneurs. Nous avons choisi une carte Arduino Uno équipée d'un ATMEGA328P qui permet de lire les infos des capteurs et de choisir les comportement à effectuer afin d'activer ou non les actionneurs.

Pour le bloc "Action" le cahier des charges nous impose une exigences :

- **EXIG_DEPLACEMENT** : Pour répondre à cette exigence un pont en H est utilisé permettant ainsi le choix du sens des deux moteurs ainsi que leur vitesse via deux broches GPIO et deux broches PWM.

Nous avons également décidé de rajouter deux composants, à savoir une LED permettant de savoir si le robot sumo est sous tension ainsi qu'un buzzer qui produit un son lorsque le robot est en mode attente, c'est-à-dire lorsque le bouton "Attention" de la télécommande est pressé.

2.1.1 Choix des capteurs adversaire avant

Référence de pré-conception : CPR02

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

Exigences client vérifiées par préconception : **EXIG_ADVERSAIRE**

Le robot mini-sumo doit être capable d'identifier un adversaire situé à une distance d'au moins 40 cm devant lui.

Pour satisfaire l'exigence **EXIG_ADVERSAIRE**, nous avons commencé par réaliser une analyse comparative des capteurs de mesure disponibles, synthétisée dans un tableau d'aide à la décision pour sélectionner les capteurs les plus adaptés.

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Distance min et max de détection	Facilité d'utilisation du signal de sortie	Tension avec les sources disponibles	Faible consommation en courant	Faible PTH	Disponibilité	Total (produit des valeurs précédentes)
Capteur de mesure Sharp GP2Y0A41SK0F	0	4cm/30cm	Analog	4,5 to 5,5V	typ (12mA) max(22mA)	9,9	/	
		4	4	5	5	3,5	5	0
Capteur de mesure Sharp GP2Y0A021YK	1	10 cm / 80 cm	Analog	4,5 to 5,5V	30mA	11,5	/	
		3,5	4	5	4	3	5	4200
Capteur de mesure Sharp GP2Y0A02YK	1	20cm / 150cm	Analog	4,5 to 5,5V	33mA	12,8	/	
		2	4	5	3	3	5	1800
Capteur de mesure Ultrasons HCSR04	1	2cm / 4m	Num	5V	15 mA	3,9	/	15625
		5	5	5	5	5	5	
IUT de Bordeaux Département GEII		Référence : SUMO_FAD Révision : 1.6 – 19/09/2022						

Figure 2 : FAD pour le choix des capteurs adversaire avant

Ce tableau évalue les capteurs selon leur portée, la facilité de traitement du signal de sortie, leur coût, ainsi que d'autres critères importants.

Après analyse, nous avons retenu le capteur à ultrasons HC-SR04 comme solution principale. Ce capteur fonctionne à une tension nominale de 5 V, compatible avec notre régulateur linéaire, et présente le coût le plus bas à 3,90 € l'unité. La sortie numérique du capteur HC-SR04 facilite le traitement du signal comparée à une sortie analogique, car elle ne requiert pas de conversion analogique-numérique (ADC), supprimant donc une étape de calcul supplémentaire.

Enfin, avec une portée de détection de 2 cm à 4 m, le HC-SR04 est idéal pour notre application : il détecte un adversaire à 40 cm ou plus, jusqu'à l'extrémité du dohyo, tout en étant précis à courte

distance (2 cm) lorsque l'adversaire est proche, offrant une grande polyvalence. Ainsi, deux capteurs HC-SR04 ont été intégrés à l'avant du robot pour une détection frontale fiable.

Pour valider le bon fonctionnement du capteur dans notre application, nous avons effectué des tests préliminaires visant à vérifier sa portée minimale et maximale, ainsi que l'angle maximal de détection de l'adversaire.

Nous avons ainsi rédigé un code Arduino simple, permettant de tester la détection dans plusieurs cas de figure.

```
// Definition des numeros de port
const int trigPin = 4; // Trigger (émission)
const int echoPin = 2; // Echo (réception)
// Variables utiles
long duree; // durée de l'écho
int distance; // distance

void setup() {
  pinMode(trigPin, OUTPUT); // Configuration du port du Trigger comme une SORTIE
  pinMode(echoPin, INPUT); // Configuration du port de l'Echo comme une ENTREE
  Serial.begin(9600); // Démarrage de la communication série
}

void loop() {
  // Émission d'un signal de durée 10 microsecondes
  digitalWrite(trigPin, LOW);
  delayMicroseconds(5);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Écoute de l'écho
  duree = pulseIn(echoPin, HIGH);

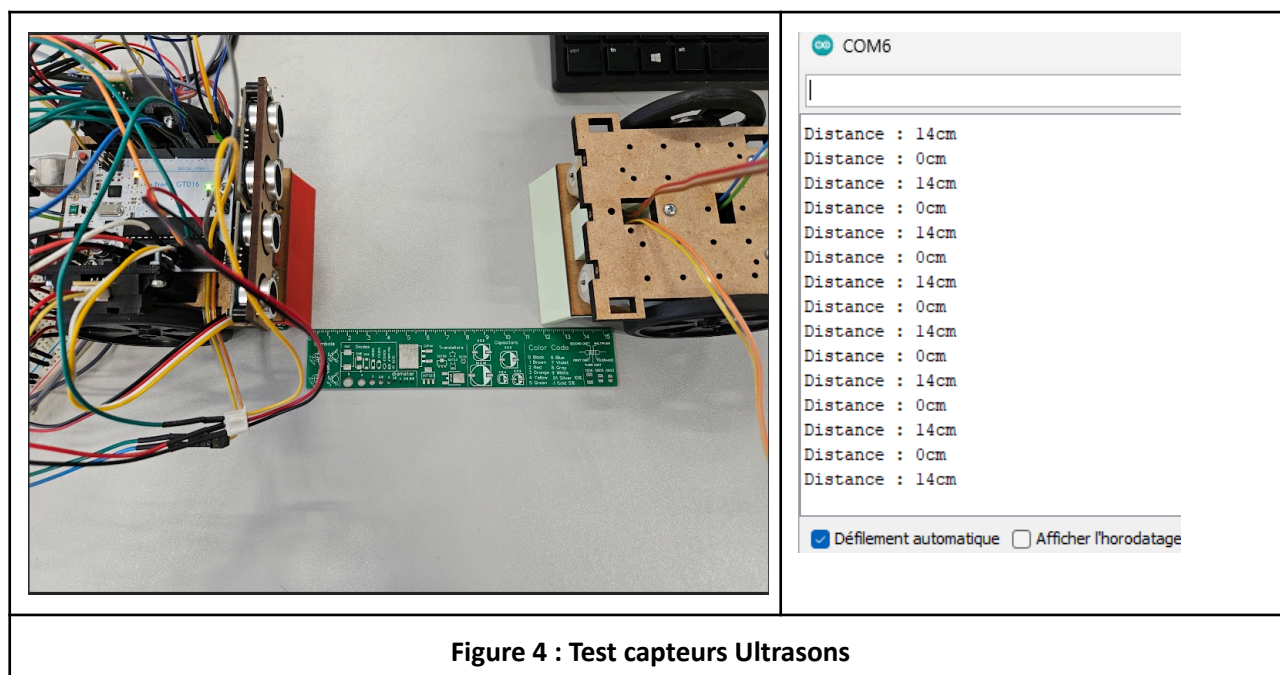
  // Calcul de la distance
  distance = duree*0.034/2;

  // Affichage de la distance dans le Moniteur Série
  Serial.print("Distance : ");
  Serial.print(distance);
  Serial.println("cm");
}
```

Figure 3 : Code Arduino utilisé pour les test préliminaires

Robot Mini-Sumo (RMS)

Ainsi on chargeant le code dans le robot sumo, placé à 14cm d'un robot sumo adverse nous obtenons les résultats suivants :



Nous avons également effectué d'autres tests avec le robot à des distances variant de 2 cm à 60 cm. Les résultats se sont révélés concluants, sauf pour les distances inférieures à 3 cm. Toutefois, pour notre application, ce type de capteur est parfaitement adapté.

2.1.2 Choix des capteurs adverse latéraux

Référence de pré-conception : CPR03

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

Exigences client vérifiées par préconception : EXIG_ADVERSAIRE

Pour répondre à l'exigence **EXIG_ADVERSAIRE**, qui demande surtout au robot mini-sumo de repérer et de réagir à un adversaire devant lui, nous avons décidé d'aller plus loin en ajoutant deux capteurs télémétriques sur les côtés gauche et droit du robot. Même si ce n'est pas obligatoire, ces capteurs permettent de détecter les adversaires sur les flancs, rendant le robot plus réactif et polyvalent sur le dohyo. Pour choisir les bons capteurs, nous nous sommes appuyés sur une fiche d'aide à la décision ci-dessous.

Robot Mini-Sumo (RMS)

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	(valeur de 1 à 5)						Total (produit des valeurs précédentes)
		Distance min et max de détection	Facilité d'utilisation du signal de sortie	Compatibilité de la tension avec les sources disponibles	Faible consommation en courant	Faible Prix	Disponibilité	
Capteur de mesure Sharp GP2Y0A41SK0F	1	4cm/30cm	Analog	4,5 to 5,5V	typ (12mA) max(22mA)	9,9	/	625
		5	5	5	5	5	/	
Capteur de mesure Sharp GP2Y0A021YK	1	10 cm / 80 cm	Analog	4,5 to 5,5V	30mA	11,5	/	240
		4	5	5	3	4	/	
Capteur de mesure Sharp GP2Y0A02YK	1	20cm / 150cm	Analog	4,5 to 5,5V	33mA	12,8	/	90
		3	5	5	2	3	/	
IUT de Bordeaux	Référence : SUMO_FAD							
Département GEII	Révision : 1.6 – 19/09/2022							

Figure 5 : FAD pour le choix des capteurs latéraux

Après une analyse comparative des capteurs disponibles, nous avons retenu le capteur télémétrique Sharp GP2Y0A41SK0F pour les détections latérales. Ce choix s'explique par son coût attractif de 9,90 € et sa faible consommation énergétique comparée aux autres options. Fonctionnant à une tension nominale de 5 V, il s'intègre parfaitement à notre étage d'acquisition alimenté par un régulateur 5 V. Sa plage de détection, de 4 cm à 30 cm, est également adaptée à notre besoin, notamment au démarrage du match où les robots sont positionnés sur les côtés. Ainsi, ce capteur permet de repérer un adversaire à courte distance (4cm) tout en offrant une portée suffisante pour détecter les mouvements latéraux durant le combat.

2.1.3 Choix des capteurs de sol

Référence de pré-conception : CPR04

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

Exigences client vérifiées par préconception : EXIG_LUMINOSITE

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	13/ 134
----------------------------------	---	------------

Robot Mini-Sumo (RMS)

Pour satisfaire l'exigence **EXIG_LUMINOSITE**, nous avons choisi un capteur de contraste capable de détecter le franchissement d'une ligne blanche, ce qui signifie sortir du Doyō sans être perturbé par des variations brusques de luminosité, comme celles causées par des flashes d'appareils photo. Après plusieurs tests réalisés à l'aide de la lumière d'un téléphone, nous avons pu confirmer que cette exigence était bien respectée.

A	B	C	D	E	F	G	H	I
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Diamètre min et max de détection	Adapté de la tension avec les sources disponibles	Respect de la stratégie de combat	Faible Prix	Disponibilité	Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
0 CNY avant + 0 CNY arrière	1	/	oui	/	0	/	0	
		0	5	0	5	5		
1 CNY avant + 0 CNY arrière	1	0-5mm	oui	/	4,97	/		
		5	5	1	4	5	500	
2 CNY avant + 0 CNY arrière	1	0-5mm	oui	/	9,94	/		
		5	5	2	3	5	750	
0 CNY avant + 1 CNY arrière	1	0-5mm	oui	/	4,97	/		
		5	5	0,5	4	5	250	
1 CNY avant + 1 CNY arrière	1	0-5mm	oui	/	9,94	/		
		5	5	3	3,5	5	1312,5	
2 CNY avant + 1 CNY arrière	1	0-5mm	oui	/	14,91	/		
		5	5	4	3	5	1500	
2 CNY avant + 2 CNY arrière	1	0-5mm	oui	/	19,88	/		
		5	5	5	2	5	1250	
IUT de Bordeaux	Référence : SUMO_FAD							
Département GEII	Révision : 1.6 – 19/09/2022							4 / 7

Figure 6 : FAD pour le choix des capteurs de contraste

Bien que le choix des capteurs ne réponde pas à une exigence spécifique, il est crucial pour maximiser les chances de victoire dans les combats de robots sumos. Nous avons donc sélectionné deux capteurs CNY à l'avant et un capteur CNY à l'arrière. Cette configuration stratégique vise à minimiser les angles morts et à empêcher le robot de sortir du Doyō, tout en restant économique avec un coût total de 14,91 €. De plus, la distance de détection des capteurs CNY (0 à 5 mm) est idéale pour leur positionnement sous le robot, très proche du sol, assurant ainsi une efficacité optimale.

Une alternative consistant à placer deux capteurs à l'avant et deux à l'arrière aurait offert une détection plus précise des risques de sortie, mais elle aurait significativement augmenté les coûts. Cette configuration plus complexe avec davantage de capteurs aurait nécessité un programme plus

élaboré, augmentant la complexité du code et compromettant l'optimisation de notre solution. Le choix actuel équilibre donc performance, simplicité et budget.

Pour confirmer le bon fonctionnement des capteurs dans notre système, nous avons réalisé des tests préliminaires afin de valider leur capacité à détecter correctement par rapport à la distance au sol et les changements de couleur (noir/blanc).

À cet effet, nous avons développé un programme Arduino simple, conçu pour évaluer la détection dans différents scénarios.

```
int valeurDeReference = 600; // seuil pour noir et blanc

void setup() {
  Serial.begin(115000);
}

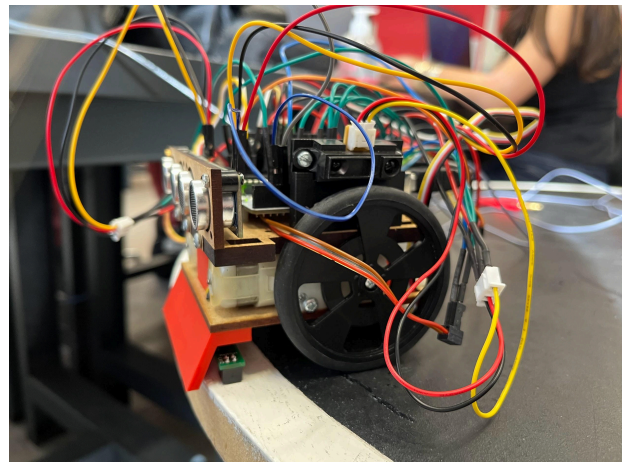
void loop() {
  int valeurDuCapteurG = analogRead(A0);
  int valeurDuCapteurD = analogRead(A1);
  // mesure de la tension sur la broche A2 de l'Arduino
  if (valeurDuCapteurG > valeurDeReference) {
    Serial.println("Surface : Noir");
  } else {
    Serial.println("Surface : Blanc");
  }

  delay(1); // Une courte pause pour un fonctionnement fluide.
}
```

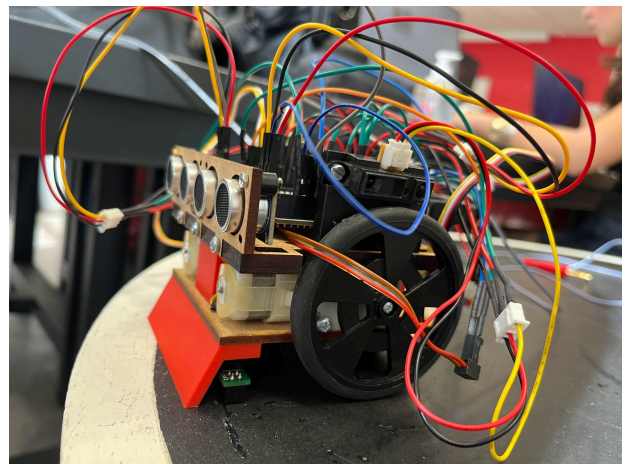
Figure 7 : Code Arduino utilisé pour les test préliminaires

Robot Mini-Sumo (RMS)

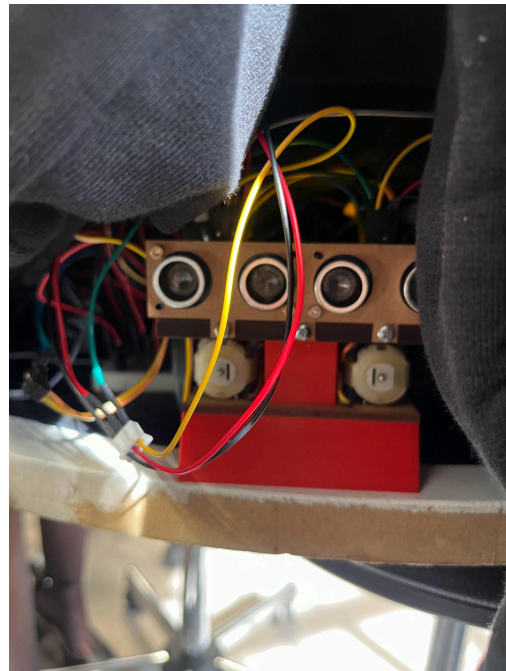
Surface : Blanc

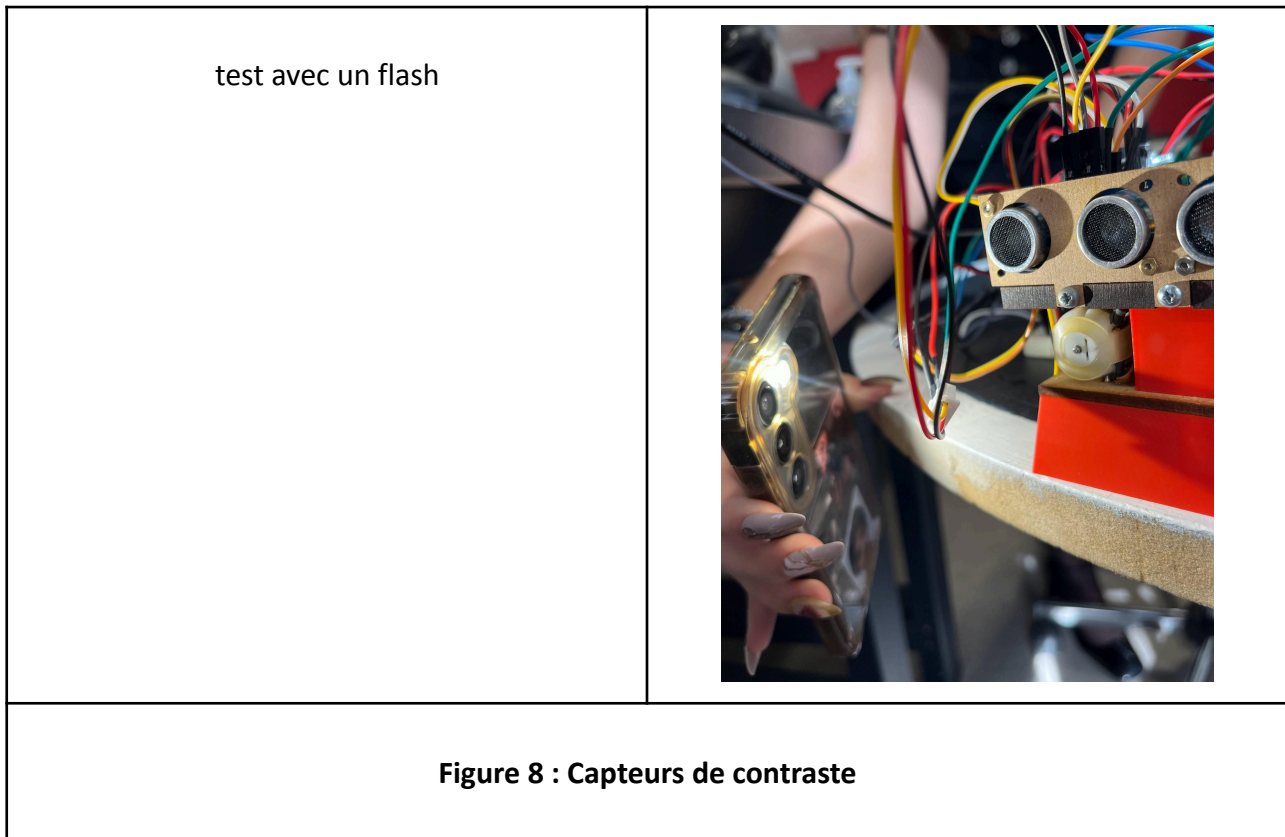


Surface : Noir



test dans le noir





2.1.4 Choix du capteur infrarouge

Référence de pré-conception : CPR05

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

Exigences client vérifiées par préconception : **EXIG_DEPART**

Pour satisfaire l'exigence **EXIG_DEPART**, nous avons opté pour un capteur infrarouge capable de recevoir les signaux émis par la télécommande de l'arbitre. Ce capteur doit détecter les commandes "Attention", "Start" et "Stop", qui autorisent respectivement la préparation, le début et l'arrêt du combat. Ces signaux sont essentiels pour permettre au robot d'initier ses actions en toute conformité, sans quoi il n'est pas autorisé à engager l'adversaire.

Robot Mini-Sumo (RMS)

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)						Total des valeurs	Commentaires (si nécessaire)
		Distance min et max de détector	Facilité d'utilisation du signal de sortie	Compatibilité de la tension avec les sources disponibles	Faible consommation en courant	Faible Prix	Disponibilité		
TSOP 18338	1	24 m	Analog	2.5 V to 5.5 V	0,70 mA (typ) 0,90 mA (max)	0,71	oui	15625	
IR ST027	1	20 cm to 150 cm	Analog	5V	?	1,46 to 4,10	oui	0	Guide de mise en marche et exemple de programme fournis
		5	5	5	0	2	5		
IUT de Bordeaux Département GEII		Référence : SUMO_FAD Révision : 1.6 – 19/09/2022							3 / 7

Figure 9 : FAD pour le choix du capteur infrarouge

Nous avons sélectionné le capteur TSOP 18338 pour plusieurs raisons. Tout d'abord, ce composant est bien documenté, et ses caractéristiques techniques répondent à nos besoins attendus. Ensuite, il est nettement plus économique que l'autre solution envisagée. Bien que l'autre capteur infrarouge proposé soit accompagné d'un guide de mise en œuvre et d'un exemple de programme, notre expérience antérieure avec le TSOP 18338, utilisé dans un projet précédent, nous apporte une maîtrise de son branchement et de sa programmation. Nous disposons déjà d'un programme fonctionnel, ce qui réduit les risques et accélère le développement.

2.1.5 Choix du pont en H

Référence de pré-conception : CPR06

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT

L'exigence **EXIG_DEPLACEMENT** stipule que le robot mini-sumo est libre dans ses déplacements. Ainsi, il est capable d'avancer, de tourner à gauche ou à droite.

Afin de répondre à cette exigence nous devons pouvoir contrôler le sens et la vitesse du moteur, pour cela nous utiliserons un Pont en H. Deux types de Pont en H sont disponibles dans le stock :

le Polulu DRV8835 et le Polulu DRV8833.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	18/ 134
----------------------------------	---	------------

Robot Mini-Sumo (RMS)

Afin de déterminer lequel des deux est le plus adéquate à notre utilisation nous avons utilisé une fiche d'aide à la décision que nous retrouvons à la figure 10,

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	de la tension avec les sources disponibles	Courant max de sortie du pont	Manière de piloter le pont	Faible Prix	Disponibilité	Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
Pololu Commande de 2 moteurs CC DRV8835 2x1,2A	1	0- 11 et 2 - 7	1,2 (1,5peak)	PWM/GPIO	4,49	Oui	3125	
		5	5	5	5	5		
Pololu Commande de 2 moteurs CC DRV8833 2x1,2A	1	2,7 - 10,8	1,2 (2peak)	PWM	5,4	Oui	2000	
		5	5	4	4	5		
IUT de Bordeaux Département GEII	Référence : PRJ_FAD Révision : 1.6 – 19/09/2022							5 / 7

Figure 10 : FAD pour le choix du pont en H

Le DRV8835 nécessite deux sources d'alimentation distinctes : une pour les commandes des moteurs et une pour la logique, avec des plages de tension respectives de 0 à 11 V et de 2 à 7 V. Il peut fournir un courant maximum en crête de 1,5 A. Ce module peut être contrôlé via un GPIO pour déterminer le sens de rotation du moteur et un signal PWM pour réguler la vitesse. Alternativement, il est possible de piloter le DRV8835 uniquement avec un signal PWM pour gérer à la fois le sens et la vitesse. Son faible encombrement constitue un avantage significatif, notamment en raison de la taille imposée par le shield. De plus, le prix du DRV8835 de Pololu est inférieur à celui du DRV8833.

Afin de bien choisir le pont en H nous avons fait un test de consommation de courant sur un des deux moteurs du robot sumo mis à disposition, nous avons réalisé le montage de la figure X sur breadboard.

Robot Mini-Sumo (RMS)

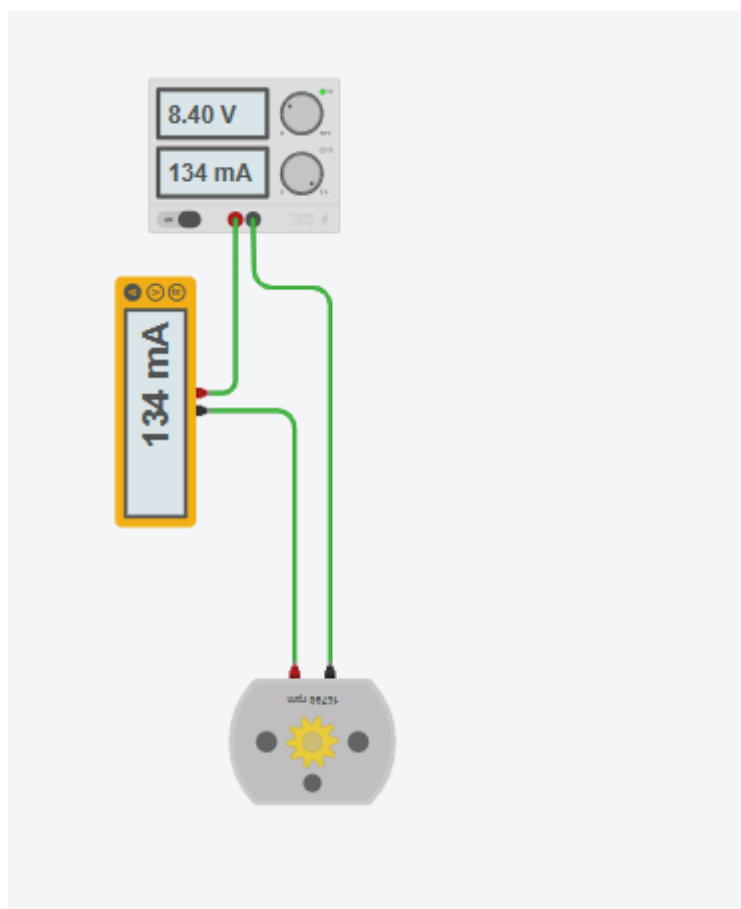


Figure 11 : Schéma électrique du test de courant

(les valeurs sur la figure X ne sont pas à prendre en compte en raison de la nature de la simulation, le moteurs ici présent n'est pas celui utilisés lors du projet)

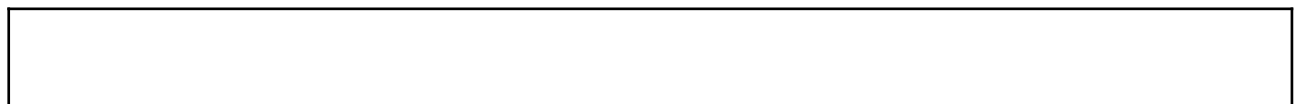
A la tension V_{cc} de 8.4V le moteur consomme en peak environ 470 mA (cf figure X) ce qui est acceptable sachant que le courant supportés par un moteur un courant de 1,5A en peak.

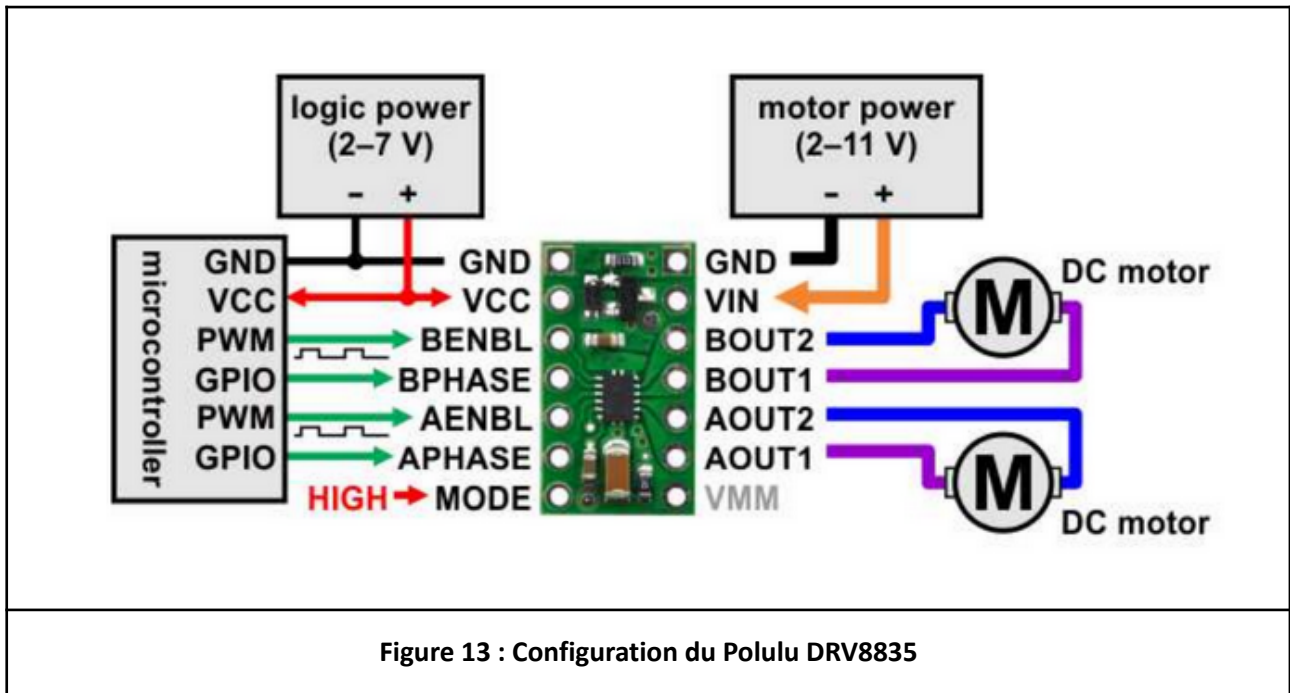


Figure 12 : Résultats du test de courant

En tenant compte des caractéristiques des ponts en H, le DRV8835 représente le choix le plus avantageux pour optimiser le temps et le coût. La principale différence notable entre le DRV8835 et le DRV8833 réside dans le courant maximum en crête, respectivement de 1,5 A et 2 A. Cependant, le courant d'un moteur, à la tension maximale d'un accumulateur LiPo 2S (soit 8,4 V), s'élève à environ 470 mA, ce qui reste dans la plage de courant acceptable pour le DRV8835.

Le pont en H nécessite donc 2 pin I/O GPIO et 2 pin I/O PWM selon la configuration de la figure 11.





Afin de valider le bon fonctionnement du pont en H nous avons monté sur une breadboard le pont en H relié à une carte arduino ainsi qu'aux deux moteurs le tout alimenté par une alimentation de table à plusieurs tensions différentes entre 6,7V et 8,4V afin de simuler la perte de puissance de l'accumulateur, puis nous avons écrit un bout de code (cf figure 12) permettant de faire avancer, tourner et reculer le robot.

```

int MOTA = 9;
int MOTB = 10;
int PWMA = 11;
int PWMB = 6;
int HIGH_MODE = 12;

void setup() {
  pinMode(MOTA, OUTPUT);
  pinMode(MOTB, OUTPUT);
  pinMode(PWMA, OUTPUT);
  pinMode(PWMB, OUTPUT);
  pinMode(HIGH_MODE, OUTPUT);
  digitalWrite(HIGH_MODE, 1);

  //Moteur dans le sens horraire
  digitalWrite(MOTA, 1);
  digitalWrite(MOTB, 1);

  //On avance avec une vitesse à 100%
  analogWrite(PWMA, 255);
  analogWrite(PWMB, 255);
  delay(1000);
  //On tourne en avançant
  analogWrite(PWMA, 255);
  analogWrite(PWMB, 0);
  delay(500);
  //On tourne en avançant mais dans l'autre sens
  analogWrite(PWMA, 0);
  analogWrite(PWMB, 255);
  delay(1000);
  //On change le sens des moteurs
  digitalWrite(MOTA, 0);
  digitalWrite(MOTB, 0);
  //On recule avec une vitesse à 100%
  analogWrite(PWMA, 255);
  analogWrite(PWMB, 255);

}

```

Figure 14 : Programme de test du pont en H

Robot Mini-Sumo (RMS)

Après avoir compilé et transféré le code vers la carte arduino nous avons pu confirmer le bon fonctionnement du Pont en H dans la mesure où grâce aux codes, le robot sumo a pu avancer, tourner ainsi que reculer.

2.1.6 Choix du régulateur de tension

Référence de pré-conception : CPR07

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

Exigences client vérifiées par préconception : EXIG_AUTONOMIE EXIG_SECUR_BATT

FICHE D'AIDE A LA DECISION (FAD)									
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)							Total (produit des valeurs précédentes)
		Tension max d'entrée	Tension de sortie	Faible chute de tension sous-charge	Courant de sortie max	Batterie composant facile à utiliser/charger	Faible PTH	Disponibilité	
LP2985AIM5-3.3V/NCP	1	16 V	3.3 V	0.3 V @150 mA	150 mA	SOT-23 CMS (soudable)	Moyen	Oui	4500
LP2985AIM5-5.0V/NCP	1	16 V	5.0 V	0.3 V @150 mA	150 mA	SOT-23 CMS	Moyen	Oui	16875
MC33269DT-3.3G IC	1	20 V	3.3 V	1.0 V @800 mA	800 mA	OPRK CMS plus grand, facile à souder	Faible	Oui	46875
MC33269DT-5.0G IC	1	20 V	5.0 V	1.0 V @800 mA	800 mA	OPRK CMS	Faible	Oui	28125
IUT de Bordeaux Département GEII									6 / 7

Figure 15 : FAD pour le choix régulateur tension

Bien que plusieurs régulateurs linéaires répondent aux exigences de base, le MC33269DT-5.0G s'est révélé être le choix le plus adapté pour alimenter l'Arduino et les capteurs en 5 V. En effet, il offre un courant de sortie maximal de 800 mA, garantissant une alimentation stable même lorsque plusieurs modules sont actifs simultanément. Les régulateurs LP2985AIM5-3.3V et LP2985AIM5-5.0V ont été écartés en raison de leur courant limité à 150 mA, insuffisant pour notre application. Le modèle MC33269DT-3.3G, quant à lui, fournit une tension de 3,3 V, inadaptée au fonctionnement de l'Arduino.

Ainsi, le choix du MC33269DT-5.0G permet de concilier fiabilité, simplicité d'intégration et coût réduit, tout en assurant une tension stable et conforme aux besoins du système d'alimentation.

2.1.7 Choix de la batterie

Référence de pré-conception : CPR08

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	24/ 134
----------------------------------	---	------------

Exigences client vérifiées par préconception : EXIG_AUTONOMIE EXIG_SECUR_BATT

FICHE D'AIDE A LA DECISION (FAD)						
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)			Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Encombrement	Faible Prix	Disponibilité		
Lipo CONRAD ENERGY 7.4 V 1200 mAh	0	72*36mm/75g	20,99	/		
		3	3	5	0	
Lipo CONRAD ENERGY 7.4 V 800 mAh	1	56*31mm / 52g	12,99	/		
		5	4	5	100	
Lipo CONRAD ENERGY 7.4 V 450 mAh	1	56*31mm / 31g	8,29	/		
		5	5	5	125	
IUT de Bordeaux	Référence : SUMO_FAD					
Département GEII	Révision : 1.6 – 19/09/2022					7 / 7

Figure 16 : FAD pour le choix du batterie

Parmi les trois batteries disponibles, les paramètres pris en compte sont le poids, le prix, la capacité et les dimensions.

Nous pouvons éliminer l'une d'entre elles, la batterie LiPo Conrad Energy 7,4 V – 1200 mAh, car ses dimensions sont trop grandes pour entrer dans le compartiment prévu sous le robot.

Le choix se porte donc sur les deux batteries restantes : 800 mAh et 450 mAh.

Il faut maintenant vérifier laquelle permet de respecter l'exigence d'autonomie du robot.

Nous avons réalisé un montage sur une planche de prototypage (breadboard) comprenant plusieurs composants du robot :

- 2 capteurs à ultrasons,
- 2 capteurs télémètres,
- 3 capteurs de ligne,
- une carte Arduino Uno,
- un module Pololu,
- 2 moteurs GM3.

À l'aide d'un ampèremètre, nous avons mesuré un courant moyen d'environ 600 mA et un courant de pointe de 1,3 A.

Ces mesures nous permettent d'estimer la consommation totale du robot et de déterminer la capacité minimale de la batterie nécessaire pour garantir l'autonomie attendue.

Nous avons mesuré le courant moyen consommé par le robot avec tous ses capteurs et moteurs : environ 600 mA. En utilisant cette valeur, nous pouvons estimer l'autonomie des batteries restantes. La batterie de 800 mAh permettrait de faire fonctionner le robot pendant environ 1 h 20 min, tandis que la batterie de 450

Robot Mini-Sumo (RMS)

mAh ne tiendrait qu'environ 45 minutes. Ainsi, pour respecter l'autonomie souhaitée, la batterie de 800 mAh est la meilleure option.

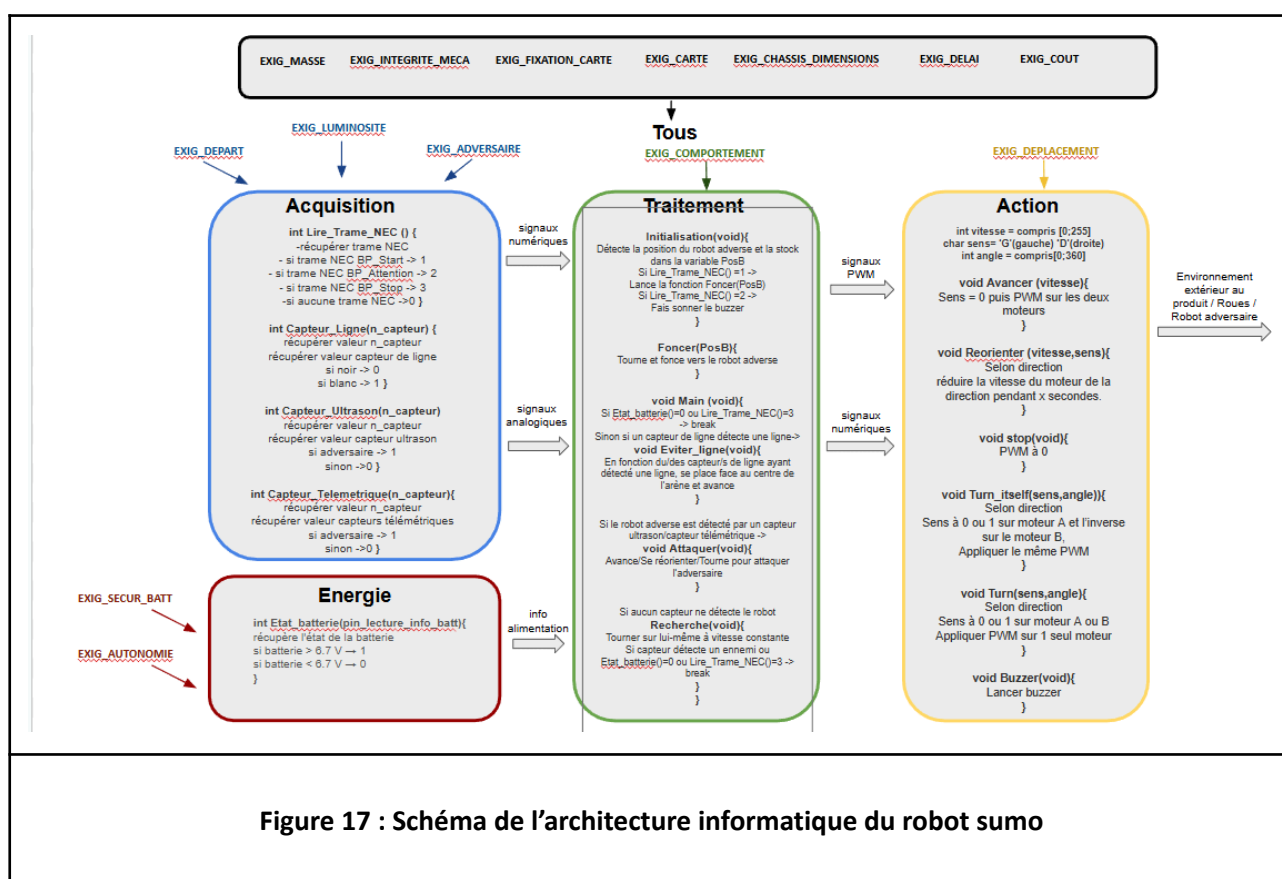
2.2 Architecture Informatique

Référence de pré-conception : CPR09

Compétences GEII : C1a3 C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

Exigences client vérifiées par préconception : EXIG_COMPORTEMENT EXIG_AUTONOMIE
EXIG_SECUR_BATT EXIG_DEPLACEMENT EXIG_ADVERSAIRE EXIG_LUMINOSITE EXIG_DEPART
EXIG_CARTE EXIG_COUT EXIG_CHASSIS_DIMENSIONS EXIG_MASSE EXIG_INTEGRITE_MECA
EXIG_FIXATION_CARTE

Nous avons réalisé sous la forme d'un schéma synoptique l'architecture informatique, nécessaire au bon fonctionnement du robot sumo.



2.2.1 Fonctions d'acquisition

Référence de pré-conception : CPR10

Compétences GEII : C1a4 C1a9 C1a10 C1a11 C1d5 C1d7 C1d8

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	27/ 134
----------------------------------	---	------------

Exigences client vérifiées par préconception : EXIG_ADVERSAIRE EXIG_LUMINOSITE EXIG_DEPART

Quatre fonctions ont été implémentées pour assurer l'acquisition des données :

Pour répondre à l'exigence **EXIG_DEPART** qui stipule que le robot doit rester immobile en attendant le départ. Le robot mini-sumo démarre dès que le juge appuie sur une touche de la télécommande infrarouge, nous avons implémenté la fonction :

- **lire_Trame_NEC()** : Cette fonction lit la trame NEC du capteur infrarouge , identifiant trois trames distinctes pour les commandes "Start", "Attention" et "Stop" de la télécommande, permettant au robot de répondre aux signaux de l'arbitre. Ainsi selon la trame détectée, elle retourne
 - 0 : aucune trame NEC détecté / mauvaise trame NEC
 - 1 : la trame NEC du bouton "Start" à été détecté
 - 2 : la trame NEC du bouton "Attention" à été détecté
 - 3 : la trame NEC du bouton "Stop" à été détecté

Nous avons pris en compte l'exigence **EXIG_LUMINOSITE** qui stipule que l'éclairage ambiant pourra être élevé sans que cela soit une obligation, pour la fonction suivante

- **Capteurs_Ligne(n_capteur)** : La fonction récupère la valeur du capteur de ligne spécifié par n_capteur et retourne 0 si la couleur détectée est noire, ou 1 si elle est blanche.

Pour répondre à l'exigence **EXIG_ADVERSAIRE** qui stipule que le robot mini-sumo est capable de savoir si son adversaire est devant lui à une distance de 40 cm minimum nous avons implémenté les fonctions :

- **Capteurs_Ultrasons(n_capteur)** : La fonction capteurs_Ultrasons(n_capteur) récupère la valeur du capteur à ultrasons spécifié par n_capteur et retourne 1 si un adversaire est détecté, ou 0 sinon.
- **Capteurs_Telemetriques(n_capteur)** : La fonction récupère la valeur du capteur à télémétrique spécifié par n_capteur et retourne 1 si un adversaire est détecté, ou 0 sinon.

2.2.2 Fonctions d'action

Référence de pré-conception : CPR11

Compétences GEII : C1a4 C1a9 C1a10 C1a11

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT

L'exigence **EXIG_DEPLACEMENT** stipule que le robot mini-sumo est libre dans ses déplacements. Ainsi, il est capable d'avancer, de tourner à gauche ou à droite.

Afin de répondre à cette exigence nous devons créer des fonctions capables d'exécuter les différents mouvements liés au déplacement.

Pour cela nous avons regroupé toutes les fonctions dans le bloc ACTION de l'architecture fonctionnelle informatique de la figure 14.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	28/ 134
----------------------------------	---	------------

Robot Mini-Sumo (RMS)

Après réflexion et développement du comportement du robot 5 fonctions essentielles ont été élaborées afin de permettre au robot d'être libre de ses mouvements.

La fonction "Avancer" (cf Figure 15) permet au robot, comme est stipulé dans son nom, d'avancer avec une vitesse X comprise entre 0 et 255.

```
void Avancer (vitesse){  
  Sens = 0 puis PWM sur les deux  
  moteurs  
}
```

Figure 18 : Fonction "Avancer" du Robot Sumo

Pour permettre cela, nous envoyons aux broches liées au GPIO du moteur gauche et du moteur droit un signal LOW, permettant ainsi la rotation horaire des deux moteurs à la même vitesse.

La fonction "Stop" (cf Figure 16) permet au robot, comme est stipulé dans son nom, de se stopper.

```
void stop(void){  
  PWM à 0  
}
```

Figure 19 : Fonction "stop" du Robot Sumo

Pour permettre cela, nous envoyons aux broches liées au PWM du moteur gauche et du moteur droit un signal nul (égale à 0) permettant le blocage du moteur et des roues.

La fonction "Turn_itself" (cf Figure 17) permet au robot de tourner sur lui-même sans avancer ou reculer.

```
void Turn_itself(direction,angle){  
  Selon direction  
  Sens à 0 ou 1 sur moteur A et l'inverse  
  sur le moteur B,  
  Appliquer le même PWM  
}
```

Figure 20 : Fonction "Turn_itself" du Robot Sumo

Pour permettre cela, nous envoyons au GPIO du moteur lié à la direction, un signal LOW, et au moteur lié à l'autre direction, un signal LOW. Puis la vitesse est calculée par rapport à l'angle spécifié dans les paramètres.

La fonction “Turn_itself” (cf Figure 18) permet au robot de tourner sur lui-même sans avancer ou reculer.

```
void Turn(direction,angle){
    Selon direction
    Sens à 0 ou 1 sur moteur A ou B
    Appliquer PWM sur 1 seul moteur
}
```

Figure 21 : Fonction “Turn” du Robot Sumo

Pour permettre cela, nous envoyons au GPIO du moteur du côté opposé à au paramètre direction, un signal LOW ainsi qu’un PWM calculé par rapport à l’angle fourni en paramètre.

La fonction “Turn” (cf Figure 19) permet au robot de tourner en avançant ou reculant à droite ou à gauche.

```
void Reorienter (vitesse,direction){
    Selon direction
    réduire la vitesse du moteur de la
    direction pendant x secondes.
}
```

Figure 22 : Fonction “Reorienter” du Robot Sumo

Pour permettre cela en fonction de la direction et de la vitesse, un signal PWM inférieur à la valeur de base est appliqué au moteur du côté concerné, pendant un court instant, jusqu’à ce que les deux capteurs détectent une distance similaire, comprise entre 2 et 3 cm.

2.2.3 Fonctions d’énergie

Référence de pré-conception : CPR12

Compétences GEII : C1a4 C1a9 C1a10 C1a11

Exigences client vérifiées par préconception : EXIG_AUTONOMIE EXIG_SECUR_BATT

La fonction “Etat_Batterie” (cf Figure 20) permet au robot récupérer une information sur l’état de la batterie.

```
int Etat_batterie(pin_lecture_info_batt){
  récupère l'état de la batterie
  si batterie > 6.7 V → 1
  si batterie < 6.7 V → 0
}
```

Figure 23 : Fonction "Etat_Batterie" du Robot Sumo

Pour cela, l'état logique du comparateur relié à une broche GPIO de la carte Arduino Uno est lu, ce qui permet de savoir si la tension au borne de la batterie est supérieure ou inférieure à 6,7V. Dans ce cas, la partie traitement s'occupera de couper les moteurs.

Un second fonction qui lira la valeur analogique en sortie d'un pont diviseur de tension grâce à un ADC de l'Arduino pourra être envisagé pour faire de debug ou comme solution de repli si la première ne fonctionnerait pas.

2.2.4 Fonctions de traitement

Référence de pré-conception : CPR13

Compétences GEII : C1a4 C1a9 C1a10 C1a11

Exigences client vérifiées par préconception : EXIG_COMPORTEMENT

Afin de répondre du mieux que possible à cette exigence, nous avons réalisé un grafcet permettant de visualiser le fonctionnement du robot sumo.

Lien vers Grafcet :

https://drive.google.com/file/d/1kCRh7_fNJzgpExwtmyAiocMVKV7_gBsO/view?usp=sharing

Robot Mini-Sumo (RMS)

DetG	Détection d'un robot à gauche
DetD	Détection d'un robot à droite
PosB	Position du robot (Avant, Arrière, Gauche, Droite)
TramStart	Détection de la trame NEC du bouton start
TramAtt	Détection de la trame NEC du bouton attention
TramStop	Détection de la trame NEC du bouton stop
Etat_Batterie	L'état de la batterie est inférieur à 6,7V
DetAvG	Détéction d'un robot par le capteur avant gauche
DetAvD	Détéction d'un robot par le capteur avant droit
StopxBat	L'état de la batterie est inférieur à 6,7V ou la trame Nec du bouton stop est détecté

Figure 24 : Abréviation sur le Grafcet

Afin de ne pas être surchargé, ce grafcet est séparé en plusieurs parties/fonctions et chacune d'entre elles est expliquées ci-dessous.

Robot Mini-Sumo (RMS)

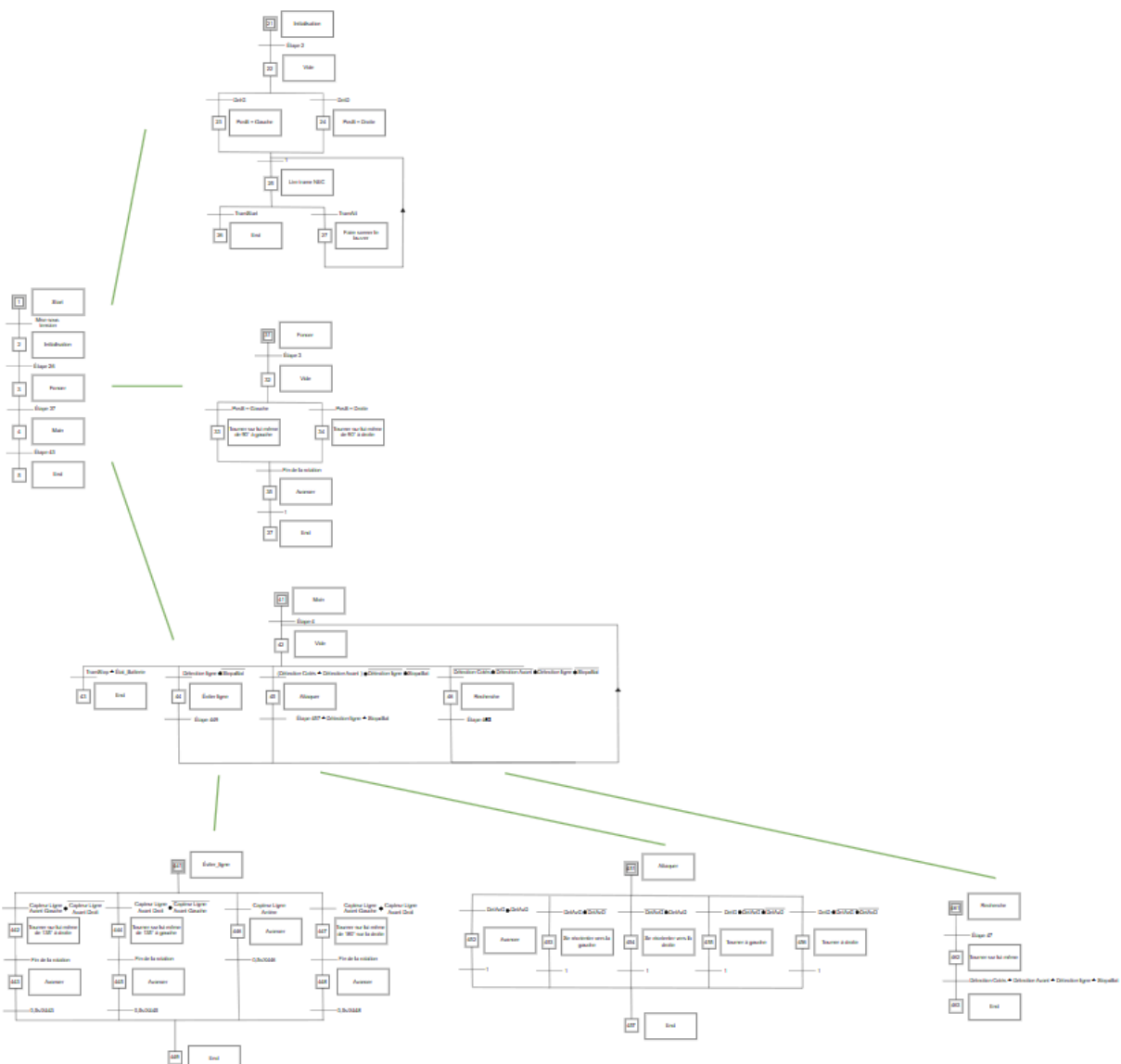


Figure 25 : Vue globale du grafcet

Afin de ne pas être surchargé, ce grafcet est séparé en plusieurs parties/fonctions et chacune d'entre elles est expliquées ci-dessous. De plus, celui-ci est construit sous forme de grafcets fonctionnant simultanément, cependant le fonctionnement de deux grafcet en même temps est rendu impossible grâce au transition. Cette forme permet simplement de ne pas surcharger un même grafcet, ce qui permet une meilleure lisibilité et une meilleure compréhension globale du fonctionnement du robot.

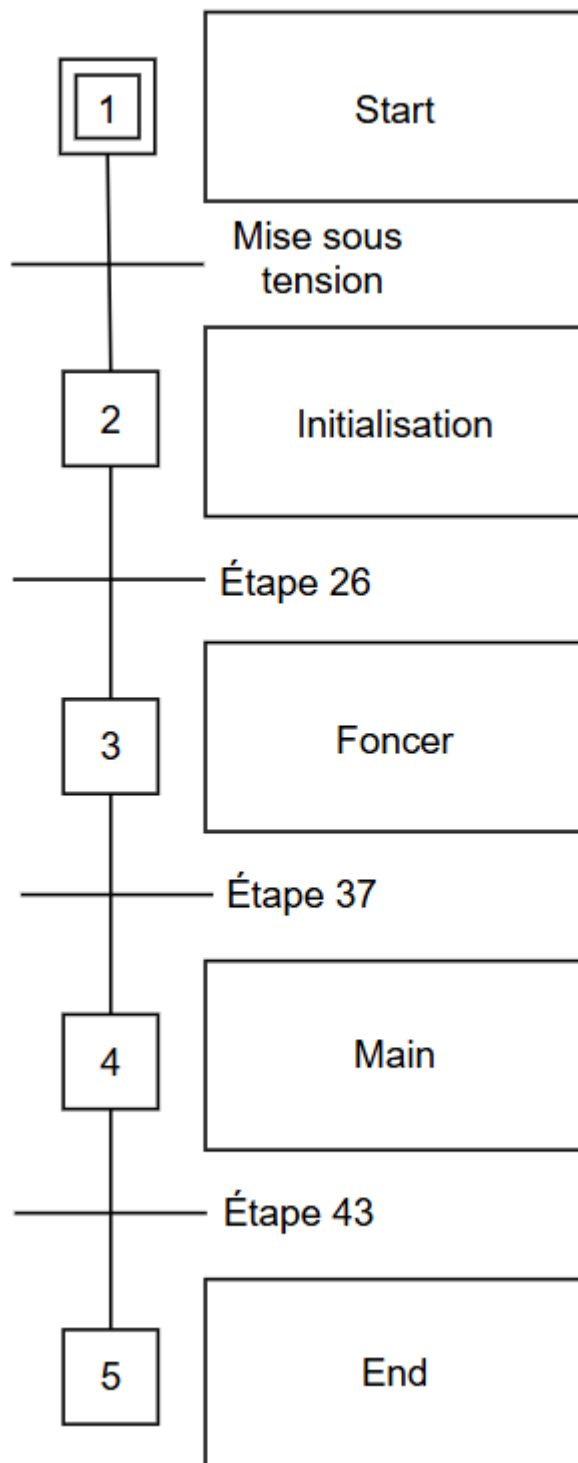


Figure 26 : Algorithme principale

Robot Mini-Sumo (RMS)

Il s'agit de l'algorithme principal, celui-ci commence par la mise sous tension du robot, puis poursuit par une fonction d'initialisation, puis au début du combat le robot va foncer vers son adversaire. Il entrera ensuite dans le main. Il s'agit d'une boucle qui définit le comportement du robot tant que le combat n'est pas fini ou que le robot n'a pas épuisé sa batterie.

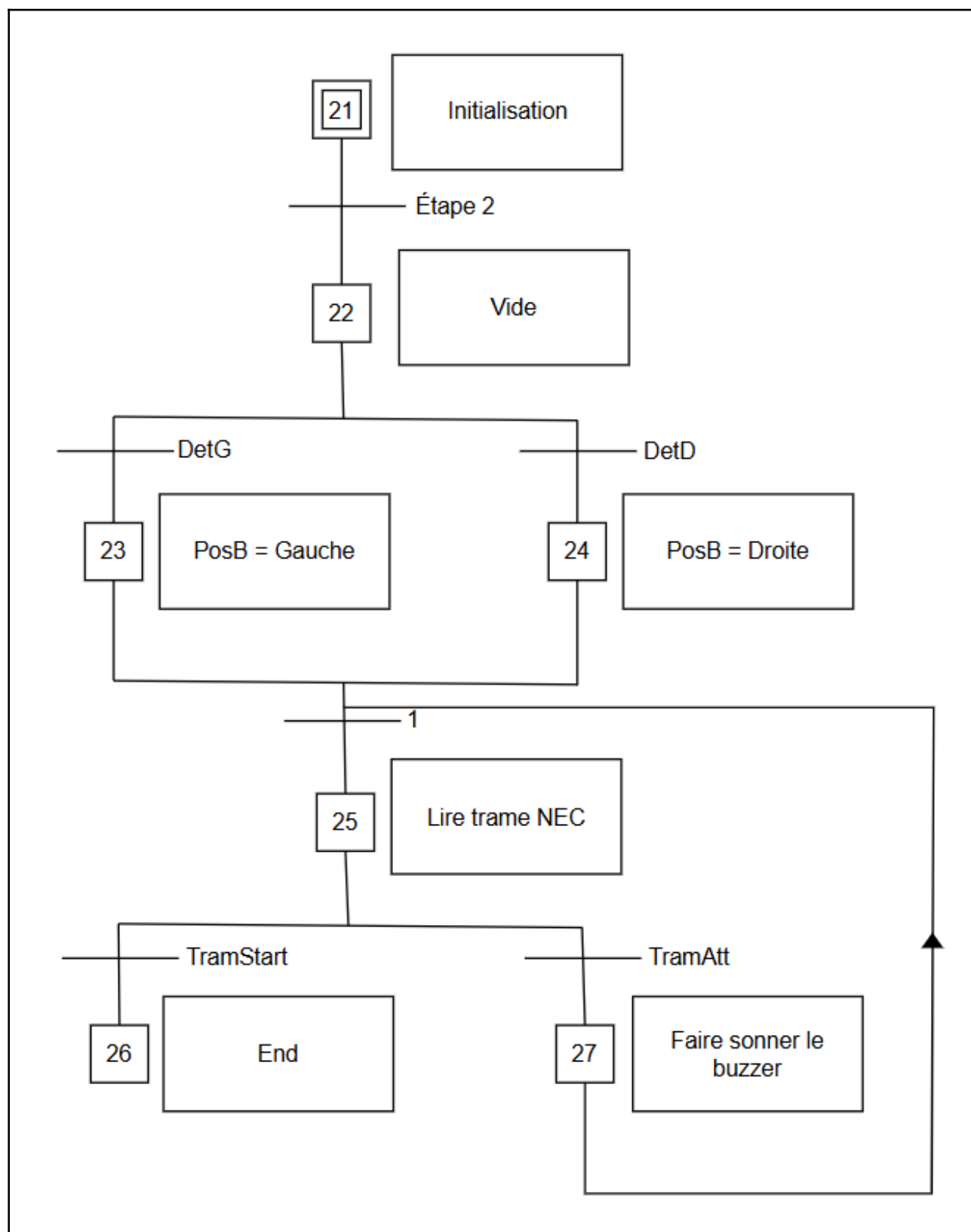


Figure 27 : Initialisation

L'initialisation consiste à récupérer l'information sur la position du robot adverse grâce au capteur se trouvant à droite et à gauche de notre robot. Par la suite, celui-ci attendra l'instruction pour démarrer le combat, ou l'instruction pour indiquer qu'il est prêt et qu'il est bien reconnu par la télécommande via l'actionnement d'un buzzer.

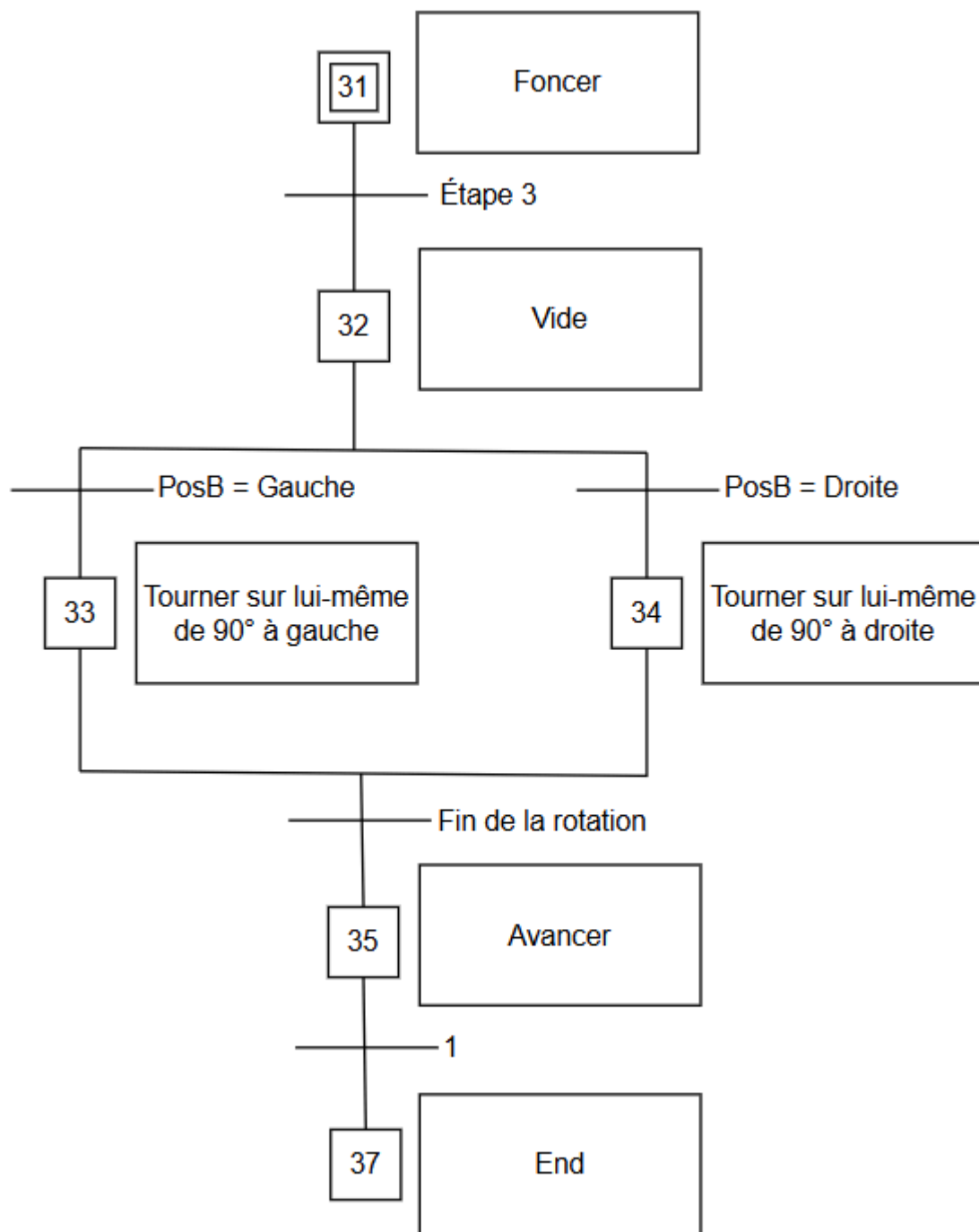
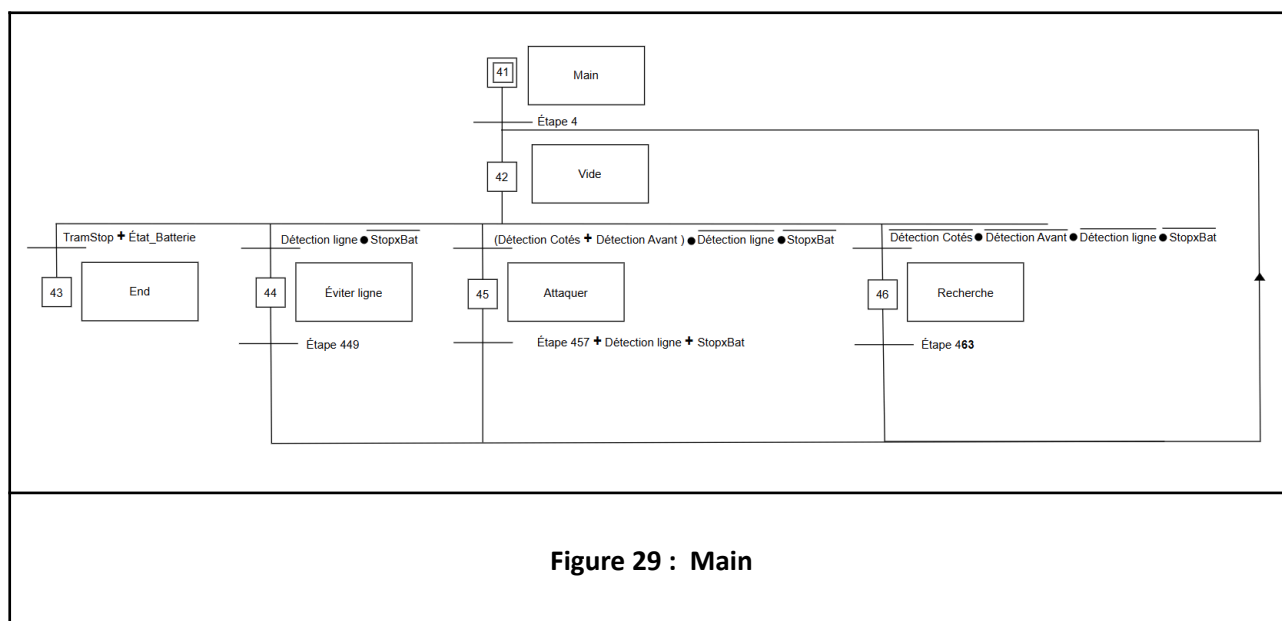


Figure 28 : Foncer

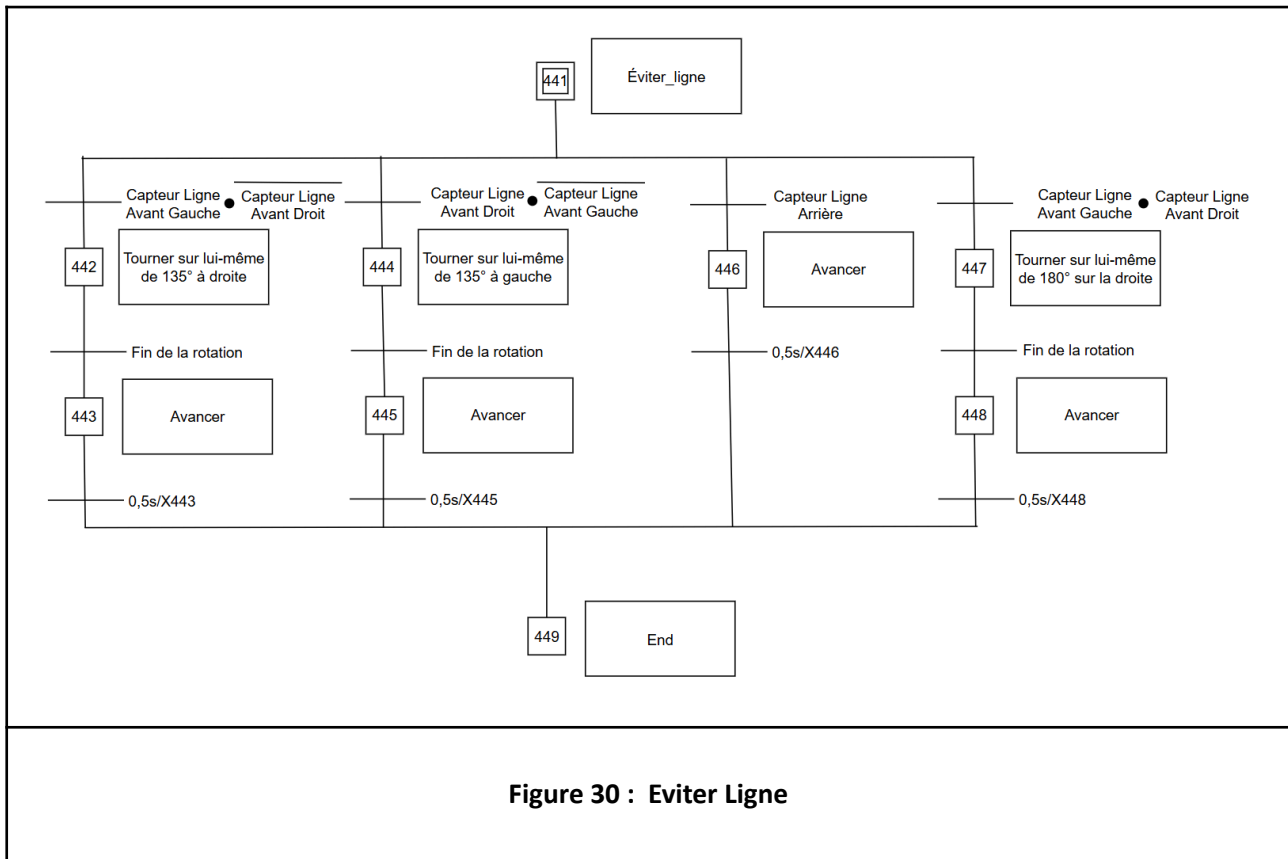
Robot Mini-Sumo (RMS)

La fonction “Foncer” est la première à être effectuée une fois le combat lancé. Celle-ci permet au robot sumo de se tourner vers son adversaire et d’avancer, offrant un temps de réaction très rapide et permettant de prendre l’avantage sur son adversaire.



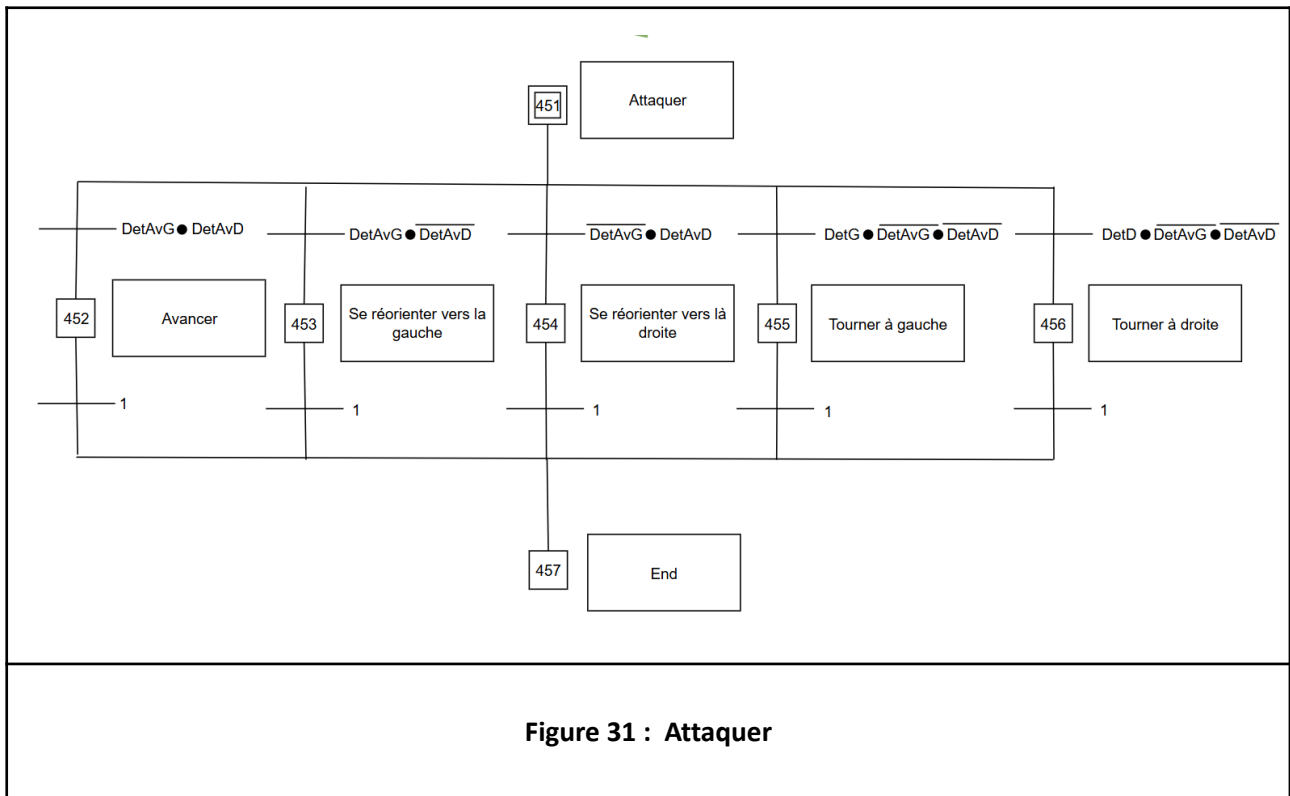
Le main correspond au comportement du robot tout au long du duel. Celui-ci arrêtera le robot en cas d’appui sur le bouton stop de la télécommande, ou si la batterie du robot est trop faible. Sinon, l’algorithme permettra au robot de lancer plusieurs fonctions selon les informations reçues de la part des différents capteurs. La priorité est donnée aux capteurs de ligne qui activeront la fonction “Éviter ligne” afin que quelque soit la position du robot adverse, notre robot se replace pour éviter de se mettre en danger au bord du Dohyo. Ensuite, si le robot adverse est détecté par un des capteurs ultrasons présents à l’avant ou par un des capteurs télémètres présents sur les côtés, la fonction “Attaquer” s’active afin de permettre au robot de suivre son adversaire afin de le pousser. Enfin, si aucune de ces conditions sont remplies, la fonction “Recherche” s’active pour permettre à notre robot de retrouver le robot adverse.

Robot Mini-Sumo (RMS)

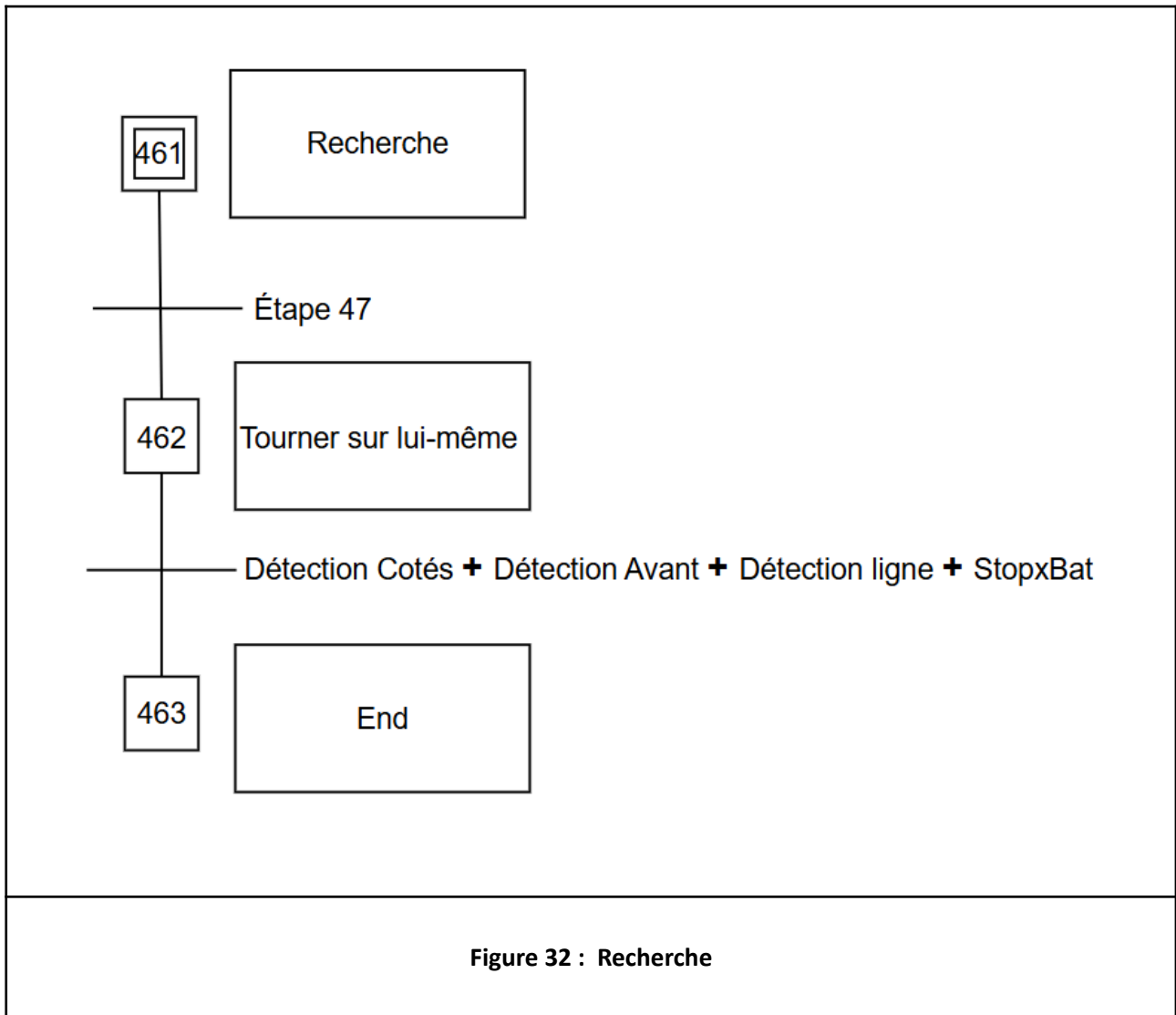


La fonction “Éviter ligne” permet au robot de se replacer sur le terrain lorsqu’un capteur de ligne détecte une ligne à l’avant ou à l’arrière. En fonction de l’emplacement des capteurs ayant détecté une ligne, le robot va s’orienter vers le centre via différentes rotations puis avancer pendant un court instant.

Robot Mini-Sumo (RMS)



La fonction “Attaquer” permet au robot de suivre son adversaire afin de le pousser. Si l’adversaire est détecté par les deux capteurs avant, celui-ci va simplement avancer. Si seul un des deux capteurs avant détecte l’adversaire, celui-ci va se réorienter en bridant l’un de ses moteurs. Si c’est l’un des capteurs télémètres qui détecte l’adversaire, le robot va tourner vers son adversaire.



La fonction “Rechercher” permet au robot de retrouver son adversaire. Pour cela, le robot sumo va tourner sur lui-même jusqu’à retrouver son adversaire ou, jusqu’à ce que la batterie soit trop faible ou que l’appui sur le bouton stop de la télécommande soit détecté.

```

Initialisation(){
Détece la position du robot adverse et la stock
dans la variable PosB
Si Lire_Trame_NEC() =1 ->
Lance la fonction Foncer(PosB)
Si Lire_Trame_NEC() =2 ->
Fais sonner le buzzer
}

Foncer(PosB){
Tourne et fonce vers le robot adverse
}

void Main (void){
Si Etat_batterie()=1 ou Lire_Trame_NEC()=3
-> break
Sinon si un capteur de ligne détecte une ligne->
void Eviter_ligne(void){
En fonction du/des capteur/s de ligne ayant
détecté une ligne, se place face au centre de
l'arène et avance
}

Si le robot adverse est détecté par un capteur
ultrason/capteur télémétrique ->
void Attaquer(void){
Avance/Se réorienter/Tourne pour attaquer
l'adversaire
}

Si aucun capteur ne détecte le robot
Recherche(void){
Tourner sur lui-même à vitesse constante
Si capteur détecte un ennemi ou
Etat_batterie()=1 ou Lire_Trame_NEC()=3 ->
break
}
}

```

Figure 33 : Fonctions de Traitement du Robot Sumo

Les fonctions suivantes reprennent le fonctionnement du Grafcet.

2.3 Conclusion de la conception préliminaire du produit

À l'issue de la phase de conception et des tests effectués, nous constatons que les exigences du cahier des charges ont été respectées. Les fonctionnalités principales, telles que la détection de l'adversaire, le déclenchement via la télécommande et l'adaptation aux conditions de luminosité ont été validées avec succès lors des expérimentations. Le robot détecte un adversaire à une distance minimale de 40 cm grâce aux capteurs à ultrasons HC-SR04 et évite la sortie du dohyo à l'aide des capteurs de contraste, qui se montrent fiables face aux variations de lumière. Pour plus d'efficacité nous avons également intégré des capteurs latéraux Sharp GP2Y0A41SK0F offrant une portée de détection limitée à 30 cm, offrant ainsi un avantage lors du départ. Les tests sur moteur se sont montrés concluants et l'autonomie estimée montre que les résultats sont conformes aux exigences.

Ainsi, la conception préliminaire nous a permis de nous faire une idée des différents composants dont nous aurons besoin pour concevoir notre prototype de manière conforme aux différentes exigences du cahier des charges. Le dimensionnement, la programmation et les branchements seront étudiés de manière bien plus détaillée lors de la prochaine phase de conception.

3. Conception détaillée du produit

Ce chapitre détaille la conception du produit développé. Il constitue une preuve de la conformité du produit. Chaque paragraphe de cette étude fait donc clairement référence aux exigences client issues du [CDC].

3.1 Conception détaillée du robot mini-sumo

Chaque bloc fonctionnel doit faire l'objet d'un chapitre de conception détaillé, présenté comme suit.

Référence de conception : CDT01

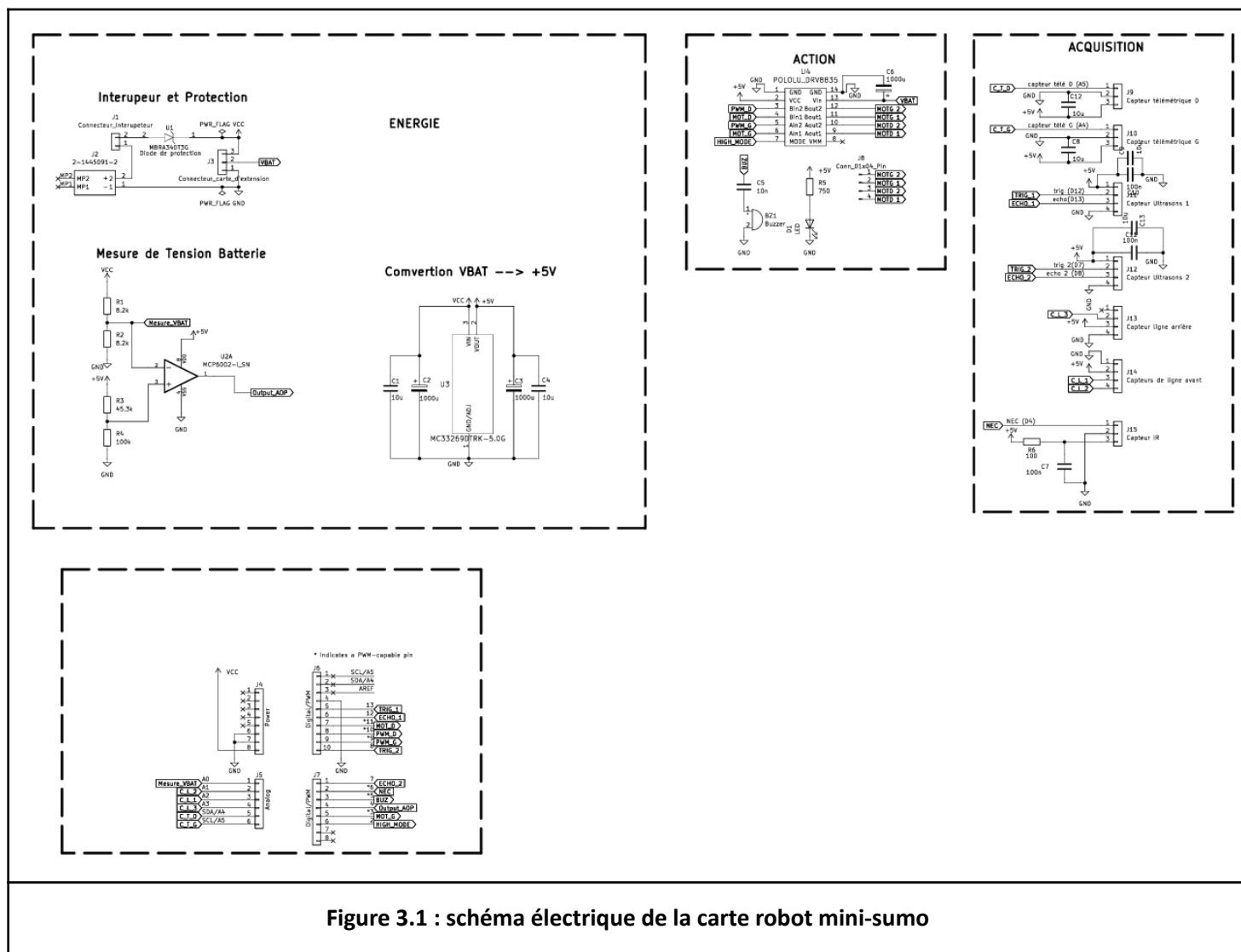
Compétences GEII : C1b 21 22 25

Exigences client vérifiées : EXIG_COMPORTEMENT EXIG_AUTONOMIE EXIG_SECUR_BATT
EXIG_DEPLACEMENT EXIG_ADVERSAIRE EXIG_LUMINOSITE EXIG_DEPART EXIG_CARTE
EXIG_COUT EXIG_CHASSIS_DIMENSIONS EXIG_MASSE EXIG_INTEGRITE_MECA
EXIG_FIXATION_CARTE

A la suite de la conception préliminaire, l'activité de conception détaillée a été menée. Le schéma électrique complet de la carte est fourni ci-dessous.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	44/ 134
----------------------------------	---	------------

Robot Mini-Sumo (RMS)



3.2 Conception détaillée de la partie acquisition

Dans cette partie, nous présentons deux codes distincts pour chaque capteur :

- Le premier, réalisé au préalable, permet de vérifier le bon fonctionnement des capteurs de même type. Il se limite à afficher les résultats sur le moniteur série.
- Le second correspond à l'implémentation dans le programme final. Il transmet de véritables informations au robot, lui permettant ainsi de réagir ou d'effectuer une action en fonction des mesures obtenues.

Par ailleurs, concernant le choix des broches auxquelles sont connectés nos capteurs, nous avons sélectionné celles qui n'étaient pas utilisées par les autres exigences nécessitant un placement précis, comme par exemple les broches dédiées au PWM.

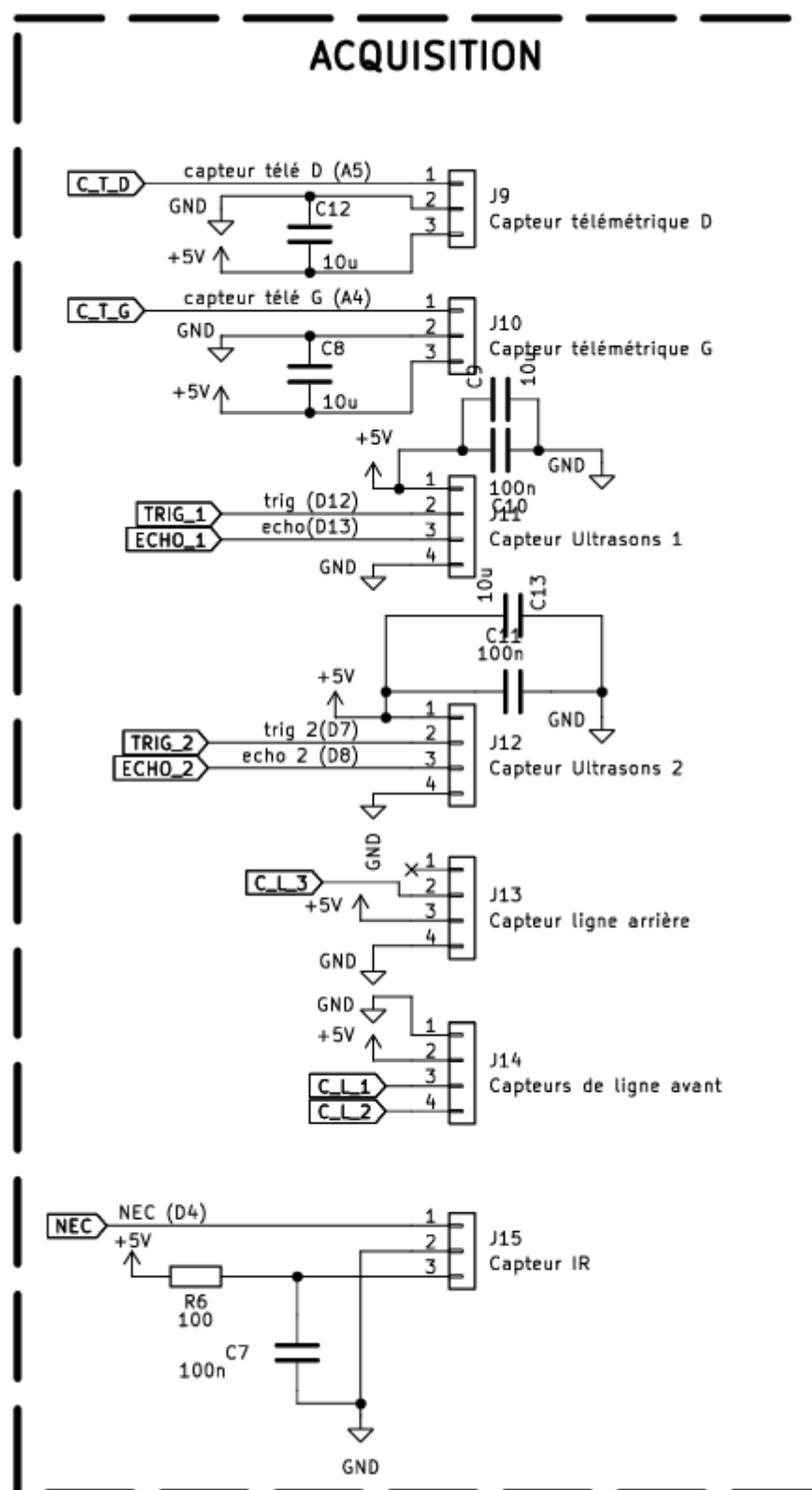


Figure 3.2. : Vue d'ensemble du schéma électrique de la partie acquisition

3.2.1 Capteurs adversaires avant

Référence de conception : CDT02

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_ADVERSAIRE

3.2.1.1 Schéma électrique des capteurs adversaires avant

En consultant la datasheet du HC-SR04, nous avons identifié le schéma de câblage recommandé (voir Figure X ci-dessous).



Figure 3.2.1.1 : Câblage du capteur HC-SR04

Ainsi, sur notre schéma électrique, les connecteurs J11 (capteur avant-gauche) et J12 (capteur avant-droite) respectent le brochage illustré ci-dessus. De plus, nous avons examiné les broches disponibles sur l'Arduino Uno pour connecter nos capteurs à ultrasons, en privilégiant des GPIO libres compatibles avec les signaux numériques. Ainsi, nous avons connecté le Trigger à la broche D12 et l'Écho à la broche D13 pour le capteur J11, puis le Trigger à la broche D7 et l'Écho à la broche D8 pour le capteur J12.

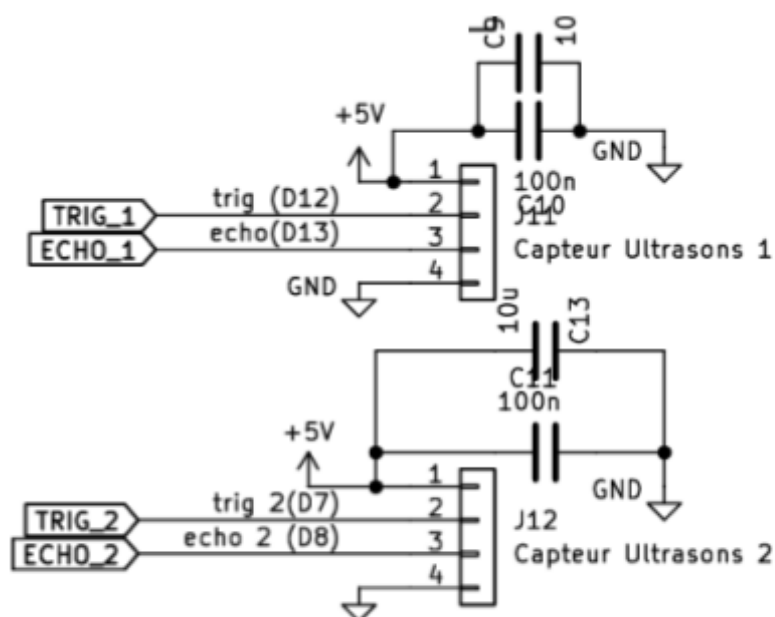


Figure 3.2.1.2 : Schéma électrique pour les capteurs ultrasons avant HC-SR04

Pour lisser le courant et éviter les pics de consommation, un condensateur de découplage est ajouté en parallèle de l'alimentation des capteurs ultrasons.

Pour dimensionner ces condensateurs, nous utilisons le HTUT 24 dans lequel nous apprenons que

$$C = I_{max} \times \frac{\Delta t}{\Delta V} = I \times \frac{\frac{1}{f}}{0.01 \times V_{CC}}$$

D'après les calculs précédents nous savons que $I = 15 \text{ mA}$ et que $V_{CC} = 5\text{V}$. Nous savons également que la fréquence du microcontrôleur est de 40 kHz ainsi :

$$C = 15 \times 10^{-3} \times \frac{\frac{1}{40000}}{0.01 \times 5} = 7.5 \mu\text{F}$$

Donc $C > 7.5 \mu\text{F}$

Après normalisation, le condensateur utilisé est de $10 \mu\text{F}$. Cependant, d'après le HTUT 24, si nous avons besoin d'un condensateur de $10 \mu\text{F}$ qui se trouve dans la catégorie des aluminiums polarisés, nous devons ajouter un autre condensateur d'une technologie différente, ici polyester. Ainsi, nous divisons $10 \mu\text{F}$ par 100, ce qui nous donne un condensateur de 100nF .

3.2.1.2 Code informatique des capteurs adversaires avant

Nous avons conçu un programme permettant de détecter la présence d'un adversaire à l'avant du robot grâce aux capteurs à ultrasons. Comme pour les autres capteurs, nous avons d'abord écrit et testé le code

séparément afin de vérifier son bon fonctionnement.

Avant de l'intégrer dans le programme principal, qui permettra au robot de déterminer la direction à suivre pour attaquer l'adversaire, nous avons choisi d'afficher les distances mesurées sur le moniteur série afin de confirmer le bon fonctionnement du programme.

```
// Définition des numéros de port
#define trigPin_d 12 // Trigger (émission) capteur droit
#define echoPin_d 13 // Echo (réception) capteur droit
#define trigPin_g 7 // Trigger (émission) capteur gauche
#define echoPin_g 8 // Echo (réception) capteur gauche

// Variables
long duree; // Durée de l'écho
int distance_d; // Distance mesurée pour le capteur droit
int distance_g; // Distance mesurée pour le capteur gauche
const int TOLERANCE = 2; // Tolérance en cm pour considérer les distances égales (précision des mesures selon les capteurs pouvant varier)
const int PERIMETRE = 57; // Périmètre de détection (77 cm(taille doyo) - 2*10 cm(taille robots))

void setup() {
  pinMode(trigPin_g, OUTPUT); // Configuration du port Trigger gauche comme SORTIE
  pinMode(echoPin_g, INPUT); // Configuration du port Echo gauche comme ENTREE
  pinMode(trigPin_d, OUTPUT); // Configuration du port Trigger droit comme SORTIE
  pinMode(echoPin_d, INPUT); // Configuration du port Echo droit comme ENTREE
  Serial.begin(9600); // Démarrage de la communication série
}

// Fonction pour le capteur droit
int Capteur_Ultrason_d() {
  // Émission d'un signal de durée 10 microsecondes
  digitalWrite(trigPin_d, LOW);
  delayMicroseconds(5);
  digitalWrite(trigPin_d, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin_d, LOW);

  // Écoute de l'écho
  duree = pulseIn(echoPin_d, HIGH);

  // Calcul de la distance
  distance_d = (duree * 0.034) / 2;

  return distance_d; // Retourner la distance mesurée
}

// Fonction pour le capteur gauche
int Capteur_Ultrason_g() {
  // Émission d'un signal de durée 10 microsecondes
  digitalWrite(trigPin_g, LOW);
  delayMicroseconds(5);
  digitalWrite(trigPin_g, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin_g, LOW);

  // Écoute de l'écho
  duree = pulseIn(echoPin_g, HIGH);

  // Calcul de la distance
  distance_g = (duree * 0.034) / 2;

  return distance_g; // Retourner la distance mesurée
}
```

```

void loop() {
  dist_g = Capteur_Ultrason_g();
  dist_d = Capteur_Ultrason_d();
  if (dist_g <= PERIMETRE){
    Serial.print("capteur gauche ");
    Serial.print(dist_g);
    Serial.println("cm");
  }
  if (dist_d <= PERIMETRE){
    Serial.print("capteur droit ");
    Serial.print(dist_d);
    Serial.println("cm");
  }

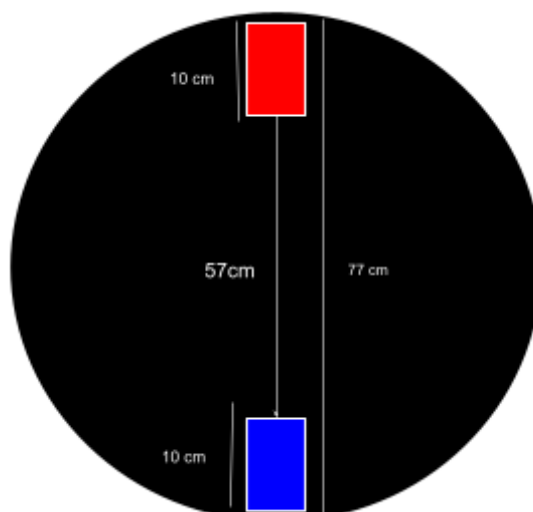
  delay(1000); // Pause pour éviter des lectures trop rapides
}

```

Figure 3.2.1.3 : Code test des capteurs ultrasons

Le programme ci-dessus est le code permettant le bon fonctionnement des deux capteurs ultrasons placés à l'avant. Pour cela, nous avons commencé par définir et configurer tous les ports utilisés par les capteurs : c'est-à-dire deux TRIGGER pour l'émission et deux ECHO pour la réception. Ensuite nous avons définis les variables utiles à savoir :

- `duree` : la durée de l'écho
- `distance_g` et `distance_d` : la distance déterminée par le capteur gauche et droit
- `TOLERANCE` : correspond à tolérance en cm pour laquelle on considère que la distance est égale, dû à l'imprécision entre les capteurs.
- `PERIMETRE` : correspond au périmètre nécessaire pour couvrir tout le dohyo. En effet le dohyo ayant diamètre de 77cm et les deux robots présents dessus mesurant 10 cm chacun, le robot sumo n'a plus besoin que d'un périmètre de $77 - 2 \times 10 = 57\text{cm}$



Deux fonctions ont par la suite été créées pour les deux capteurs ultrasons :

- `Capteur_Ultrason_d()`
- `Capteur_Ultrason_g()`

Ces deux fonctions ont le même principe :

Nous déclenchons une impulsion de 10 μ s sur le TRIG pour permettre au capteur d'émettre suffisamment d'informations et ainsi garantir une mesure fiable.

Ensuite, la fonction `pulseIn()` mesure la durée pendant laquelle le signal ECHO (réception) reste à l'état HAUT, c'est-à-dire le temps de l'aller-retour de l'onde ultrasonique.

Cette durée nous permet de calculer la distance à l'adversaire grâce à la formule : Distance = Vitesse * temps

Dans notre cas Distance = Vitesse du son dans l'air * durée / 2

avec une vitesse du son de 343 m/s (soit 0,034 cm/ μ s) dans l'air. Le $\div 2$ tient compte du trajet aller-retour.

Enfin, dans la fonction `loop()`, nous vérifions si la distance mesurée est valide : le capteur a une portée maximale de 4 m, mais nous ne conservons que les valeurs inférieures au périmètre utile du dohyo précédemment déterminé.

```
void Attaquer(void) {
    if ((UltrasonG == 1) && (UltrasonD == 1)) {
        Avancer(255);
    }
    else if (UltrasonG == 1) {
        Reorienter(255, 'G');
        PosB = 'G';
    }
    else if (UltrasonD == 1) {
        Reorienter(255, 'D');
        PosB = 'D';
    }
    else if (TelemetriqueG == 1) {
        Turn_itself('G', 0);
        PosB = 'G';
    }
    else if (TelemetriqueD == 1) {
        Turn_itself('D', 0);
        PosB = 'D';
    }
}
```

```

void loop() {
  while(Broken!= true){
    LigneAvG = Capteur_Ligne(CapteurLigneAvantGauche_Pin);
    LigneAvD = Capteur_Ligne(CapteurLigneAvantDroite_Pin);
    LigneAr = Capteur_Ligne(CapteurLigneArriere_Pin);
    UltrasonG = Capteur_Ultrason(CapteurAvantGauche_trig, CapteurAvantGauche_echo);
    UltrasonD = Capteur_Ultrason(CapteurAvantDroite_trig, CapteurAvantDroite_echo);
    TelemetriqueG = Capteur_Telemetrique(CapteurGauche_Pin);
    TelemetriqueD = Capteur_Telemetrique(CapteurDroite_Pin);
    if(Etat_batterie(DigitalBatterie_Pin)==1 || Lire_Trame_NEC()==3){
      Avancer(0);
      Broken = true;
      break;
    }
    else if((LigneAvG==1) || (LigneAvD==1) || (LigneAr==1)){
      Eviter_ligne();
    }
    else if((UltrasonG==1) || (UltrasonD==1) || (TelemetriqueG==1) || (TelemetriqueD==1)){
      Attaquer();
    }
    else{
      Recherche();
    }
  }
}

```

Figure 3.2.1.4 : Code implémenté des capteurs ultrasons

3.2.2 Capteurs de sol (ligne)

Référence de conception : CDT03

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_LUMINOSITE

3.2.2.1 Schéma électrique des capteurs de sol

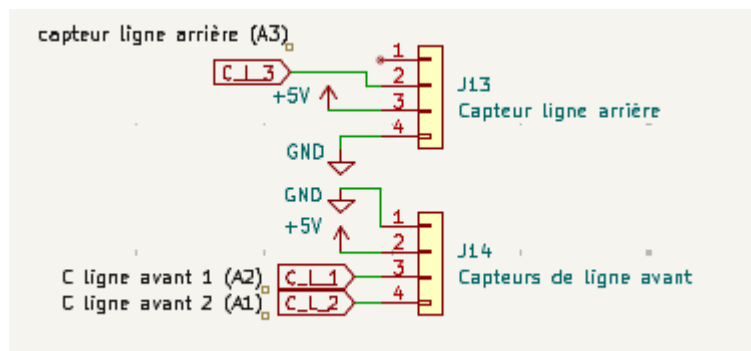
Pour répondre à l'exigence EXIG_LUMINOSITE, nous avons intégré un capteur optique de contraste CNY70. Ce capteur détecte le passage de la ligne blanche délimitant le dohyō en comparant le taux de réflexion entre la surface noire du ring et la bordure blanche.

En consultant la datasheet du CNY70, nous avons identifié le schéma de câblage recommandé (voir Figure ci-dessous).



Figure 3.2.2.1 : Câblage du capteur CNY70

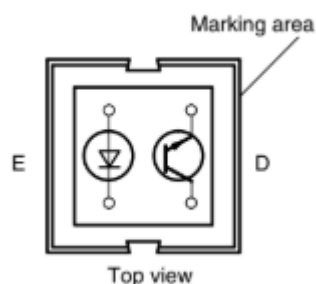
Nous avons ainsi réalisé le schéma électrique sur KiCad avec deux connecteurs à quatre broches, J14 et J13, pour brancher les capteurs de ligne de façon simple et pratique.

**Figure 3.2.1.2 : Schéma électrique pour les capteurs de ligne CNY70**

Le connecteur J14 gère les deux capteurs avant. Il fournit le +5 V sur la broche 2, envoie le signal du capteur gauche sur A2 (broche 3), le signal du capteur droit sur A1 (broche 4), et la masse sur la broche 1.

Le connecteur J13 est dédié au capteur arrière. Il donne le +5 V sur la broche 3, transmet le signal vers A3 (broche 2), laisse la broche 1 libre pour une utilisation future, et connecte la masse sur la broche 4.

Sur le PCB des CNY70 il y a deux résistances associées à chaque capteur (inclus dans le PCB fourni), car ils sont constitués de deux parties internes :

**Figure 3.2.2.3 : composition du capteur de constat CNY70 issue de la datasheet**

- Une LED infrarouge : elle émet un faisceau sur la surface placée devant le capteur. La lumière réfléchi si surface claire ou brillante (comme le blanc) et absorbée sir la surface est sombre (comme le noir).
- Un phototransistor : il reçoit la lumière réfléchi et la convertit en courant électrique, dont l'intensité varie en fonction de la quantité de lumière renvoyée par la surface.

Chaque résistance est donc associée à une de ces deux parties.

Résistance associée à la LED IR :

Objectif : créer une chute de tension proportionnelle au courant qui passe dans la LED pour fixer le courant à une valeur sûre.

La LED du CNY70 a une tension directe. D'après la datasheet la valeur typique est de $V_f = 1,25 \text{ V}$.

Puisqu'on place une résistance en série, alors le courant qui traverse la LED est le même que celui qui traverse la résistance. $I_f = 20 \text{ mA}$ d'après la datasheet.

$$R_{LED\ IR} = \frac{V_{cc} - V_f}{I_f} = \frac{5 - 1,25}{0,020} = 175 \Omega$$

On prend donc une valeur normalisée de 180Ω (série E12 +/- 10%).

Résistance associée au phototransistor :

Objectif : mettre une résistance de pull-up entre le collecteur et V_{cc} pour convertir ce courant en tension mesurable. Cela permet d'obtenir :

- une tension de sortie proportionnelle à la lumière réfléchi (plus la surface est claire, plus la tension varie)
- un signal exploitable par un microcontrôleur ou un comparateur (0 / 1 logique, ou tension analogique)

$$V_{sortie} = V_{cc} - I_C * R_C$$

Quand on aura une surface blanche I_C sera élevé et donc V_{out} faible et inversement pour une surface noire.

Pour dimensionner la résistance de pull up on applique alors la formule suivante :

$$R_c = \frac{V_{CC} - V_{seuil}}{I_C}$$

avec notre V_{seuil} correspondant à la valeur ADC du seuil que nous avons fixés dans notre code pour passer du blanc à noir. Notre "valeurDeReference" est de 600 donc pour trouver la tension V_{seuil} associé on effectue la formule :

$$V_{seuil} = \frac{ADC}{1023} * V_{CC} = \frac{600}{1023} * 5 = 2,93 \text{ V}$$

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	54/ 134
----------------------------------	---	------------

On a donc :

$$R_c = \frac{V_{CC} - V_{seuil}}{I_c} = \frac{5 - 2,93}{0,001} = 2067 \Omega$$

Donc pour la résistance de pull-up on peut choisir comme valeur normalisée 2,2 kΩ de la série E12 (+/-10 %).

3.2.2.2 Code informatique des capteurs de sol

Nous avons conçu un programme permettant de déterminer si le robot se trouve sur une surface blanche ou noire du dohyo. Comme pour les autres capteurs, nous avons d'abord écrit et testé le code séparément afin de vérifier son bon fonctionnement.

Avant de l'intégrer dans le programme principal, qui permettra au robot d'éviter les zones blanches et d'adapter son comportement en conséquence, nous avons choisi d'afficher la couleur détectée sur le moniteur série afin de confirmer le bon fonctionnement du programme.

```

#define capteur_avant_droit  A1 // valeur renvoyée dans n_capteurs_L[i]: 15
#define capteur_avant_gauche  A2 // valeur renvoyée dans n_capteurs_L[i]: 16
#define capteur_arriere  A3 // valeur renvoyée dans n_capteurs_L[i]: 17

int valeurDeReference = 600; // seuil pour passer du blanc à noir trouvé par test
int n_capteurs_L[]={capteur_avant_droit,capteur_avant_gauche,capteur_arriere};

void setup() {
  Serial.begin(9600);
  pinMode(capteur_avant_droit, INPUT);
  pinMode(capteur_avant_gauche, INPUT);
  pinMode(capteur_arriere, INPUT);
}

char Capteur_Ligne(int i){
  int valeur = analogRead(n_capteurs_L[i]); // Lire la valeur analogique
  Serial.println(valeur); // affichage de la valeur sur le moniteur série
  if (valeur < valeurDeReference){ // comparaison de la valeur lue avec la valeur de référence
    return 'B'; } // B = surface blanche
  else{
    return 'N'; } // N = surface noire
}

void loop() {

for (int i = 0; i < 3; i++) { // boucle qui permet de lire tout les capteurs
  Serial.print("Le Capteur ");
  Serial.print(n_capteurs_L[i]);
  Serial.print(" (A"); // broche analogique du capteur
  Serial.print(i+1); // numéro de la broche 1, 2 ou 3
  Serial.print(")");
  Serial.print(" est sur une surface : ");
  Serial.println(Capteur_Ligne(i)); // B ou N
}
  delay(500); // Attendre 500 ms avant la prochaine lecture
}

```

Figure 3.2.2.4 : Code test des capteurs de ligne

Dans notre programme dédié aux capteurs de ligne, nous fixons un seuil de référence qui sert de valeur de comparaison avec la valeur renvoyée par chaque capteur. Ce seuil permet de déterminer si la surface détectée est blanche ou noire. Nous avons déterminé ce seuil de 600 grâce à des tests expérimentaux réalisés sur nos surfaces blanche et noire, car ce seuil peut varier selon différents facteurs tels que l'intensité de la couleur blanche, la texture du support ou la propreté du Dohyo.

On considère donc que si la valeur lue est inférieure à 600, le capteur détecte une surface blanche, sinon il détecte une surface noire.

Le tableau `n_capteurs_L[]` regroupe les trois capteurs de ligne, ce qui permet d'y accéder facilement avec une boucle.

Nous utilisons trois fonctions :

- `setup ()` : elle initialise le système en définissant chaque capteur de ligne
- `Capteur_Ligne ()` : On lit la valeur renvoyée par le capteur de ligne, puis on la compare au seuil de référence afin de déterminer la nature de la surface détectée. En fonction du résultat, la fonction retourne 'B' pour une surface blanche ou 'N' pour une surface noire.
- `loop ()` : La boucle for parcourt les 3 capteurs (indices 0 à 2). À chaque itération :
 - Elle affiche le numéro du capteur (A1, A2, A3).
 - Elle appelle `Capteur_Ligne(i)` pour déterminer la couleur détectée.
 - Elle affiche le résultat : B (blanc) ou N (noir).

```
void loop() {
    while(Broken!= true){
        LigneAvG = Capteur_Ligne(CapteurLigneAvantGauche_Pin);
        LigneAvD = Capteur_Ligne(CapteurLigneAvantDroite_Pin);
        LigneAr = Capteur_Ligne(CapteurLigneArriere_Pin);
        UltrasonG = Capteur_Ultrason(CapteurAvantGauche_trig, CapteurAvantGauche_echo);
        UltrasonD = Capteur_Ultrason(CapteurAvantDroite_trig, CapteurAvantDroite_echo);
        TelemetriqueG = Capteur_Telemetrique(CapteurGauche_Pin);
        TelemetriqueD = Capteur_Telemetrique(CapteurDroite_Pin);
        if(Etat_batterie(DigitalBatterie_Pin)==1 || Lire_Trame_NEC()==3){
            Avancer(0);
            Broken = true;
            break;
        }
        else if((LigneAvG==1) || (LigneAvD==1) || (LigneAr==1)){
            Eviter_ligne();
        }
        else if((UltrasonG==1) || (UltrasonD==1) || (TelemetriqueG==1) || (TelemetriqueD==1)){
            Attaquer();
        }
        else{
            Recherche();
        }
    }
}
```

Figure 3.2.2.5 : Code implémenté des capteurs de ligne

3.2.3 Capteur infrarouge

Référence de conception : CDT04

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_DEPART

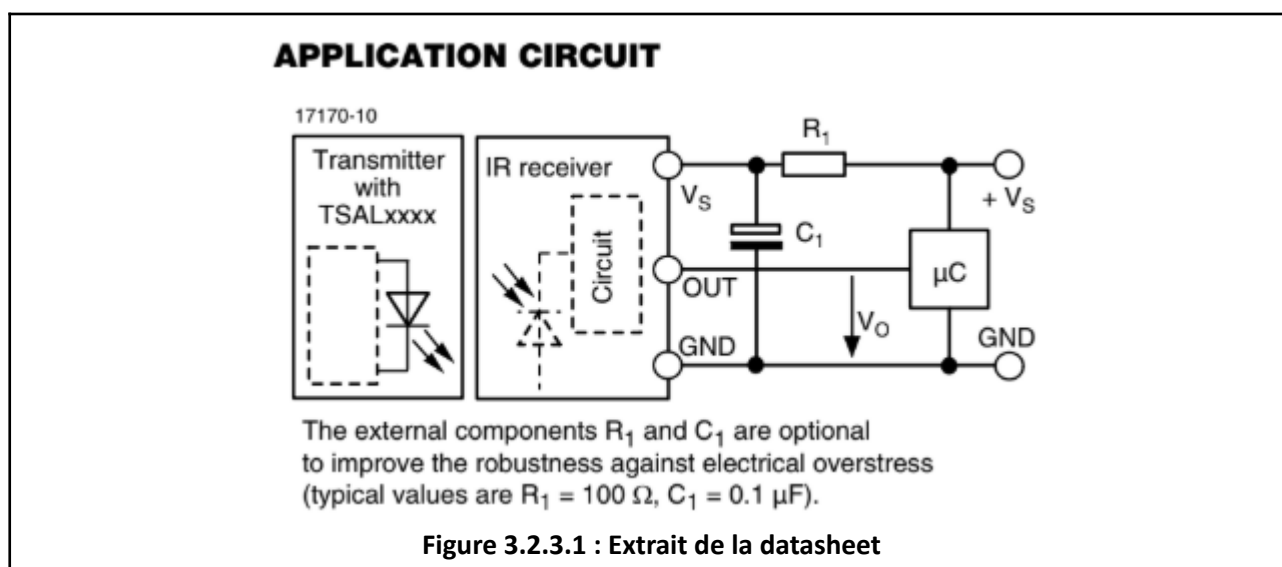
3.2.3.1 Schéma électrique du capteur infrarouge

Afin de répondre à l'exigence EXIG_DEPART, un capteur infrarouge a été intégré pour capter les signaux transmis par la télécommande de l'arbitre. Celui-ci identifie les instructions « Attention », « Start » et « Stop », correspondant respectivement à la phase de préparation, au lancement et à l'interruption du combat.

Dans cette perspective, nous avons tout d'abord choisi le capteur infrarouge requis pour assurer un fonctionnement optimal, avant de dimensionner les résistances et condensateurs à y associer.

Dimensionnements de résistance et de condensateur :

Pour concevoir le schéma électrique de notre capteur IR TSOP18338, nous nous sommes référés aux recommandations de sa datasheet. Nous avons précédemment travaillé sur un autre projet avec un capteur de la même famille, le TSOP4438, partageant la même fréquence de 38 kHz et compatible avec le protocole NEC. La datasheet de ce dernier proposait des valeurs suggérées pour la résistance et le condensateur (voir figure X), que nous avons d'ailleurs adoptées avec succès dans ce projet précédent.



Nous avons donc réalisé des tests avec notre capteur TSOP18338 en utilisant une résistance R_1 de 100Ω série E6 (tolérance $\pm 20\%$) et un condensateur C_1 de 100 nF série E6 (tolérance $\pm 20\%$). Nous avons choisi cette série pour les deux composants car nous n'avons aucune exigence et nous pouvons donc prendre la moins coûteuse.

Dans le schéma d'application du TSOP18338, la résistance et le condensateur permettent de réduire les ondulations et/ou le bruit sur l'alimentation.

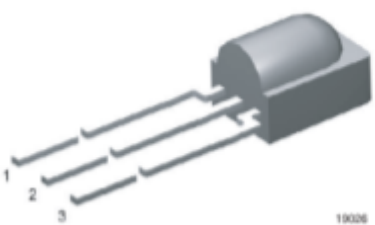
Robot Mini-Sumo (RMS)

R1, placée en série sur la ligne d'alimentation, limite les courants parasites et forme un filtre RC avec le condensateur, ce qui atténue les perturbations de tension, notamment en cas d'alimentation instable ou exposée à du bruit électrique.

C1, connecté entre VS et GND, agit comme un filtre passe-bas, stabilisant la tension d'alimentation en absorbant les variations rapides (parasites ou ondulations), améliorant ainsi la fiabilité du récepteur IR, particulièrement dans des environnements bruyants.

En consultant la datasheet du capteur, nous avons identifié le schéma de câblage recommandé (voir Figure X ci-dessous).

IR Receiver Modules for Remote Control Systems



19026

DESIGN SUPPORT TOOLS [click here to get started](#)

3D Models Available

FEATURES

- Improved dark sensitivity
- Improved immunity against optical noise
- Improved immunity against Wi-Fi noise
- Low supply current
- Photo detector and preamplifier in one package
- Internal filter for PCM frequency
- Supply voltage: 2.5 V to 5.5 V
- Material categorization: for definitions of compliance please see www.vishay.com/doc?99912

MECHANICAL DATA

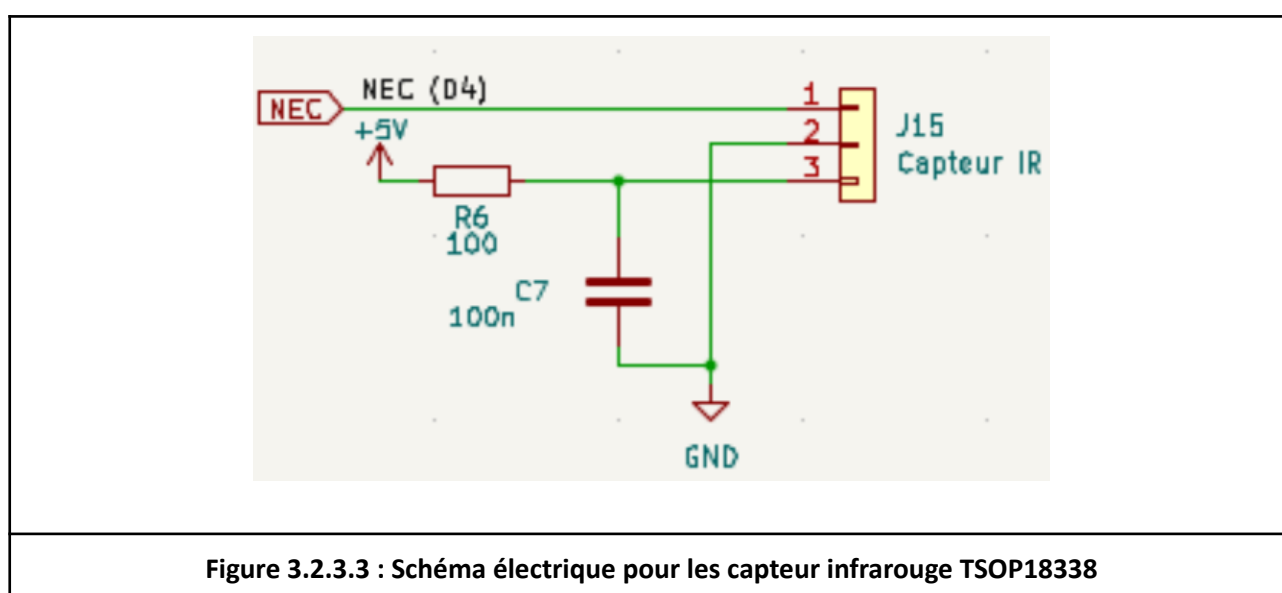
Pinning for TSOP18...:

1 = OUT, 2 = GND, 3 = V_{DD}



Figure 3.2.3.2 : Câblage du capteur TSOP18338

Finalement, à partir de la datasheet (cf Figure 43 et Figure 44), nous avons réalisé le schéma électrique sur KiCad avec un connecteur trois broche J15 comme ci-dessous :



3.2.3.2 Code informatique du capteur infrarouge

Nous avons écrit un programme permettant de décoder les trames NEC issues de la télécommande, afin de transmettre au robot l'information indiquant si l'utilisateur a appuyé sur le bouton "attention", "start" ou "stop". Comme pour les autres capteurs, nous avons d'abord écrit et testé le code séparément afin de vérifier son bon fonctionnement.

Avant de l'intégrer dans le programme général, nous avons choisi d'afficher les résultats des boutons pressés sur le moniteur série, afin de nous assurer que la communication infrarouge fonctionnait correctement.

```
#include <arduino.h>
#include <IRremote.h> // librairie de la télécommande infrarouge

#define Recepteur_Infrarouge 6 // broche du signal infrarouge
#define Adresse_NEC_1 0xFF30CF // Trame associée au bouton START
#define Adresse_NEC_2 0xFF18E7 // Trame associée au bouton ATTENTION
#define Adresse_NEC_3 0xFF7A85 // Trame associée au bouton STOP

IRrecv irrecv(Recepteur_Infrarouge); // associe la réception infrarouge à la broche 13 " Recepteur_Infrarouge"
decode_results Valeur; // décoder la trame NEC reçu

void setup() {
  pinMode(Recepteur_Infrarouge, INPUT); // Configure la broche infrarouge en entrée
  Serial.begin(115200); // Initialiser la communication série
  irrecv.enableIRIn(); // Active la réception IR
}

int Lire_Trame_NEC(){
  if(irrecv.decode(&Valeur)){
    Serial.println(Valeur.value);
    switch(Valeur.value){ // si le récepteur reçoit une trame alors 3 choix possibles

      case Adresse_NEC_1: //Dans le cas où le bouton 1 (START) est appuyé
        Serial.println("Le bouton START est appuyé"); // Afficher sur le moniteur série "Le bouton START est appuyé"
        break;

      case Adresse_NEC_2: //Dans le cas où le bouton 2 (ATTENTION) est appuyé
        Serial.println("Le bouton ATTENTION est appuyé"); // Afficher sur le moniteur série "Le bouton ATTENTION est appuyé"
        break;

      case Adresse_NEC_3: //Dans le cas où le bouton 3 (STOP) est appuyé
        Serial.println("Le bouton STOP est appuyé");// Afficher sur le moniteur série "Le bouton STOP est appuyé"
        break;

    }
    irrecv.resume(); // Préparer le récepteur pour le prochain signal
  }
}

void loop() {
  Lire_Trame_NEC();
}
```

Figure 51 : Code test du capteur infrarouge

Nous avons utilisé la bibliothèque "IRremote.h", qui permet d'accéder à différentes fonctions destinées à la réception de données provenant d'une télécommande infrarouge.

On associe le récepteur infrarouge à la broche 6. Les trames correspondant aux différents boutons de la télécommande sont définies à partir des informations fournies dans sa datasheet.

Dans notre code pour le capteur infrarouge, nous faisons appel à trois fonctions principales :

- `setup ()` : cette fonction configure la broche du récepteur infrarouge, initialise la communication série et active la réception infrarouge afin d'être prêt à recevoir une donnée de la télécommande grâce à "irrecv.enableIRIn()" provenant de la bibliothèque "IRremote.h".
- `Lire_Trame_NEC()` : cette fonction décode les informations envoyées par la télécommande grâce à la fonction `IrReceiver.decode()`. Si une trame valide est reçue, le bouton correspondant est affiché sur le moniteur série. On utilise la fonction `irrecv.resume()`, également issue de la bibliothèque `IRremote.h` pour actualiser les valeurs stockées afin d'être prêt à la réception d'une nouvelle donnée.
- `loop()` : cette fonction appelle en continu `Lire_Trame_NEC()` afin d'assurer une détection permanente des signaux infrarouges.

```
IRrecv irrecv(Recepteur_Infrarouge); // associe la réception infrarouge à la broche " Recepteur_Infrarouge"
decode_results Valeur; // décoder la trame NEC reçue
int Lire_Trame_NEC(void) {
    int ValeurNEC = 0;
    if (irrecv.decode(&Valeur)) {
        switch (Valeur.value) { // si le récepteur reçoit une trame alors 3 choix possibles

            case Adresse_NEC_1: //Dans le cas où le bouton 1 (START) est appuyé
                ValeurNEC = 1;
                break;

            case Adresse_NEC_2: //Dans le cas où le bouton 2 (ATTENTION) est appuyé
                ValeurNEC = 2;
                break;

            case Adresse_NEC_3: //Dans le cas où le bouton 3 (STOP) est appuyé
                ValeurNEC = 3;
                break;

        }
        irrecv.resume(); // Préparer le récepteur pour le prochain signal
    }
    return ValeurNEC;
}
```

Figure 3.2.3.4 : Code implémenté du capteur infrarouge

3.2.4 Capteurs adversaires latéraux

Référence de conception : CDT05

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_ADVERSAIRE

3.2.4.1 Schéma électrique des capteurs adversaires latéraux

Pour répondre à l'exigence EXIG_ADVERSAIRE, qui exige une détection frontale de l'adversaire, nous avons ajouté deux capteurs télémétriques latéraux (gauche et droit). Cette amélioration, non obligatoire, permet au robot de repérer l'adversaire sur ses flancs et d'agir avec plus de réactivité sur le dohyō.

Ainsi, nous avons consulté les datasheets des capteurs télémétriques Sharp GP2Y0A41SK0F choisis pour définir leur câblage, puis réalisé le schéma en intégrant ces connexions aux broches appropriées de l'Arduino.

Connector signal	
	Signal name
①	V _o
②	GND
③	V _{cc}

Figure 3.2.4.1 : Câblage du capteur télémétrique Sharp GP2Y0A41SK0F

Conformément à la recommandation de la datasheet du GP2Y0A41SK0F (page 8), un condensateur de découplage de 10 μ F a été ajouté en parallèle entre les broches VCC et GND, le plus près possible du capteur pour stabiliser l'alimentation.

Nous avons ainsi réalisé le schéma électrique sur KiCad, visible ci dessous:

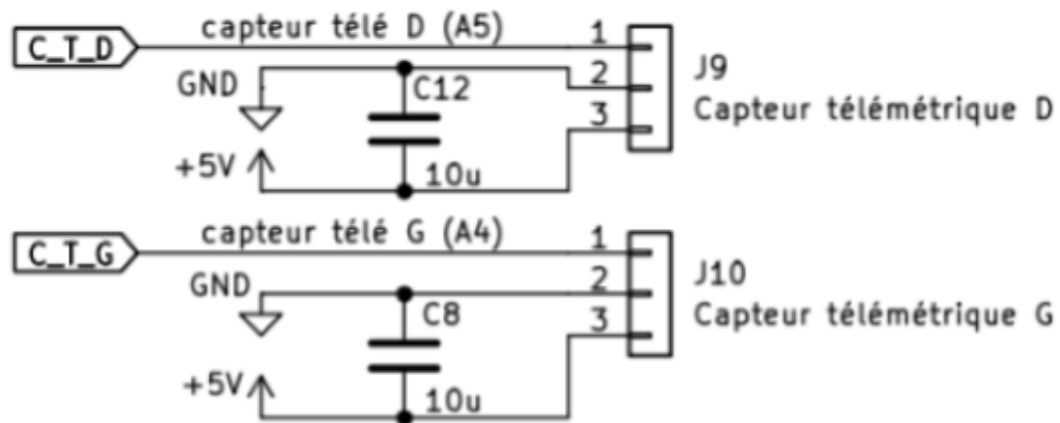


Figure 3.2.4.2 : Schéma électrique pour les capteurs télémétriques Sharp GP2Y0A41SK0F

3.2.4.2 Code informatique des capteurs adversaires latéraux

Nous avons écrit un programme qui permet de mesurer la distance à l'aide de deux capteurs télémétriques analogiques placés sur le côté droit et le côté gauche. Comme les autres programmes des capteurs, nous avons dans un premier temps écrit les codes séparément pour être certain de leur bon fonctionnement. Nous avons donc décidé, avant d'implémenter ce bout de code dans le programme général, d'afficher sur le moniteur série de résultats des distances entre l'adversaire et les différents capteurs.

Robot Mini-Sumo (RMS)

```
// Définition des broches des capteurs
#define tele_droite    A5
#define tele_gauche    A4

void setup() {
    Serial.begin(9600); // lancer la communication série à 9600 bauds
    pinMode(A5, INPUT);
    pinMode(A4, INPUT);
}

int Capteur_telemetrique_d(){
    // Lecture du capteur télémétrique droit
    int valeurBrute_d = analogRead(tele_droite);
    float tension_d = valeurBrute_d * (5.0 / 1023.0); // Calcul de la tension de la valeur récupérée par le capteur droit
    float distance_d = 13.0 * pow(tension_d, -1); // Estimation empirique de la distance entre le capteur télémétrique droit et l'adversaire
    // Affichage des résultats sur le moniteur série
    Serial.print("droite : ");
    Serial.print(distance_d);
    Serial.println(" cm");

    Serial.println();
}

int Capteur_telemetrique_g(){
    // Lecture du capteur télémétrique gauche
    int valeurBrute_g = analogRead(tele_gauche);
    float tension_g = valeurBrute_g * (5.0 / 1023.0); // Calcul de la tension de la valeur récupérée par le capteur gauche
    float distance_g = 13.0 * pow(tension_g, -1); // Estimation empirique de la distance entre le capteur télémétrique gauche et l'adversaire
    // Affichage des résultats sur le moniteur série
    Serial.print("gauche : ");
    Serial.print(distance_g);
    Serial.println(" cm");

    Serial.println();
}

int Capteur_telemetrique_g(){
    // Lecture du capteur télémétrique gauche
    int valeurBrute_g = analogRead(tele_gauche);
    float tension_g = valeurBrute_g * (5.0 / 1023.0); // Calcul de la tension de la valeur récupérée par le capteur gauche
    float distance_g = 13.0 * pow(tension_g, -1); // Estimation empirique de la distance entre le capteur télémétrique gauche et l'adversaire
    // Affichage des résultats sur le moniteur série
    Serial.print("gauche : ");
    Serial.print(distance_g);
    Serial.println(" cm");

    Serial.println();
}

void loop() {
    Capteur_telemetrique_g();
    Capteur_telemetrique_d();

    delay(300); // Pause entre deux mesures
}
```

Figure 3.2.4.3: Code test des capteurs télémétriques

Dans notre code pour le capteur infrarouge, nous faisons appel à quatre fonctions principales :

- `setup ()` : cette fonction initialise la communication série à 9600 bauds, et configuré les capteurs télémétriques en entrée car ils reçoivent les signaux analogiques.
- `Capteur_telemetrique_d()` et `Capteur_telemetrique_g()` : ces fonctions gèrent respectivement le capteur télémétrique droit et gauche. Elles lisent la valeur brute du capteur puis calculent les tensions correspondantes en volt. Elles réalisent ensuite une estimation empirique de la distance en centimètres. Elles affichent, enfin, les valeurs de distances correspondantes.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	64/ 134
----------------------------------	---	------------

Robot Mini-Sumo (RMS)

- `loop()` : cette fonction appelle les deux `fonctionsCapteur_telemetrique_d()` et `Capteur_telemetrique_g()` successivement afin d'effectuer les mesures de distance pour chaque capteur. Une pause de 300 ms est appliquée entre deux séries de mesures afin d'éviter une lecture trop rapide des données.

```
//Fonction du bloc Traitement
void Initialisation(void) {
    while ((PosB != 'G') || (PosB != 'D')) { //Boucle détectant la position
du robot adverse au début du combat, soit à Gauche ('G') soit à Droite
('D') ou coupant la recherche en cas d'appui sur le bouton start de la
télécommande
        if (Lire_Trame_NEC() == 1) {
            break;
        }
        else if (Capteur_Telemetrique(CapteurGauche_Pin) == 1) {
            PosB = 'G';
        }
        else if (Capteur_Telemetrique(CapteurDroite_Pin) == 1) {
            PosB = 'D';
        }
    }
    while (1) { //Boucle permettant au robot de démarrer le combat(break) en
cas d'appui sur le bouton start de la télécommande, ou de faire sonner le
buzzer en cas d'appui sur le bouton attention de la télécommande
        if (Lire_Trame_NEC() == 1) {
            break;
        }
        else if (Lire_Trame_NEC() == 2) {
            Buzzer();
        }
    }
}
```

Figure 3.2.4.4 : Code implémenté des capteurs télémétriques

3.3 Conception détaillée de la partie action

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	65/ 134
----------------------------------	---	------------

3.3.1 Pont en H

Référence de conception : CDT06

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_DEPLACEMENT

3.3.1.1 Schéma électrique du pont en H

L'exigence **EXIG_DEPLACEMENT** stipule que le robot mini-sumo est libre dans ses déplacements. Ainsi, il est capable d'avancer, de tourner à gauche ou à droite.

Afin de répondre à cette exigence nous avons retenu le polulu DRV8835 dès la conception préliminaire, il est capable de contrôler deux moteurs avec une tension entre 2-7V pour la logique et 0-12V pour les moteurs, chaque moteur acceptant 1.2A en continu et 1.5A en courant crête.

Nous avons réalisé des tests complémentaires à ceux effectués lors de la conception préliminaire, afin de vérifier la faisabilité de l'ajout d'un booster de tension. Pour cela, nous avons alimenté un moteur sous une tension de 9 V et mesuré le courant consommé. Les résultats obtenus sur la figure 3.3.1 montrent un courant de crête de 533 mA et un courant continu de 159 mA. Ces valeurs se situent largement dans les plages de courant acceptables par le driver Pololu DRV8835.



Figure 3.3.1 : Courant continu et max consommé par un moteur alimenté avec une tension de 9V

Le pont en H Pololu DRV8835 peut être commandé selon deux modes différents :

Robot Mini-Sumo (RMS)

- le mode IN/IN, qui utilise 4 signaux PWM (deux par moteur) pour contrôler à la fois le sens et la vitesse, offrant une plus grande flexibilité mais nécessitant une programmation plus complexe ;
- le mode PHASE/ENABLE, qui utilise 2 GPIO pour le sens de rotation des moteurs et 2 signaux PWM pour la vitesse, permettant une programmation plus complexe mais avec moins de flexibilité.

Afin de privilégier la simplicité de programmation et de gagner du temps, nous avons choisi le mode PHASE/ENABLE. Selon la datasheet du Pololu DRV8835 (page 4), l'utilisation de ce mode nécessite de placer la broche MODE à l'état haut (HIGH), ce que nous réalisons en la connectant à une sortie GPIO configurée en HIGH.

Nous avons attribué les broches suivantes :

- sorties 9 et 10 pour les signaux PWM des deux moteurs,
- sorties 2, 3 et 11 respectivement pour le contrôle du sens des moteurs et pour maintenir la broche MODE à l'état HIGH.

Enfin, un connecteur 4 broches a été ajouté afin de faciliter le raccordement des deux moteurs au driver DRV8835.

Afin de stabiliser le bruit qu'il peut y avoir dans le pololu nous y ajoutons un condensateur de découplage au plus proche de l'alimentation des moteurs. Puis afin d'avoir une valeur acceptable et réaliste nous avons modifié le PWM des broches PWM en sortie de l'arduino afin d'avoir une fréquence de 10kHz.

D'après l'HTUT n°24 la formule pour calculer la valeur du condensateur de découplage est la suivante :

$$C > \frac{I}{F \Delta V}$$

Nous avons donc un courant I de 200mA pour deux moteurs en continu à 7.4V et une fréquence de 10kHz :

$$C > \frac{\frac{0.2}{10 \cdot 10^3}}{0.01 \cdot 7.4} > 270 \mu F$$

Nous n'avons pas de condensateur à 470uF dans le stock, nous utiliserons donc la valeur la plus proche de 270uF dans le stock qui est de 1000uF .

Le schéma électrique du pont en H est le suivant :

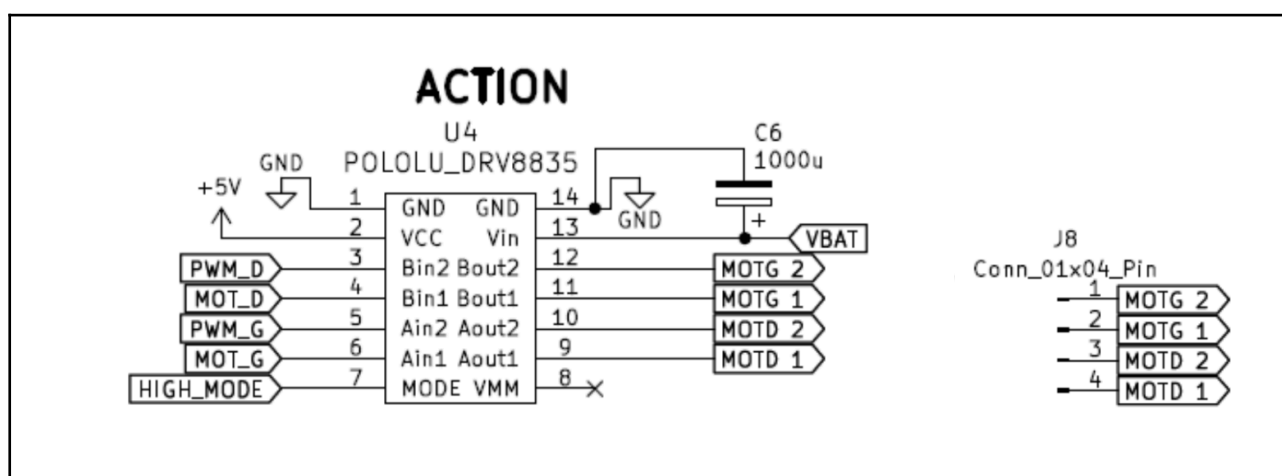


Figure 3.3.2 : Schéma électrique du pont en H

3.3.1.2 Code informatique du pont en H

Nous avons écrit un programme qui permet d'avancer, reculer, tourner et se réorienter si besoin grâce aux commandement du pont en H ainsi qu'au deux motoréducteurs sur le robot. Avant d'implanter ce code dans le fichier principal nous avons créé des fonctions indépendantes afin de les tester sur le robot. Pour cela nous avons conçu un montage de prototypage rapide sur breadboard comme ceci :

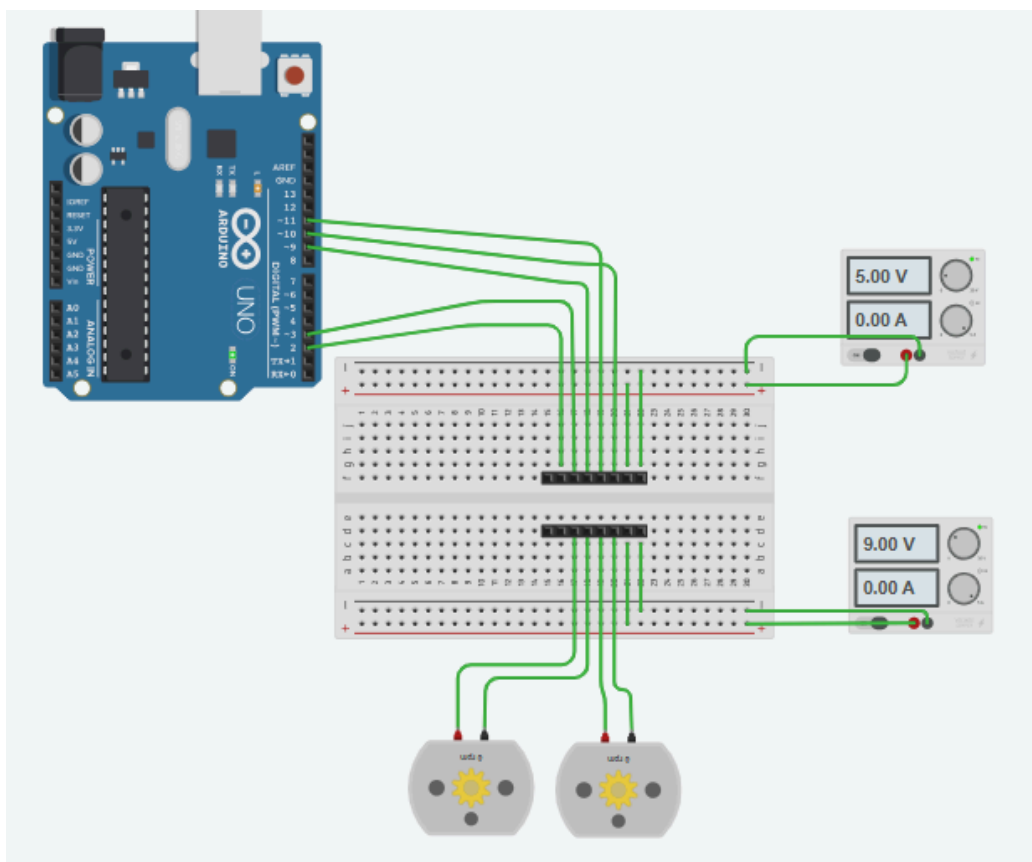


Figure 3.3.3 : Montage du prototypage rapide sur breadboard

Après avoir monté ceci nous avons compilés et transmis à la carte arduino notre code :

```
int Temps_n;
```


Robot Mini-Sumo (RMS)

```
int Temps_n_1;
bool temps = true;
#define MOTG 3
#define MOTD 11
#define PWMG 9
#define PWMD 10

void Turn_itself(int sens, int angle) {
    temps = true;
    if (sens == 'G'){
        if (angle == 0){
            digitalWrite(MOTD,0);
            digitalWrite(MOTG,1);
            analogWrite(PWMD,255);
            analogWrite(PWMG,255);
        }
        else if (angle == 90){
            while (temps){
                Temps_n = millis();
                digitalWrite(MOTD,0);
                digitalWrite(MOTG,1);
                analogWrite(PWMD,255);
                analogWrite(PWMG,255);
                if ((Temps_n-Temps_n_1) > 224) {
                    Temps_n_1 = Temps_n;
                    analogWrite(PWMD,0);
                    analogWrite(PWMG,0);
                    temps = false;
                }
            }
        }
    }
    else if (angle == 180){
        while (temps){
            Temps_n = millis();
            digitalWrite(MOTD,0);
            digitalWrite(MOTG,1);
            analogWrite(PWMD,255);
            analogWrite(PWMG,255);
            if ((Temps_n-Temps_n_1) > 550) {
```

```

        Temps_n_1 = Temps_n;
        analogWrite(PWMD,0);
        analogWrite(PWMG,0);
        temps = false;
    }
}

else if (angle == 180){
    while (temps){
        Temps_n = millis();
        digitalWrite(MOTD,0);
        digitalWrite(MOTG,1);
        analogWrite(PWMD,255);
        analogWrite(PWMG,255);
        if ((Temps_n-Temps_n_1) > 550){
            Temps_n = Temps_n_1;
            analogWrite(PWMD,0);
            analogWrite(PWMG,0);
            temps = false;
        }
    }
}

if (sens == 'D'){
    if (angle == 0){
        digitalWrite(MOTD,1);
        digitalWrite(MOTG,0);
        analogWrite(PWMD,255);
        analogWrite(PWMG,255);
    }
    else if (angle == 90){
        while (temps){
            Temps_n = millis();
            digitalWrite(MOTD,1);
            digitalWrite(MOTG,0);
            analogWrite(PWMD,255);
            analogWrite(PWMG,255);
            if ((Temps_n-Temps_n_1) > 224){

```

```

        Temps_n_1 = Temps_n;
        analogWrite(PWMD,0);
        analogWrite(PWMG,0);
        temps = false;
    }
}
}
else if (angle == 180){
    while (temps){
        Temps_n = millis();
        digitalWrite(MOTD,1);
        digitalWrite(MOTG,0);
        analogWrite(PWMD,255);
        analogWrite(PWMG,255);
        if ((Temps_n-Temps_n_1) > 550){
            Temps_n_1 = Temps_n;
            analogWrite(PWMD,0);
            analogWrite(PWMG,0);
            temps = false;
        }
    }
}
else if (angle == 180){
    while (temps){
        Temps_n = millis();
        digitalWrite(MOTD,1);
        digitalWrite(MOTG,0);
        analogWrite(PWMD,255);
        analogWrite(PWMG,255);
        if ((Temps_n-Temps_n_1) > 550){
            Temps_n_1 = Temps_n;
            analogWrite(PWMD,0);
            analogWrite(PWMG,0);
            temps = false;
        }
    }
}
}

```

```

}

void Reculer(int vitesse) {
    digitalWrite(MOTD, 0);
    digitalWrite(MOTG, 0);
    analogWrite(PWMD, vitesse);
    analogWrite(PWMG, vitesse);
}

void Avancer(int vitesse) {
    digitalWrite(MOTD, 1);
    digitalWrite(MOTG, 1);
    analogWrite(PWMD, vitesse);
    analogWrite(PWMG, vitesse);
}

void Reorienter(int vitesse, char sens) {
    if (sens == 'D') {
        analogWrite(PWMG, vitesse * 0.50);
        analogWrite(PWMD, vitesse);
    }
    if (sens == 'G') {
        analogWrite(PWMD, vitesse * 0.50);
        analogWrite(PWMG, vitesse);
    }
}

```

Figure 3.3.4 : Code informatique du bloc Action

Ce code la est constitué de 4 fonctions principales :

- `Avancer(vitesse)` permet de faire avancer à une vitesse choisie par l'utilisateur. Cette fonction est constituée de deux `analogWrite` pour envoyer le signal PWM associé à la vitesse ainsi que deux `digitalWrite` pour envoyer le sens des moteurs.
- `Reculer(vitesse)` permet comme la fonction avancer mais dans le sens inverse
- `Reorienter(Capteur_Droit, Capteur_Gauche)` prend en paramètre le résultat du traitement des capteurs ultrasons droit et gauche et qui vas si l'un des capteurs ne capte rien baisser la vitesse du moteur associé à la direction afin de se réorienter jusqu'à ce que les deux capteurs captent quelque chose.
- `Turn_itself(sens, angle)` permet au robot de tourner sur lui-même selon une direction `sens` : 'G' pour gauche, 'D' pour droite et un angle spécifique 0, 90 ou 180 degrés. Pour une

rotation, le sens est défini par les broches GPIO MOTD (droite) et MOTG (gauche). La vitesse est fixée au maximum via `analogWrite`. Pour les angles de 90° (224 ms) et 180° (550 ms), un temporisateur `millis()` est utilisé pour assurer la précision avant l'arrêt des moteurs PWM à 0. L'angle de 0° maintient la rotation continue jusqu'à une nouvelle commande ou une interruption.

Les temps de rotation sont expérimentales, le test n'as toujours pas été effectué afin de déterminer sur le robot final le temps nécessaire pour faire un angle de 90° et 180° à une tension fixe de 9V

3.3.2 LED

Référence de conception : CDT07

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : SANS OBJET

3.3.2.1 Schéma électrique de la LED

Nous avons choisi d'ajouter une LED permettant de voir si la carte est sous tension. Nous avons donc choisi la LED ASMT-UBB5-NS8Q2 pour sa disponibilité ainsi que la couleur bleu choisi arbitrairement.

D'après la datasheet de la LED ASMT-UBB5-NS8Q2 nous avons une intensité lumineuse minimum de 224 mcd et maximum de 560 mcd ainsi l'intensité lumineuse typique est de $\frac{224+560}{2} = 392 \text{ mcd}$

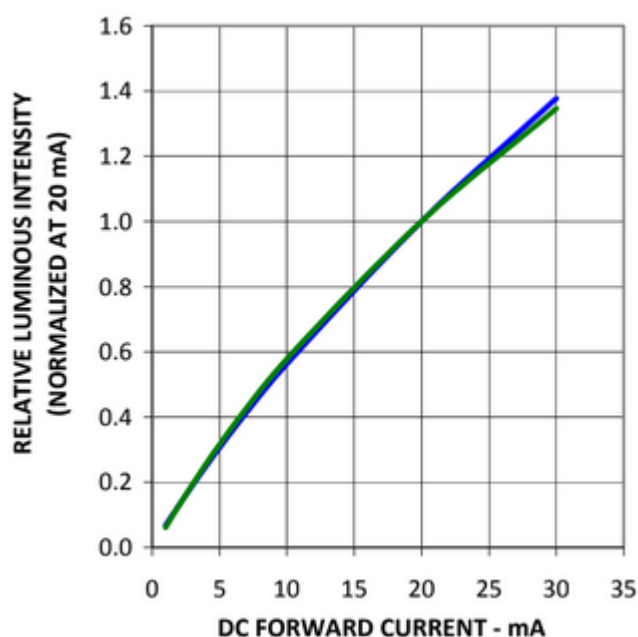


Figure 3.3.5 : Graphe de l'intensité lumineuse relative en fonction du courant

Nous avons une intensité lumineuse relative entre 0 et 1.6 sachant que nous avons une intensité lumineuse à 20 mA à 392 mcd, on choisit pour avoir une luminosité suffisante 100mcd :

$$\frac{100}{392} = 0.25$$

D'après la figure 3.3.3 nous avons donc environ 3mA de consommation pour une intensité lumineuse de 100 mcd (0.25 intensité lumineuse relative)

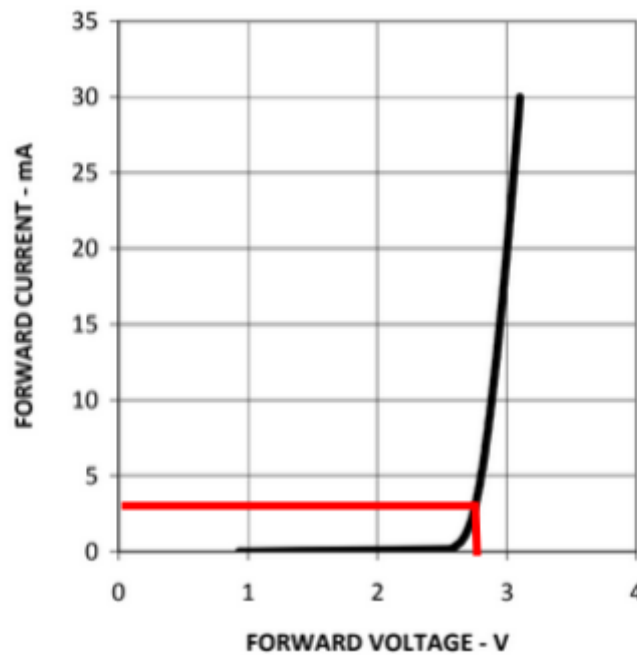


Figure 3.3.6 : Graphe du courant en fonction de la tension

D'après la figure 3.3.4 nous avons une tension de 2.7V pour une consommation de 3mA ainsi en utilisant la formule pour calculer la résistance de la LED nous avons :

$$\frac{V_{cc} - V_{LED}}{3 \cdot 10^{-3}} = 766 \Omega$$

La série E24 de résistances étant la seule disponible dans le stock nous utiliserons une résistance de 750Ω série E24.

Le schéma électrique de la LED est donc la suivante :

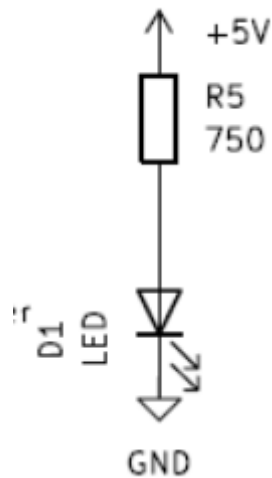


Figure 3.3.7 : Schéma électrique du buzzer

3.3.3 Buzzer

Référence de conception : CDT08

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : SANS OBJET

3.3.3.1 Schéma électrique du buzzer

Afin d'utiliser un buzzer nous devons dimensionner un condensateur afin d'avoir une fréquence de coupure inférieure à 4kHz ($\pm 100\text{Hz}$).

Pour réaliser un filtre passe haut, nous devons suivre cette formule :

$$f_{\text{coupure}} < \frac{1}{2\pi R_{\text{interneB}} C}$$

Nous devons donc déterminer la valeur du condensateur. Nous nous retrouvons donc avec cette formule :

$$C > \frac{1}{2\pi R_{\text{interneB}} f_{\text{coupure}}}$$

Cependant, la valeur de la résistance équivalente R du buzzer MCKPT-G1210-3916 n'est pas renseignée dans sa datasheet. Nous devons donc la calculer selon les informations disponibles. Nous allons donc utiliser la tension de référence de la datasheet et le courant associé à cette tension.

$$U_{\text{rated}} = R_{\text{interneB}} \times I_{\text{max}}$$

$$R_{\text{interneB}} = \frac{U_{\text{rated}}}{I_{\text{max}}} = \frac{12}{2 \times 10^{-3}} = 6000\Omega$$

Avec la résistance équivalente acquise, récupérons maintenant la valeur du condensateur. Nous utiliserons une fréquence de coupure 25% moins grande que celle demandée dans le cahier des charge pour s'assurer une marge de fonctionnement correcte.

$$C > \frac{1}{2\pi \times R_{interneB} \times f_c}$$

$$C > \frac{1}{2\pi \times R_{interneB} \times f_c \times 0.75}$$

$$C > \frac{1}{2\pi \times 6000 \times 4000 \times 0.75}$$

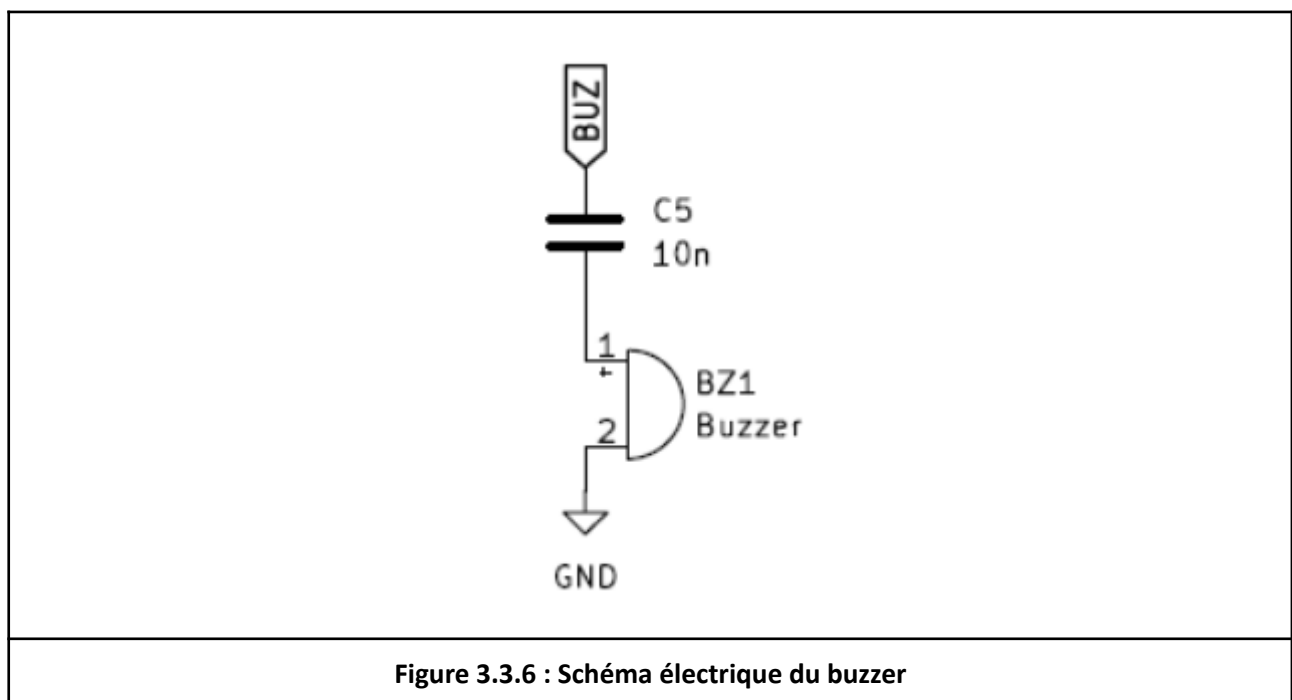
$$C > 6.6 \text{ nF}$$

En normalisant le condensateur (2), nous obtenons le condensateur ECQE2103JB d'une valeur de condensateur de 10nF.

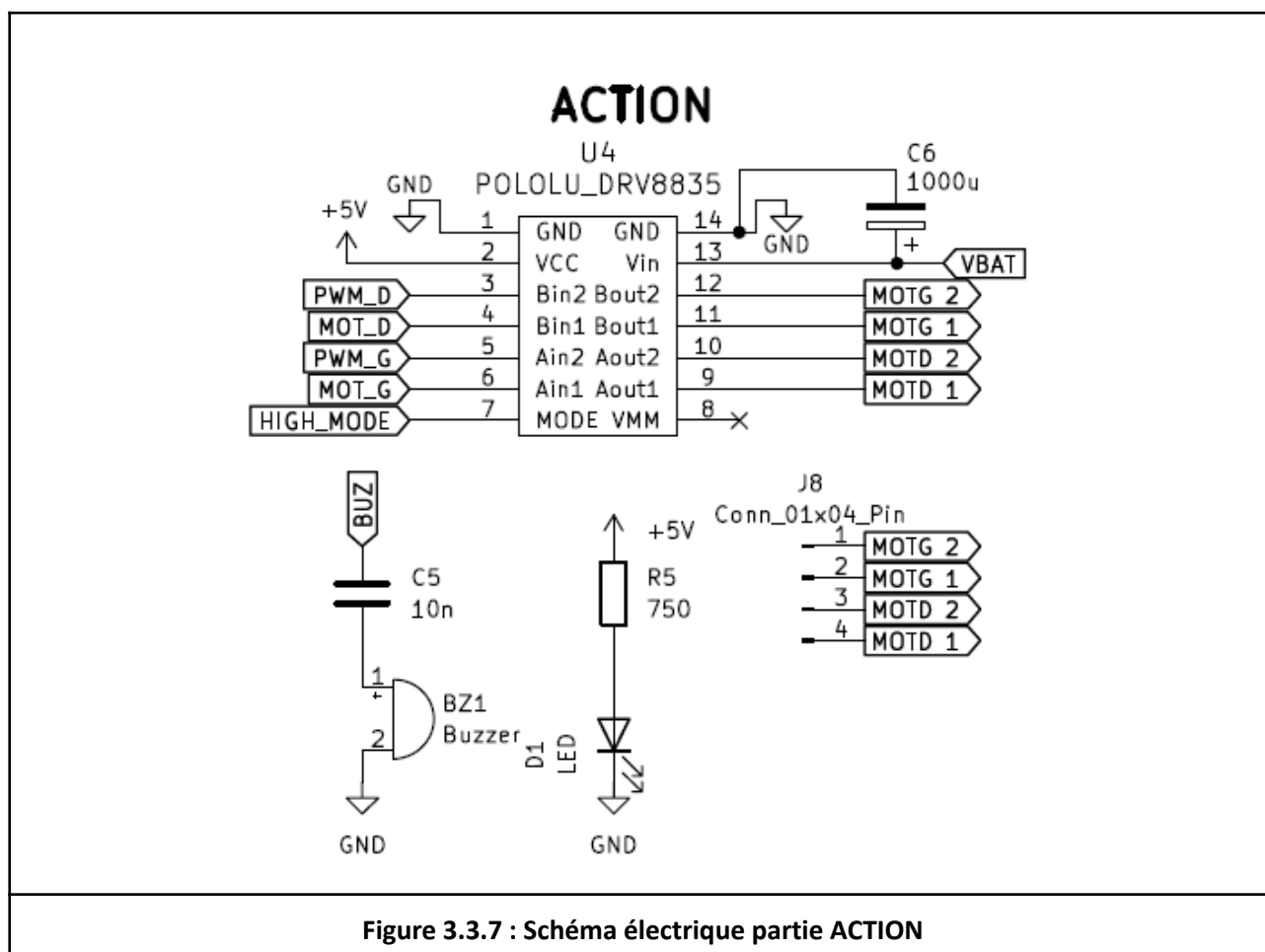
La valeur de la fréquence de coupure avec ce condensateur normalisé est de :

$$f_{coupureNorm} = \frac{1}{2\pi \times 6000 \times 10 \times 10^{-9}} = 2.65 \text{ kHz}$$

Afin de pouvoir jouer n'importe quelle note nous avons connecter le condensateur ainsi que le buzzer à la sortie 5 comme ceci :



Le schéma électrique final de la partie action est donc celui renseigné dans la figure X :



3.4 Conception détaillée de la partie énergie

3.4.1 Pont diviseur de tension pour la génération de la tension de référence

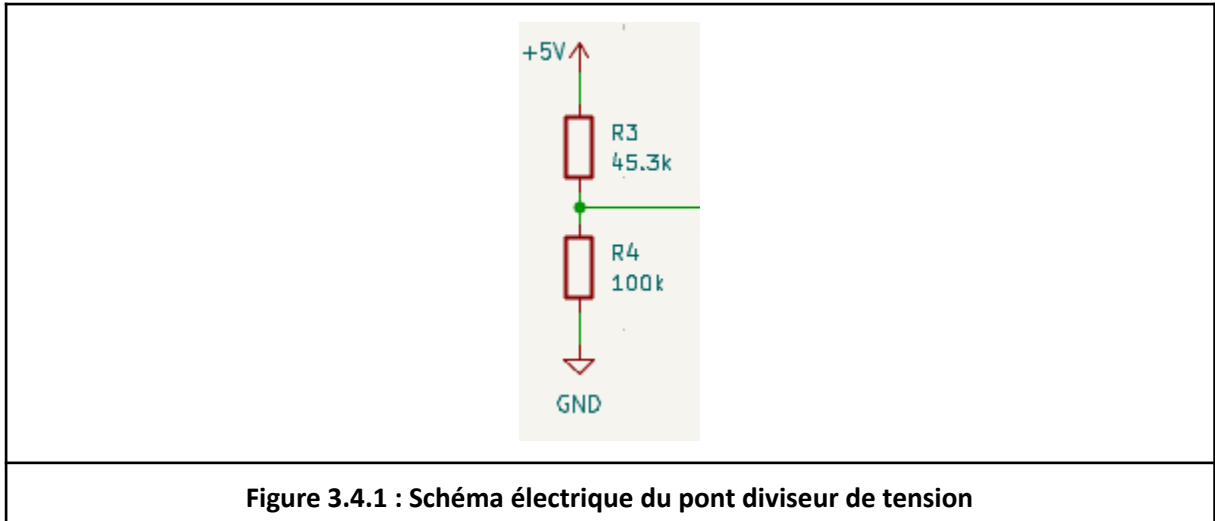
Référence de conception : CDT09

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_SECUR_BATT

3.4.1.1 Schéma électrique du pont diviseur de tension pour la tension de référence

En accord avec la conception préliminaire, un pont diviseur de tension à deux résistances sera utilisé. Son schéma électrique est le suivant (Figure 3.4.1).



Ce pont diviseur de tension permet de créer une tension de référence. Celle-ci sera ensuite comparée à la tension aux bornes de la batterie afin de déterminer si elle dépasse 6,7 V, conformément à l'exigence **EXIG_SECUR_BATT**.

Afin de garantir la stabilité de cette tension de référence, la sortie du régulateur linéaire sera utilisée comme tension d'alimentation du pont diviseur de tension.

La tension à ne pas dépasser étant de 6,7 V, une marge est prise en fixant cette valeur à 6,8 V.

De plus, la tension donnée par le régulateur linéaire étant de 5 V, la tension de référence est fixée à la moitié de 6,8 V.

Ainsi, le pont diviseur de tension est alimenté par une tension de 5 V et crée une tension de sortie de 3,4 V.

Dimensionnement des résistances

Les résistances sont dimensionnées avec une marge de 10 % et une tolérance de 2 %, tout en respectant la règle suivante :

$$I_{OUT} \ll I_{PONT}$$

Avec I_{OUT} , le courant en sortie du pont diviseur de tension et I_{PONT} , le courant le traversant.

$$I_{OUT} \leq 0,1 \times 0,02 \times I_{PONT}$$

D'après la datasheet du MCP6002 (cf. § « 1.0 ELECTRICAL CHARACTERISTICS » - « DC ELECTRICAL SPECIFICATIONS » page 3), le courant de fuite à ses entrées, I_{OUT} , est de 1 pA.

$$I_{PONT} \geq \frac{10}{0,02} \times |I_{OUT}| \Leftrightarrow I_{PONT} \geq \frac{10}{0,02} \times 1 \times 10^{-12} \Leftrightarrow I_{PONT} \geq 5 \times 10^{-10}$$

$$I_{pont} = \frac{V_{OUT}}{R4} \text{ ainsi } \frac{V_{OUT}}{R4} \geq 5 \times 10^{-10}$$

$$\Leftrightarrow \frac{V_{OUT}}{5 \times 10^{-10}} \geq R4$$

$$\Leftrightarrow \frac{3,4}{5 \times 10^{-10}} \geq R4$$

$$\Leftrightarrow R4 \leq 6,8 \text{ G}\Omega$$

Afin d'éviter des perturbations dues à des interactions humaines et de limiter la consommation de courant, la valeur normalisée retenue sera de 100 k Ω .

Ainsi, la valeur normalisée de R4 est de la série E96 100 k Ω ($\pm 1 \%$).



Afin d'obtenir une tension de 3,4 V en sortie du pont diviseur de tension, on souhaite respecter l'équation suivante.

$$V_{PONT} = V_{OUT} \times \frac{R4}{R3+R4}$$

$$\Leftrightarrow \frac{V_{OUT}}{V_{PONT}} = \frac{R3+R4}{R4}$$

$$\Leftrightarrow \frac{V_{OUT}}{V_{PONT}} = \frac{R3}{R4} + 1$$

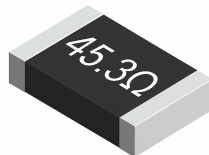
$$\Leftrightarrow R4 = \frac{R3}{\frac{V_{PONT}}{V_{OUT}} - 1}$$

$$\text{soit } \frac{R3}{\frac{V_{PONT}}{V_{OUT}} - 1} = 100 \times 10^3$$

$$\Leftrightarrow R3 = 100 \times 10^3 \times \frac{V_{PONT}}{V_{OUT}} - 100 \times 10^3$$

$$\Leftrightarrow R3 = 100 \times 10^3 \times \frac{5}{3,4} - 100 \times 10^3 = 47,059 \text{ k}\Omega$$

Ainsi, en respectant les stocks disponibles, la valeur normalisée de R3 est de la série E96 47 k Ω ($\pm 1 \%$).



3.4.2 Pont diviseur de tension pour la mesure de la tension de la batterie

Référence de conception : CDT10

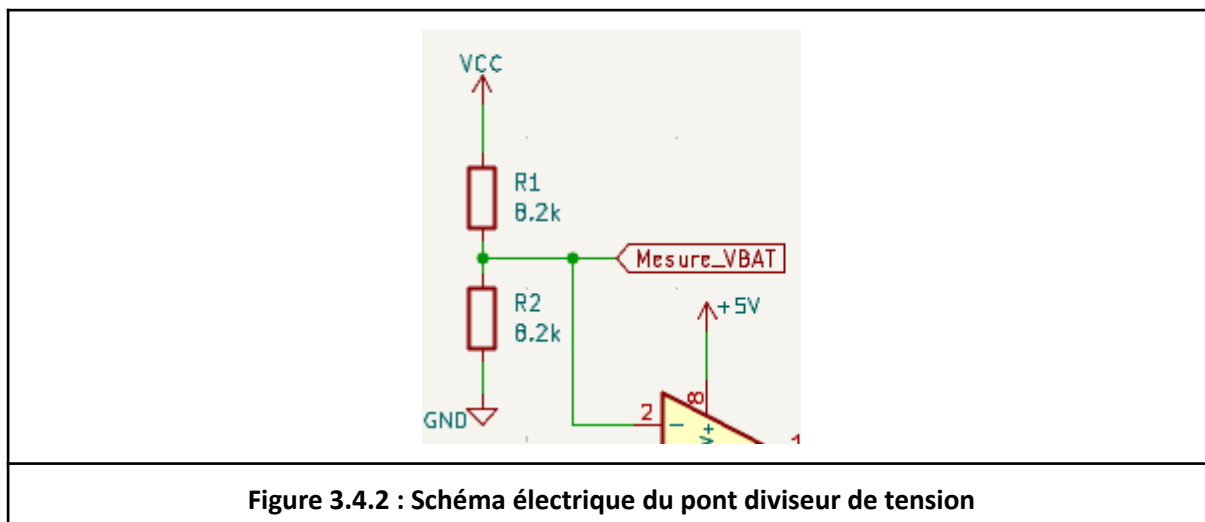
Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_SECUR_BATT

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	79/ 134
----------------------------------	---	------------

3.4.2.1 Schéma électrique du pont diviseur de tension de la batterie

En accord avec la conception préliminaire, un pont diviseur de tension à deux résistances sera utilisé. Son schéma électrique est le suivant (Figure 3.4.2).



Ce pont diviseur de tension permet de diviser par deux la tension de la batterie afin de pouvoir la comparer à la tension de référence.

Cette tension est également envoyée à la carte Arduino afin d'améliorer la précision des déplacements du robot, notamment lors des virages.

Dimensionnement des résistances

Les résistances sont donc dimensionnées avec une marge de 10 % et une tolérance de 2 %, tout en respectant la règle suivante :

$$I_{OUT} \ll I_{PONT}$$

Avec I_{OUT} , le courant en sortie du pont diviseur, et I_{PONT} , le courant le traversant.

$$I_{OUT} \leq 0,1 \times 0,02 \times I_{PONT}$$

D'après la datasheet du MCP6002 (cf. § « 1.0 ELECTRICAL CHARACTERISTICS » - « DC ELECTRICAL SPECIFICATIONS » page 3), le courant de fuite à ses entrées, I_{OUT} , est de 1 pA. D'après la datasheet de l'ATMEGA328P (cf. § "29.2.8 ATmega328P DC Characteristics" page 303), le courant de fuite maximal de ses entrées analogiques est de 1 μ A. Le courant de fuite de l'amplificateur opérationnel est donc négligeable devant celui du microcontrôleur.

$$I_{PONT} \geq \frac{10}{0,02} \times |I_{OUT}| \Leftrightarrow I_{PONT} \geq \frac{10}{0,02} \times 1 \times 10^{-6} \Leftrightarrow I_{PONT} \geq 5 \times 10^{-4} \text{ A}$$

Or, on souhaite que cette condition soit respectée quelle que soit la tension d'alimentation du pont diviseur. Le cas le plus pessimiste étant celui lorsque la tension d'alimentation est maximale. Dans le cas le plus défavorable, la tension d'alimentation est de 8,4 V, ce qui correspond à une tension en sortie de 4,2 V.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	80/ 134
----------------------------------	---	------------

$$I_{pont} = \frac{V_{OUT}}{R2} \text{ ainsi } \frac{V_{OUT}}{R2} \geq 5 \times 10^{-4} \Leftrightarrow \frac{V_{OUT}}{5 \times 10^{-4}} \geq R2 \Leftrightarrow \frac{4,2}{5 \times 10^{-4}} \geq R2 \Leftrightarrow R2 \leq 8,4 \text{ k}\Omega$$

On choisit la résistance la plus grande possible afin de minimiser la consommation de courant, tout en respectant les stocks disponibles.

Ainsi, la valeur normalisée de R2 est de la série E96 8,2 kΩ (± 1 %).

Afin d'obtenir une tension divisée par deux en sortie du pont diviseur, on souhaite respecter l'équation suivante.

$$V_{PONT} = V_{OUT} \times \frac{R2}{R1+R2} \Leftrightarrow \frac{V_{OUT}}{V_{PONT}} = \frac{R1+R2}{R2} \Leftrightarrow \frac{V_{OUT}}{V_{PONT}} = \frac{R1}{R2} + 1 \Leftrightarrow R2 = \frac{R1}{\frac{V_{PONT}}{V_{OUT}} - 1}$$

$$\text{soit } \frac{R1}{\frac{V_{PONT}}{V_{OUT}} - 1} = 8,2 \times 10^3 \Leftrightarrow R1 = 8,2 \times 10^3 \times \frac{V_{PONT}}{V_{OUT}} - 8,2 \times 10^3 \Leftrightarrow R1 = 8,2 \times 10^3 \times \frac{8,4}{4,2} - 8,2 \times 10^3$$

Donc R1 = 8,2 kΩ

Ainsi, la valeur normalisée de R1 est de la série E96 8,2 kΩ (± 1 %).

3.4.3 Régulateur de tension

Référence de conception : CDT11

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_SECUR_BATT

3.4.3.1 Schéma électrique du régulateur de tension

En accord avec la conception préliminaire, un régulateur linéaire MC33269DT-5.0G sera utilisé. On lui ajoutera des condensateurs de découplage afin de garantir le bon fonctionnement de la carte. Le schéma électrique de l'étage régulateur linéaire est le suivant (Figure 3.4.3).

Comverntion VBAT --> +5V

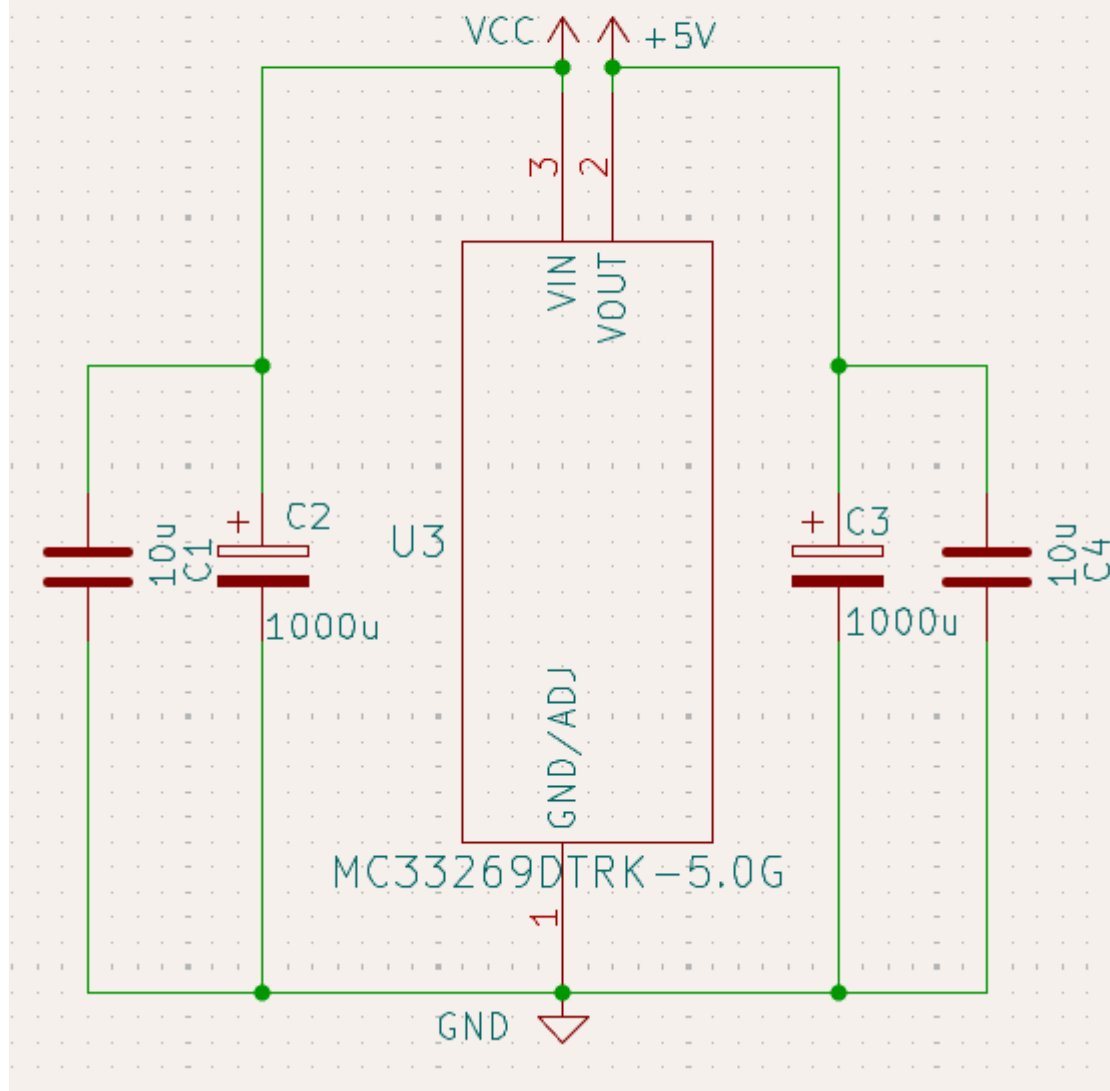


Figure 3.4.3 : Schéma électrique du régulateur de tension

3.4.3.2 Condensateurs de découplage

Dimensionnement des condensateurs

Afin de dimensionner ces condensateurs, il faut connaître la valeur de tous les condensateurs de découplage présents sur la carte. Une fois les capacités des différents modules identifiées, on peut déterminer l'ordre de grandeur de la capacité de filtrage nécessaire en entrée et en sortie du régulateur. Les condensateurs C2 et C3 sont donc dimensionnés de manière à assurer une réserve d'énergie suffisante pour l'ensemble de la carte.

$$C2 = C3 = 2 \times C_{ULTRASON} + C_{INFRAROUGE} + 2 \times C_{TÉLÉMÉTRIQUE} + C_{PONTENH}$$

$$C2 = 2 \times (10 \times 10^{-6} + 100 \times 10^{-9}) + 100 \times 10^{-9} + 2 \times 10 \times 10^{-6} + 1000 \times 10^{-6}$$

$$C2 = 1040 \times 10^{-6}$$

La valeur la plus proche dans les stocks est de 1000 μ F.

Ainsi, les valeurs normalisées de C2 et C3 sont de 1000 μ F ($\pm 10 \%$).

Les condensateurs dimensionnés présentent de bonnes performances pour le filtrage des basses fréquences. En revanche, ils sont moins efficaces au-delà de quelques kilohertz, ce qui peut perturber le fonctionnement de certains composants.

Il est donc nécessaire d'ajouter des condensateurs d'une autre technologie, adaptés aux hautes fréquences. Dans ce cas, des condensateurs polyester C1 et C4 de 10 μ F sont ajoutés.

Ainsi, les valeurs normalisées de C1 et C4 sont de 10 μ F ($\pm 10 \%$).

3.4.4 Comparateur

Référence de conception : CDT12

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_SECUR_BATT

3.4.4.1 Schéma électrique du comparateur

Afin de répondre à l'exigence EXIG_SECUR_BATT, qui stipule que le circuit doit contrôler la tension de la batterie et couper la partie puissance lorsque la tension aux bornes de celle-ci devient inférieure à 6,7 V, nous avons conçu un dispositif de surveillance reposant sur un comparateur analogique.

Ce système permet de détecter l'état de charge de la batterie et de couper automatiquement l'alimentation des moteurs lorsque la tension devient trop faible, évitant ainsi une décharge profonde susceptible d'endommager la batterie LiPo 2S.

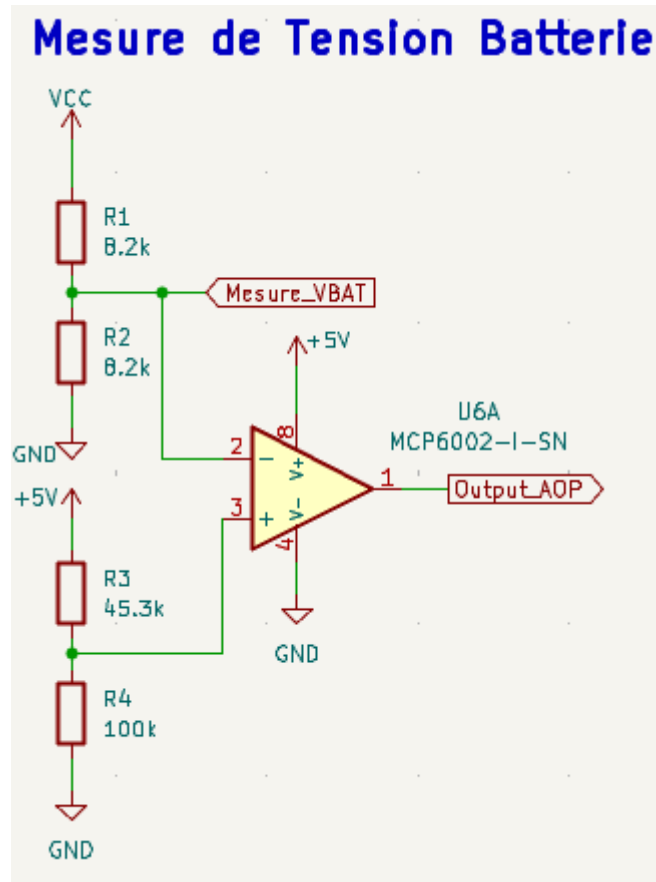


Figure 3.4.4.1 : Schéma électrique du comparateur

Le fonctionnement repose sur deux ponts diviseurs de tension :

- l'un appliqué à la tension de la batterie, pour l'adapter à la plage de mesure du comparateur (0 – 5 V) ;
- l'autre générant une tension de référence fixe, correspondant au seuil de coupure équivalent à 6,7 V.

Une marge de sécurité a été ajoutée afin de garantir un déclenchement fiable malgré les tolérances des composants et les variations de tension.

Le comparateur compare la tension issue du pont diviseur batterie à la tension de référence :

- si la tension de la batterie devient inférieure au seuil de 6,7 V, la sortie du comparateur passe à l'état haut (1) ;
- sinon, elle reste à l'état bas (0).

La sortie du comparateur est reliée à la broche numérique 4 (GPIO) de la carte Arduino UNO. Le programme peut ainsi détecter le passage sous le seuil et, le cas échéant, désactiver le pont en H Pololu DRV8835 pour couper la puissance des moteurs.

Ce dispositif assure la protection de la batterie et garantit la conformité à l'exigence de sécurité énergétique du cahier des charges.

3.4.4.2 Code informatique de l'étage comparateur

Nous avons écrit une fonction permettant de valider l'exigence EXIG_SECUR_BATT. Celle-ci prend en entrée la broche de l'Arduino reliée au comparateur, et renvoie la valeur numérique lue sur cette broche. L'exigence est ensuite respectée dans la partie "main" décrite dans le paragraphe 3.5 du DDC, « Conception détaillée de la partie traitement ».

```
int Etat_batterie(uint8_t pin_lecture_info_batt) {
    return digitalRead(pin_lecture_info_batt);
}
```

Figure 3.4.4.2 : Fonction Etat_batterie

3.4.5 Batterie

Référence de conception : CDT13

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_AUTONOMIE

3.4.5.1 Schéma électrique de la batterie

Afin de répondre à l'exigence EXIG_AUTONOMIE, qui stipule que l'énergie électrique stockée dans le robot mini-sumo doit lui permettre de combattre face à un adversaire pendant au moins 15 minutes, nous avons procédé à une série de mesures de consommation des différents composants et sous-ensembles du robot.

Conformément au cahier des charges, le robot est considéré comme conforme s'il peut se déplacer en continu pendant 65 minutes sans contact avec un obstacle, ce qui garantit une autonomie suffisante en situation réelle de combat.

Nous avons mesuré le courant moyen total consommé par l'ensemble des modules (moteurs, capteurs, Arduino, etc.) afin d'estimer la durée de fonctionnement théorique selon la capacité de la batterie.

Composant / Groupe de composants	Courant consommé (mA)
2 moteurs + pont en H	600 mA (jusqu'à 1 500 mA en crête)
Capteurs (2 ultrasons + 3 capteurs de ligne + 2 télémètres IR)	100 mA

Robot Mini-Sumo (RMS)

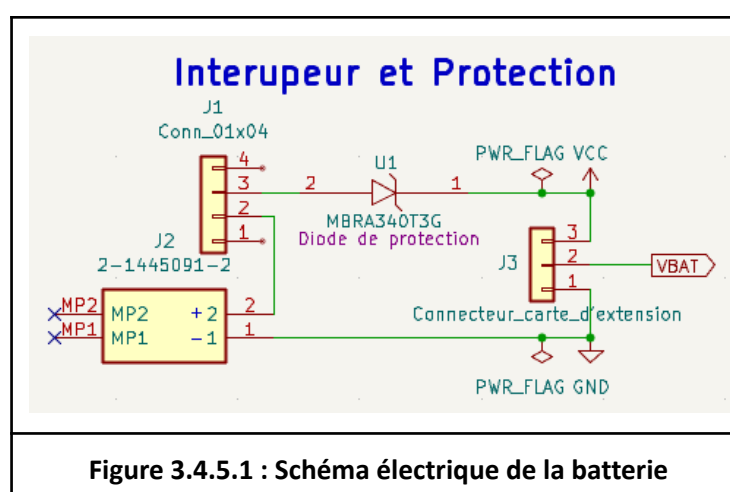
Carte Arduino Uno	100 mA
Autres composants (LED, buzzer, comparateur, régulateur)	100 mA
Total estimé	≈ 900 mA (jusqu'à 1 800 mA en crête)

En fonctionnement normal sans adversaire, la consommation moyenne est d'environ 900 mA, ce qui correspond à une autonomie théorique de près de 53 min avec une batterie de 800 mAh. Cette valeur étant une estimation, la consommation réelle sera inférieure et permettra un fonctionnement d'au moins 1 h.

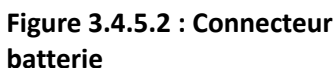
En situation de match, lors des phases d'effort moteur maximal, le courant peut atteindre 1,8 A, mais de manière ponctuelle, ce qui reste compatible avec les performances attendues.

Les résultats obtenus confirment qu'une batterie LiPo 2S de 800 mAh répond aux exigences du cahier des charges.

Les dimensions de cette batterie respectent les contraintes imposées par le bloc batterie, à savoir inférieures à 57 mm × 31 mm × 17 mm.



Pour la connexion, nous avons retenu le connecteur AMP TE Connectivity 2-1445091-2, un modèle SMD muni d'un détrompeur afin d'éviter toute inversion de polarité lors du branchement de la batterie.



3.4.7.1 Schéma électrique de l'interrupteur

Afin de mettre sous tension ou d'éteindre le robot, un interrupteur sera utilisé.

Un connecteur (Breakable male header 1x40 2,54 mm) sera implanté sur la carte pour faciliter son raccordement.

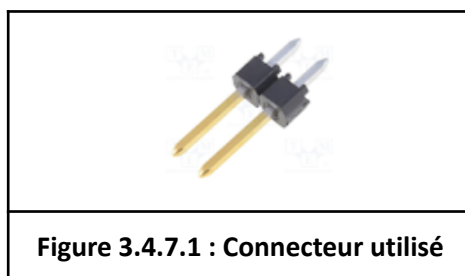


Figure 3.4.7.1 : Connecteur utilisé

L'interrupteur sera placé en série sur la borne positive de la batterie, permettant ainsi de couper ou d'établir l'alimentation générale du robot.

L'ajout d'un détrompeur n'est pas nécessaire, car l'interrupteur fonctionne de la même manière dans les deux sens.

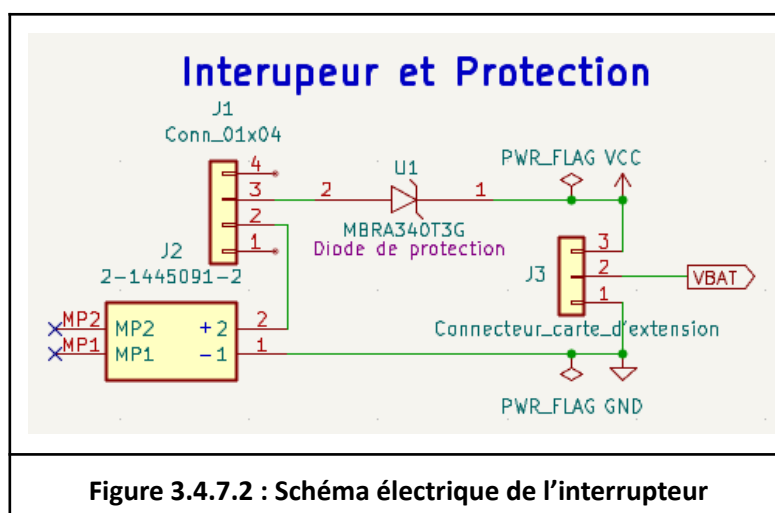


Figure 3.4.7.2 : Schéma électrique de l'interrupteur

3.4.8 Connecteur d'extension

Référence de conception : CDT16

Compétences GEII : C1b 21 22 23 24 25 26

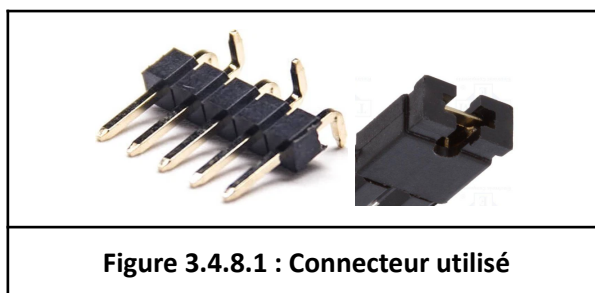
3.4.8.1 Schéma électrique du connecteur extension

Nous avons envisagé la possibilité d'ajouter un composant ou une interface permettant de contrôler la tension d'alimentation du contrôleur moteur Pololu DRV8835 (tension d'alimentation comprise entre 2 V et 11 V).

Pour cela, nous avons choisi d'intégrer un connecteur Dupont 3 broches (Header mâle SMD 2,54 mm 1x3). En utilisation normale, un cavalier sera placé entre les pins 2 et 3, ce qui permettra au module Pololu d'être alimenté directement par la tension de la batterie.

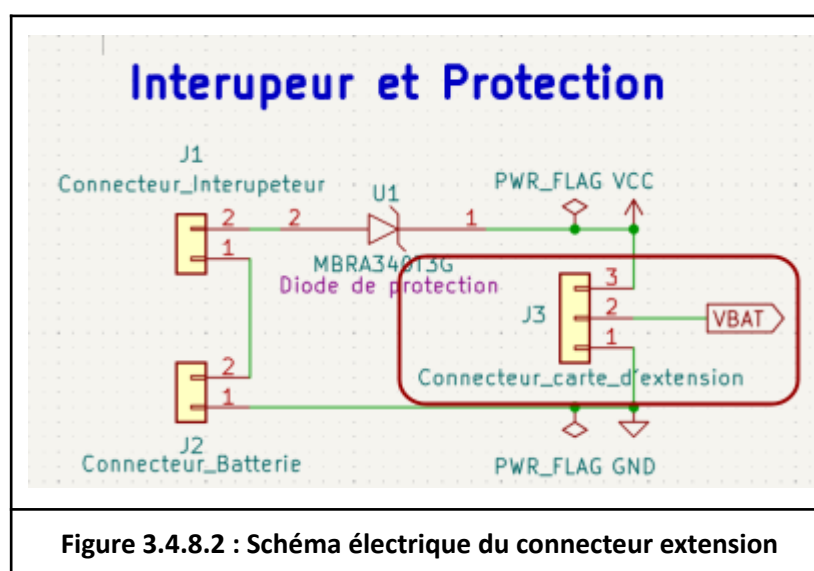
IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	88/ 134
----------------------------------	---	------------

Robot Mini-Sumo (RMS)



Une seconde configuration est également possible consistant à ajouter un booster de tension de 9 V. Celui-ci permettra d'alimenter les moteurs avec une tension constante, indépendante du niveau de charge de la batterie. Cela permet de garantir une vitesse de déplacement stable et potentiellement plus élevée du robot.

Dans ce cas, le booster de tension prélèvera la tension de la batterie sur le broche 3 (entrée) et délivrera une tension réglée de 9 V sur le broche 2 (sortie). Le broche 1 sera relié à la masse commune du système.



3.5 Conception détaillée de la partie traitement

3.5.1 Le microcontrôleur ATMEGA328P-PU

Référence de conception : CDT17

Compétences GEII : C1b 21 22 23 24 25 26

Exigences client vérifiées : EXIG_COMPORTEMENT

3.5.1.1 Schéma électrique du microcontrôleur

Afin de répondre à l'exigence EXIG_COMPORTEMENT, qui stipule que le robot doit être capable de changer de direction de façon autonome en utilisant les données des capteurs et en agissant sur les actionneurs, le traitement est assuré par une carte Arduino UNO intégrant un microcontrôleur ATmega328P-PU.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	89/ 134
----------------------------------	---	------------

Ce microcontrôleur centralise les informations issues des différents capteurs (capteurs de ligne, capteurs d'adversaire avant et latéraux, capteur de télécommande) et traite ces données selon l'algorithme de comportement du robot.

En fonction de ces informations, l'Arduino commande les actionneurs, notamment les moteurs pilotés via le pont en H Pololu DRV8835, afin d'exécuter les déplacements appropriés (avancer, tourner, reculer, s'arrêter).

Ainsi, dès l'appui sur la télécommande de départ, le robot devient entièrement autonome dans ses décisions et ses mouvements sur le dohyo.

3.5.1.2 Code informatique du microcontrôleur

Setup

Voici le code qui permet d'initialiser la carte :

```

void setup() {
  // Configurer Timer1 pour une fréquence PWM d'environ 10 kHz
  TCCR1A = 0; // Réinitialiser le registre TCCR1A
  TCCR1B = 0; // Réinitialiser le registre TCCR1B
  TCCR1A |= (1 << WGM11); // Mode Fast PWM, TOP = ICR1
  TCCR1B |= (1 << WGM12) | (1 << WGM13); // Mode Fast PWM
  TCCR1B |= (1 << CS11); // Prescaler de 8

  // Calcul pour 10 kHz : F_CPU / (Prescaler * (1 + TOP))
  // 16 MHz / (8 * (1 + 199)) = 10 kHz
  ICR1 = 199; // TOP = 199 pour obtenir ~10 kHz

  pinMode(MOTD, OUTPUT);
  pinMode(MOTG, OUTPUT);
  pinMode(PWMD, OUTPUT);
  pinMode(PWMG, OUTPUT);
  pinMode(HIGH_MODE, OUTPUT);
  digitalWrite(HIGH_MODE, 1);
  pinMode(Recepteur_Infrarouge, INPUT); // Configure la broche infrarouge en entrée
  pinMode(CapteurAvantGauche_trig, OUTPUT); // Configuration du port Trigger gauche comme SORTIE
  pinMode(CapteurAvantGauche_echo, INPUT); // Configuration du port Echo gauche comme ENTREE
  pinMode(CapteurAvantDroite_trig, OUTPUT); // Configuration du port Trigger droit comme SORTIE
  pinMode(CapteurAvantDroite_echo, INPUT); // Configuration du port Echo droit comme ENTREE
  pinMode(CapteurGauche_Pin, INPUT);
  pinMode(CapteurDroite_Pin, INPUT);
  pinMode(CapteurLigneAvantGauche_Pin, INPUT);
  pinMode(CapteurLigneAvantDroite_Pin, INPUT);
  pinMode(CapteurLigneArriere_Pin, INPUT);

  irrecv.enableIRin(); // Active la réception IR
  digitalWrite(MOTD, 1);
  digitalWrite(MOTG, 1);
  analogWrite(PWMD, 0);
  analogWrite(PWMG, 0);

  if(Etat_batterie(DigitalBatterie_Pin)==0){
    Initialisation();
    Foncer(PosB);
  }
}

```

Configuration du Timer1 pour le Pololu

Configuration des pins

Configuration IR et moteur

Fonction Initialisation et Foncer

Figure 3.5.1.1 : Programme Setup

La void setup fait appelle aux fonctions « Initialisation() » et « Foncer() »

- **Initialisation**

- Observer la position de l'adversaire
- Attendre le signal de la télécommande IR

```
void Initialisation(void) {
    while ((PosB != 'G') || (PosB != 'D')) {
        if (Lire_Trame_NEC() == 1) {
            break;
        }
        else if (Capteur_Telemetrique(CapteurGauche_Pin) == 1) {
            PosB = 'G';
        }
        else if (Capteur_Telemetrique(CapteurDroite_Pin) == 1) {
            PosB = 'D';
        }
    }
    while (1) {
        if (Lire_Trame_NEC() == 1) {
            break;
        }
        else if (Lire_Trame_NEC() == 2) {
            Buzzer();
        }
    }
}
```

Détection de la position de l'adversaire

Démarrage si la commande NEC « start » est envoyée

Activation du buzzer si la commande nec Attention

Figure 3.5.1.2 : Programme d'initialisation

- **Foncer**

- Rotation du côté de l'adversaire

```
void Foncer(int PositionRobotAdv) {
    Turn_itself(PositionRobotAdv, 90);
    Avancer(255);
}
```

Figure 3.5.1.3 : Programme foncer

Main

```

void loop() {
  while(Broken!= true){
    LigneAvG = Capteur_Ligne(CapteurLigneAvantGauche_Pin);
    LigneAvD = Capteur_Ligne(CapteurLigneAvantDroite_Pin);
    LigneAr = Capteur_Ligne(CapteurLigneArriere_Pin);
    UltrasonG = Capteur_Ultrason(CapteurAvantGauche_trig, CapteurAvantGauche_echo);
    UltrasonD = Capteur_Ultrason(CapteurAvantDroite_trig, CapteurAvantDroite_echo);
    TelemetriqueG = Capteur_Telemetrique(CapteurGauche_Pin);
    TelemetriqueD = Capteur_Telemetrique(CapteurDroite_Pin);
    if(Etat_batterie(DigitalBatterie_Pin)==1 || Lire_Trame_NEC()==3){
      Avancer(0);
      Broken = true;
      break;
    }
    else if((LigneAvG==1) || (LigneAvD==1) || (LigneAr==1)){
      Eviter_ligne();
    }
    else if((UltrasonG==1) || (UltrasonD==1) || (TelemetriqueG==1) || (TelemetriqueD==1)){
      Attaquer();
    }
    else{
      Recherche();
    }
  }
}

```

Actualisation des valeur des capteurs

Vérifier sur état batterie

Detection ligne

Detection Adverser

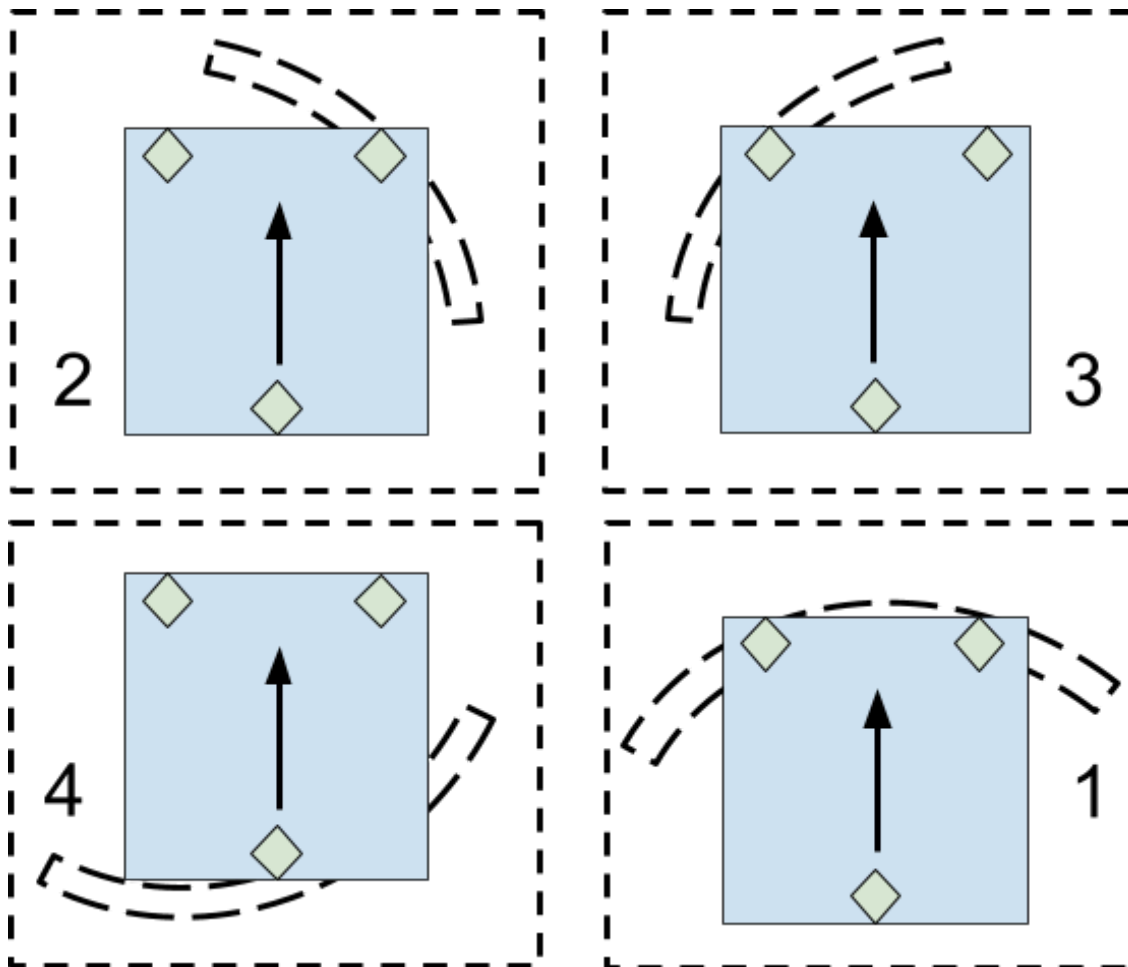
Recherche Adverser

Figure 3.5.1.4 : Programme Loop

Le void loop, correspondant au main, fait appel aux fonctions « Eviter_ligne() », « Attaquer() » et « Recherche() ».

Si l'un des capteurs de ligne détecte une ligne :

- **Eviter_ligne**



- Cas 1 : Les deux capteurs avant détectent une ligne → Le robot tourne à droite de 180°
- Cas 2 : Le capteur avant droit détecte une ligne → Le robot tourne à droite de 135°
- Cas 3 : Le capteur avant gauche détecte une ligne → Le robot tourne à gauche de 135°
- Cas 4 : Le capteur arrière détecte une ligne → Le robot avance tout droit
- Cas 5 : Aucune ligne n'est détectée → Sortie de la fonction sans aucune action

```
void Eviter_ligne(void) { // Fonction permettant au robot de s'éloigner d'une ligne sans tomber
    if ((LigneAvG == 1) && (LigneAvD == 1)) {
        Turn_itself('D', 180);
        Avancer(255);
    }
    else if (LigneAvG == 1) {
        Turn_itself('D', 135);
        Avancer(255);
        delay(500);
    }
    else if (LigneAvD == 1) {
        Turn_itself('G', 135);
        Avancer(255);
        delay(500);
    }
    else if (LigneAr == 1) {
        Avancer(255);
        delay(500);
    }
}
```

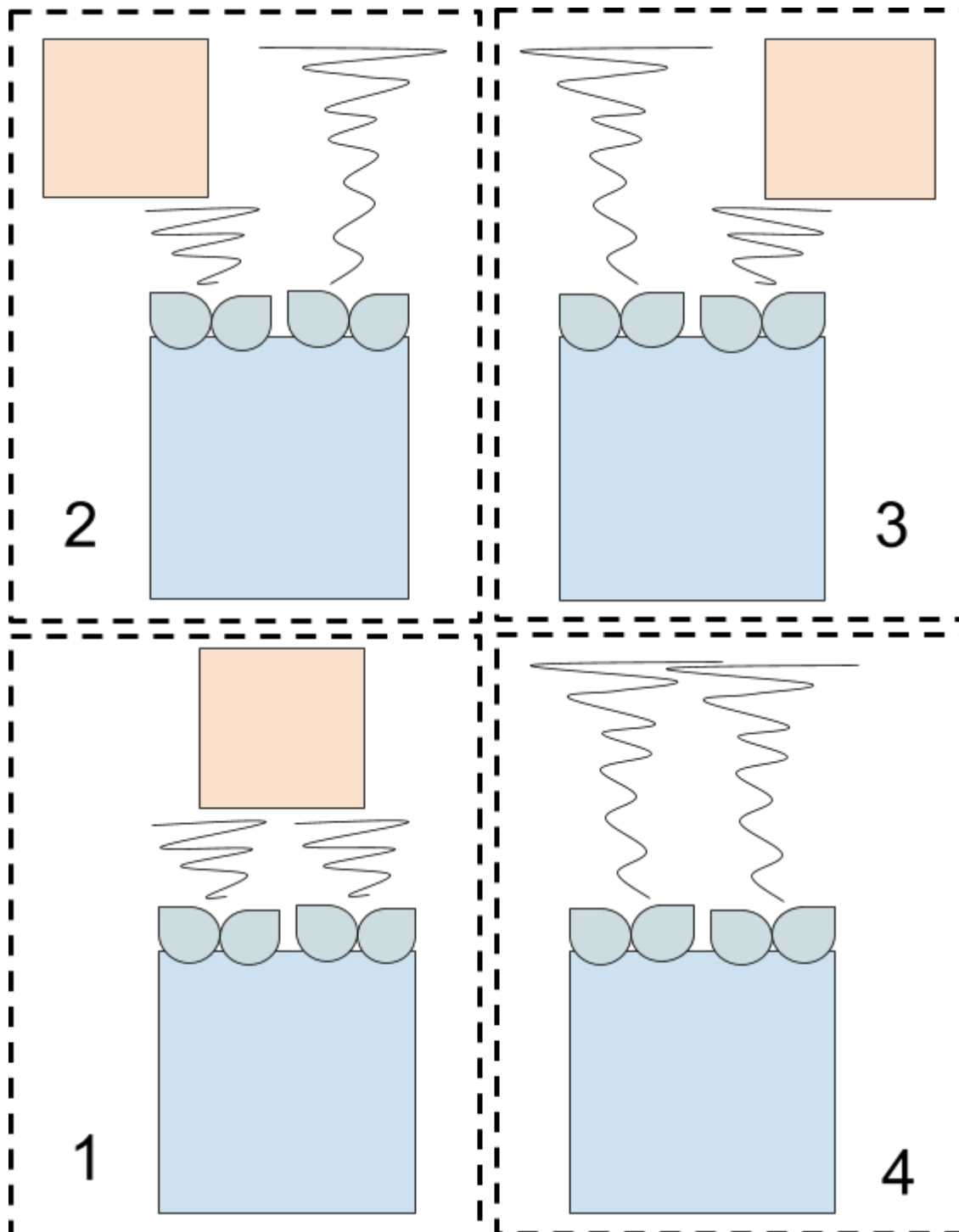
Figure 3.5.1.5 : Programme éviter ligne

Si l'un des capteurs ultrason ou télémétriques détecte un robot :

- **Attaquer**

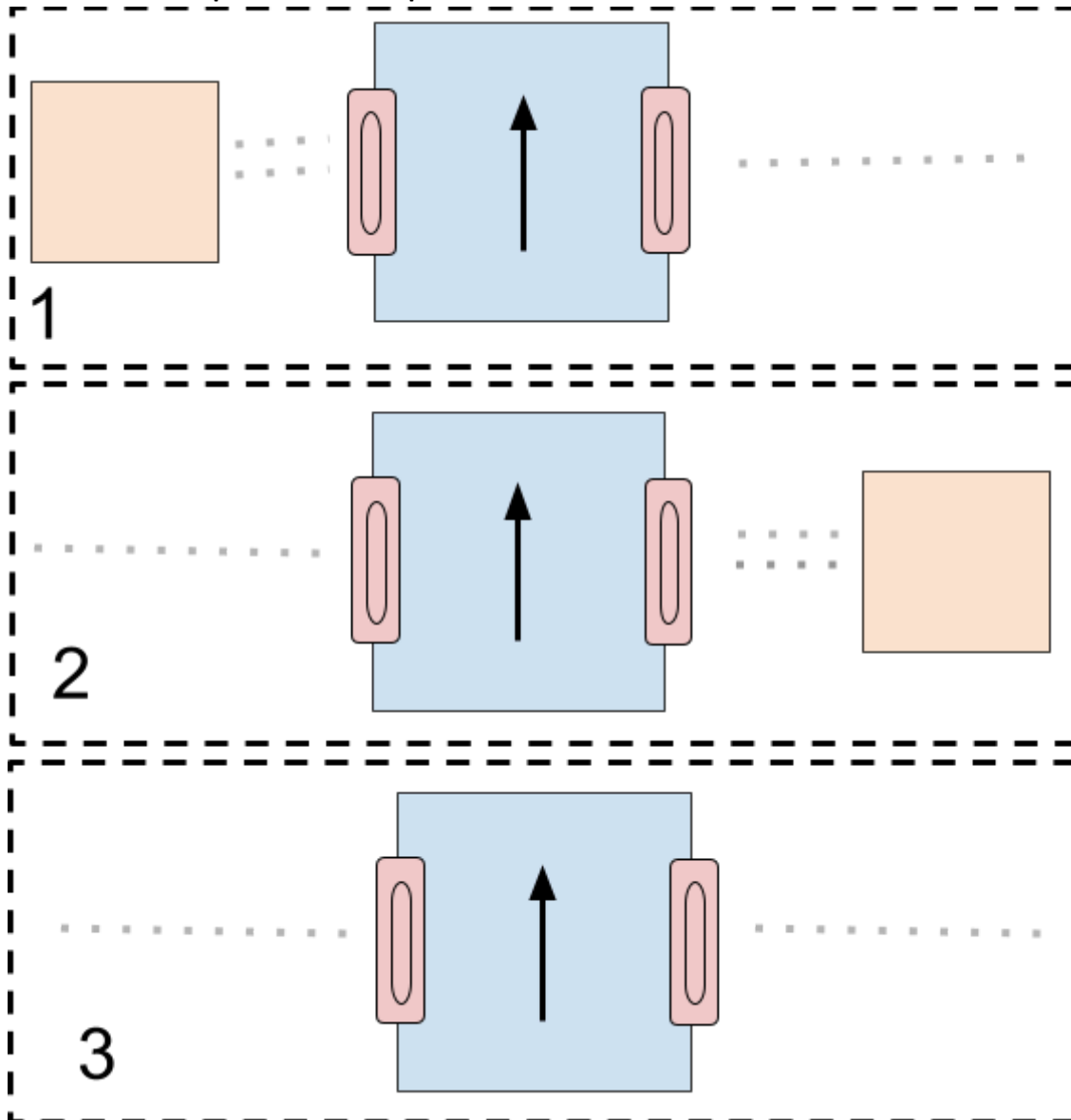
Cas incluant les capteurs ultrasons

Robot Mini-Sumo (RMS)



- Cas 1 : Les deux capteurs avant détectent un robot → Le robot avance tout droit
- Cas 2 : Le capteur avant gauche détecte un robot → Le robot avance en tournant légèrement vers la gauche
- Cas 3 : Le capteur avant droit détecte un robot → Le robot avance en tournant légèrement vers la droite
- Cas 4 : Les capteurs ne détectent aucun robot → Sortie de la fonction sans aucune action

Cas incluant les capteurs télémétriques



- Cas 1 : Le capteur gauche détecte un robot → Le robot tourne à gauche
- Cas 2 : Le capteur droit détecte un robot → Le robot tourne à droite
- Cas 3 : Les capteurs ne détectent aucun robot → Sortie de la fonction sans aucune action

```

void Attaquer(void) {
    if ((UltrasonG == 1) && (UltrasonD == 1)) {
        Avancer(255);
    }
    else if (UltrasonG == 1) {
        Reorienter(255, 'G');
        PosB = 'G';
    }
    else if (UltrasonD == 1) {
        Reorienter(255, 'D');
        PosB = 'D';
    }
    else if (TelemetriqueG == 1) {
        Turn_itself('G', 0);
        PosB = 'G';
    }
    else if (TelemetriqueD == 1) {
        Turn_itself('D', 0);
        PosB = 'D';
    }
}

```

Figure 3.5.1.6 : Programme Attaque

Si le robot se trouve dans aucune des situations précédentes :

- **Rechercher**

Dans ce cas, le robot tourne simplement dans le même sens dans lequel a eu lieu sa dernière rotation.

```

void Recherche(void) {
    Turn_itself(PosB, 0);
}

```

Figure 3.5.1.7 : Programme recherche

Lien vers le code complet : [Drive](#)

3.6 Conception détaillée de coût et de délai

3.6.1 Le coût

Référence de conception : CDT18

Compétences GEII : C1e 17 18 19 20

Exigences client vérifiées : EXIG_COUT

Afin de répondre à l'exigence de coût, nous avons réalisé l'estimation du prix. Étant en phase de conception détaillée, nous connaissons maintenant les composants nécessaires à la réalisation de ce projet. Nous avons donc répertorié cela dans le tableau ci-dessous :

Schematics Reference	Value	Manufacturer Reference	Coût unitaire hors taxes	Quantité	Coût partiel hors taxes	Reseller Reference	Reseller	URL
J9, J10, J15, J3	Sharp_connector IR_connector Carte_extensibil-conductor	SSM-103-4-SV	1,44 €	4	5,76 €	2779564	Farnell	https://fr.farnell.com/samtec/ssm-103-4-sv/connect-femelle-3-voies-1-rang/dp/2779564
J1, J2	Interrupteur_connector batterie_connector	SSM-102-4-SV	1,17	2	2,34 €	1668243	Farnell	https://fr.farnell.com/samtec/ssm-102-4-sv/embase-2-54mm-vert-2voies/dp/1668243
J14, J13, J11, J12, J8	Motor_connector ONY_connector - Ultraasm_connector	SSM-104-4-SV	1,70	5	8,50 €	1668251	Farnell	https://fr.farnell.com/samtec/ssm-104-4-sv/embase-2-54mm-vert-4voies/dp/1668251
R6	100	ASCD402-100RFT10	0,10	1	0,10 €	2078854	Farnell	https://fr.farnell.com/t-electronics-weelwn/asc402-100rft10/res-couche-epaisse-100u-1-0-063w/dp/2078854
C7, C10, C11	100n	C1210V104KCRACU	0,23	3	0,68 €	2085201RL	Farnell	https://fr.farnell.com/kemet/c1210v104kcractucond-0-1-4-500v-10-x7-12104p/2085201RL
U4	POKULU	POKULU_QW0835	7,70	1	7,70 €	32766	Gomonic	https://www.gomonic.fr/art-commande-de-2-modules-couleur835-2x1-2p-31715.htm
B21	BUZZER	MOXPYT-GL210-3016	0,76	1	0,76 €	2309143	Farnell	https://fr.farnell.com/multicomp-pro/mxpkyt-1210-3016/5voies-buzzers/dp/2309143
C5	10n	C1608K79H104K080AA	0,03	1	0,03 €	1907315	Farnell	https://fr.farnell.com/kdc/c1608k79h104k080acond-0-01-500v-10-x7-0803/dp/1907315
R5	750	ERUB2AF751V	0,06	1	0,06 €	3782115	Farnell	https://fr.farnell.com/panasonic/erub2af751v/res-acc-200-750v-0-7w-0812dp/3782115
C6, C2, C3	1000u	EEET1A102GP	0,50	3	1,51 €	2612061RL	Farnell	https://fr.farnell.com/panasonic/eeet1a102gp/condensateur-1000-f-10v-cms/dp/2612061RL
C12, C8, C9, C13	10u	COASLX7V1E104K150AC	0,10	2	0,20 €	2458327	Farnell	https://fr.farnell.com/kdc/coaslx7v1e104k150acond-1-1-5-25v-10-x7-1500-bobine/dp/2458327
D1	LED	ASMT-4885-N3803	0,23	1	0,23 €	2897148	Farnell	https://fr.farnell.com/bradcom/asmt-4885-n3803/led-cms-plcc-3-illume-bleu/dp/2897148
U1	Diode protection	NMBA340T3G	0,47	1	0,47 €	1431078RL	Farnell	https://fr.farnell.com/on-semiconductor/nmbs3403p/diode-photoly-3a-40v-sma/dp/1431078RL
R1, R2	8.2k	MP000533	0,02	2	0,04 €	3213456	Farnell	https://fr.farnell.com/multicomp-pro/mp000533/res-8k2-1-0-1w-0402-couche-epaisse/dp/3213456
R3	45.3k	ERUBENF4532V	0,02	1	0,02 €	2398892	Farnell	https://fr.farnell.com/panasonic/erubenf4532v/res-couche-epaisse-45-3k-1-0-25w/dp/2398892
R4	100k	CRW0503100KFAEA	0,02	1	0,02 €	1469649	Farnell	https://fr.farnell.com/vishay/crw0503100kfaea/res-couche-epaisse-100k-1-0-1w/dp/1469649
U2A	ACP	MC9802-4-SN	0,37	1	0,37 €	1392345	Farnell	https://fr.farnell.com/microchip/mc9802-4-sv/ajust-prec-limite-40-a-85-c-couche/dp/1392345
U3	Regulateur linéaire 5V	MC332080T8K-5.0G	0,25	1	0,25 €	2534202	Farnell	https://fr.farnell.com/onsemi/mc332080t8k-5-0p/ldo-five-5v-0-8a-30-30p/dp/2534202
J6	Connector	SSM-110-4-SV	2,56	1	2,56 €	1668264	Farnell	https://fr.farnell.com/samtec/ssm-110-4-sv/embase-2-54mm-vert-1voies/dp/1668264
J4, J7	Connector	SSM-108-4-SV	2,53	2	5,06 €	1668259	Farnell	https://fr.farnell.com/samtec/ssm-108-4-sv/embase-2-54mm-vert-8voies/dp/1668259
J5	Connector	SSM-106-4-SV	1,97	1	1,97 €	1668257	Farnell	https://fr.farnell.com/samtec/ssm-106-4-sv/embase-2-54mm-vert-8voies/dp/1668257
C1, C4	4.7u	CGAGMKT52A475K20GAB	0,39	2	0,77 €	2458354	Farnell	https://fr.farnell.com/kdc/cgagmkt52a475k20gabcond-4-7-100v-10-x7-1210-bobine/dp/2458354
Capturur télémetrique	/	Sharp GP70M41580F	9,90 €	2	19,80 €	24576	Gomonic	http://www.gomonic.fr/art-capturur-de-temper-sharp-gp70m41580f.htm
Capturur ultrasons	/	HC-SR04	3,90	2	7,80 €	32454	Gomonic	http://www.gomonic.fr/art-module-de-detection-usb-arduino-20912.htm?r=1&id=Afm8OcpmOP7Uv7DcDQ3ouP
Capturur de ligne	/	ONY70	0,76	3	2,28 €	1470063	Farnell	https://fr.farnell.com/vishay/cny70/optocommutateur-reflectif-5mm/dp/1470063?r=1&id=Afm8OcpmOP7Uv7DcDQ3ouP
Capturur IR	/	TSOP 18338	0,78	1	0,78 €	2889954	Farnell	https://fr.farnell.com/vishay/tsop18338/recepteur-ir-24m-0-12mw-m2-lateral/dp/2889954?r=1&id=Afm8OcpmOP7Uv7DcDQ3ouP
Pack d'accumulateur	Lipo 7.4 V 800 mAh		13,49 €	1	13,49 €	239060 - 62	conrad	http://www.conrad.fr/ta/fr/produit/239060/Pack-depuis-Lipo-CONRAD-ENERGY-7.4-V-800-mAh
Roue	GMFW noire pour moteurs série GM		4,40 €	2	8,80 €	25382	gotronic	http://www.gotronic.fr/art-coupe-gmfw-noire-pour-moteurs-serie-gm-32013.htm
Lot de 5 câbles Grove 20 cm	/		2,91 €	5	14,75 €	31424	gotronic	http://fr.5.cables-grove-20-cm-360006
Connecteur Grove droit	/		0,15 €	10	1,50 €	31232	gotronic	http://www.gotronic.fr/art-connecteur-grove-droit-20571.htm
Connecteur Grove courbé	/		0,15 €	10	1,50 €	31234	gotronic	http://www.gotronic.fr/art-connecteur-grove-courbe-20572.htm
Interrupteur	Interrupteur à bascule Noir Unipolaire à une direction (NOC) Verrouillable		2,17 €	1	2,17 €	706 4551	radioparc	http://fr.rs-online.com/web/a/Interrupteurs-a-bascule/7064551
10 entretoises 5mm	/		0,80 €	2	1,60 €	11540	gotronic	http://www.gotronic.fr/art-10-entretoises-5mm-134.htm
10 vis M3 20mm	/		0,65 €	2	1,30 €	11521	gotronic	http://www.gotronic.fr/art-10-vis-acier-m3-20mm-6008.htm
10 vis M4 20mm	/		0,65 €	2	1,30 €	11522	gotronic	http://www.gotronic.fr/art-10-vis-acier-m4-20mm-6009.htm
10 vis auto taraudeuses 2,9x6,5mm	/		0,10 €	2	0,20 €	11553	gotronic	http://www.gotronic.fr/art-10-vis-auto-taraudeuses-2-9x6-5mm-6084.htm
10 vis auto taraudeuses 2,9x8mm	/		0,15 €	2	0,30 €	11554	gotronic	http://www.gotronic.fr/art-10-vis-auto-taraudeuses-2-9x8mm-6085.htm
10 vis auto taraudeuses 2,9x13mm	/		0,65 €	2	1,30 €	11555	gotronic	http://www.gotronic.fr/art-10-vis-auto-taraudeuses-2-9x13mm-6086.htm
			Coût total hors taxes du projet		59,31			
			Coût total TTC Matières 1ères du projet		71,17			

Figure 3.6.1 : Tableau du coût total du projet

D'après les estimations le coût total du projet est de **59€31 HT** (71€17 TTC), soit un coût bien inférieur à 120 € HT.

3.6.2 Le délai

Référence de conception : CDT19

Compétences GEII : C1e 17 18 19 20

Exigences client vérifiées : EXIG_DELAI

Le projet de développement du robot mini sumo (phase de conception + phase de fabrication + phase de vérification + phase de rédaction + phase de présentation/démonstration) respecte l'exigence des 84 heures allouées, grâce à une gestion efficace du temps et du suivi du planning de développement ci dessous :

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	98/ 134
----------------------------------	---	------------

Robot Mini-Sumo (RMS)

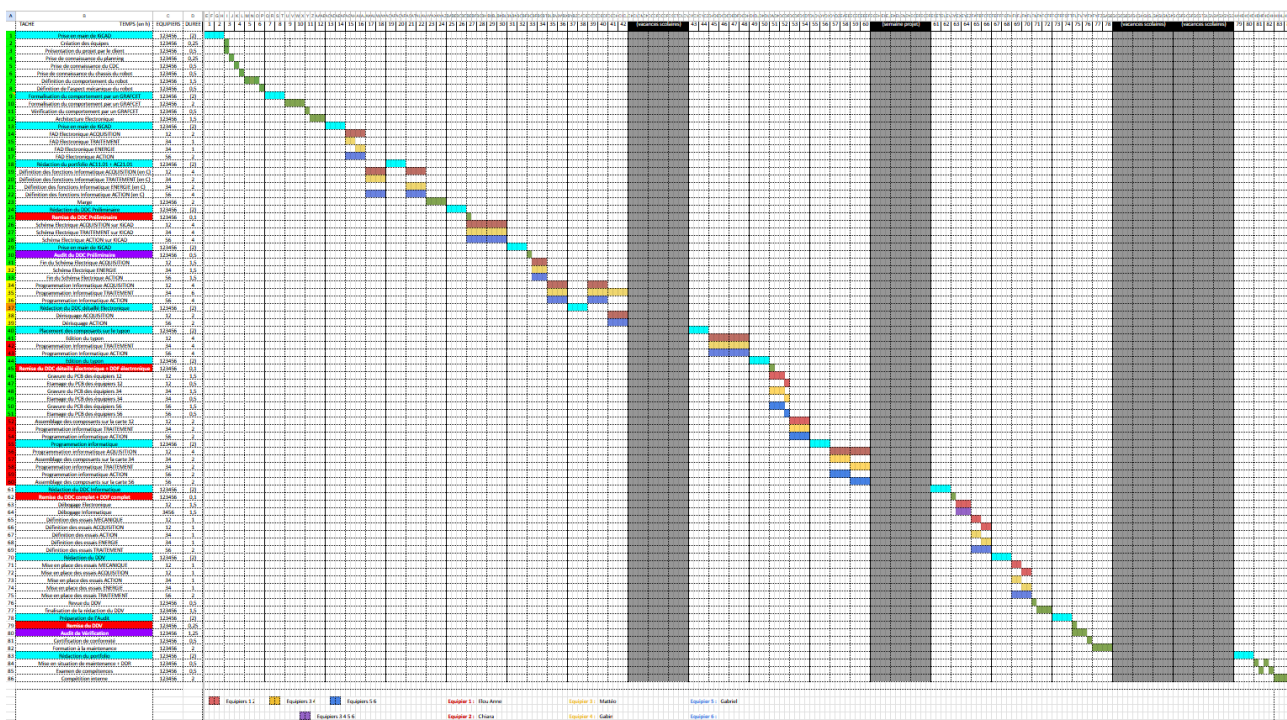


Figure 3.6.2 : Planning de développement

4. Dérivage des solutions techniques retenues

Ce chapitre détaille les activités de dérisquage des solutions techniques retenues : simulation et/ou prototypage rapide. Il constitue une preuve partielle de la conformité du produit. Chaque paragraphe de l'étude fait donc clairement référence aux exigences client issues du [CDC].

Il permet également de confirmer les résultats théoriques effectués aux paragraphes 2 et 3 en vérifiant le fonctionnement à travers des simulations et/ou prototypages rapides. Pour chaque simulation et/ou prototypage rapide est renseigné le protocole de mise en œuvre. Les résultats des simulations et/ou prototypages rapides sont confrontés aux résultats de l'étude théorique.

L'ensemble des fichiers est disponible dans le dossier : renseignez ici le chemin du dossier où sont situés les fichiers de simulation et/ou prototypage rapide du projet.

Acquisition

4.1 Simulation du Capteur infrarouge

Référence de la simulation : SIM Capteur IR

Exigences client vérifiées : EXIG DEPART

But de l'essai :

L'objectif de cet essai est de vérifier le bon fonctionnement du capteur infrarouge. Lorsque l'on appuie sur un bouton de la télécommande, une trame NEC correspondant au numéro du bouton est émise. Le capteur infrarouge doit donc être capable de recevoir correctement cette trame.

Fichiers : [Schéma électrique Capteur IR](#) et [code informatique](#)

Procédure de simulation :

Pour vérifier le capteur infrarouge, câblez-le sur breadboard selon le schéma : VCC à 5 V, GND à masse, OUT (6) Arduino et sonde oscilloscope ; intégrez R1 et C1 optimisés.

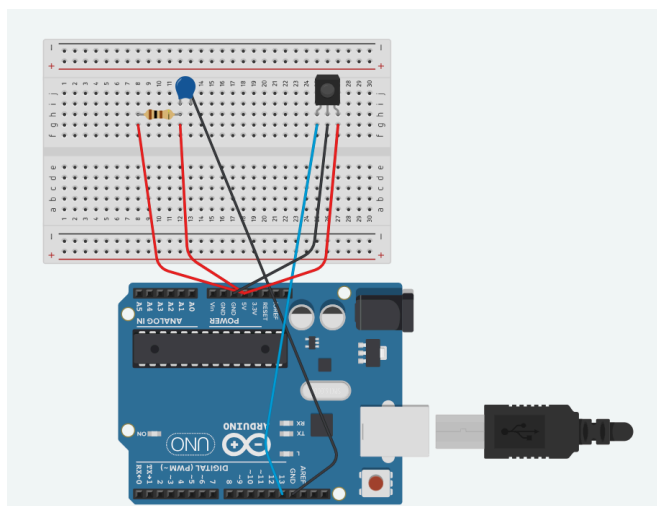


Figure 4.1.1: Schéma de câblage du capteur infrarouge

Configurez l'oscilloscope et téléversez le code de test et ouvrez le moniteur série (115200 bauds).

Testez à 20–30 cm : Décodez les trames obtenus sur l'oscilloscope afin d'obtenir ceci :

- bouton 1 → 0xFF30CF
- bouton 2 → 0xFF18E7
- bouton 3 → 0xFF7A85
- pas d'appuie → aucune trame.

Résultats attendus :

A partir des exigences client issues du Cahier Des Charges, renseignez les valeurs attendues ci-dessous.

Grandeur	Valeur attendue	Tolérance
Trame NEC Lors de l'appui sur un bouton 1	0xFF30CF	/
Trame NEC Lors de l'appui sur un bouton 2	0xFF18E7	/
Trame NEC Lors de l'appui sur un bouton 3	0xFF7A85	/
Aucune Trame NEC lorsqu'aucun bouton est appuyé	/	/

Résultats obtenus :

Au cours de la phase de test, nous avons d'abord effectué le câblage du circuit, conformément au schéma présenté précédemment (voir Figure x), sur une breadboard. Ensuite, une sonde, connectée à un oscilloscope, a été placée sur la broche OUT du capteur infrarouge pour analyser le signal, comme illustré

ci-dessous :

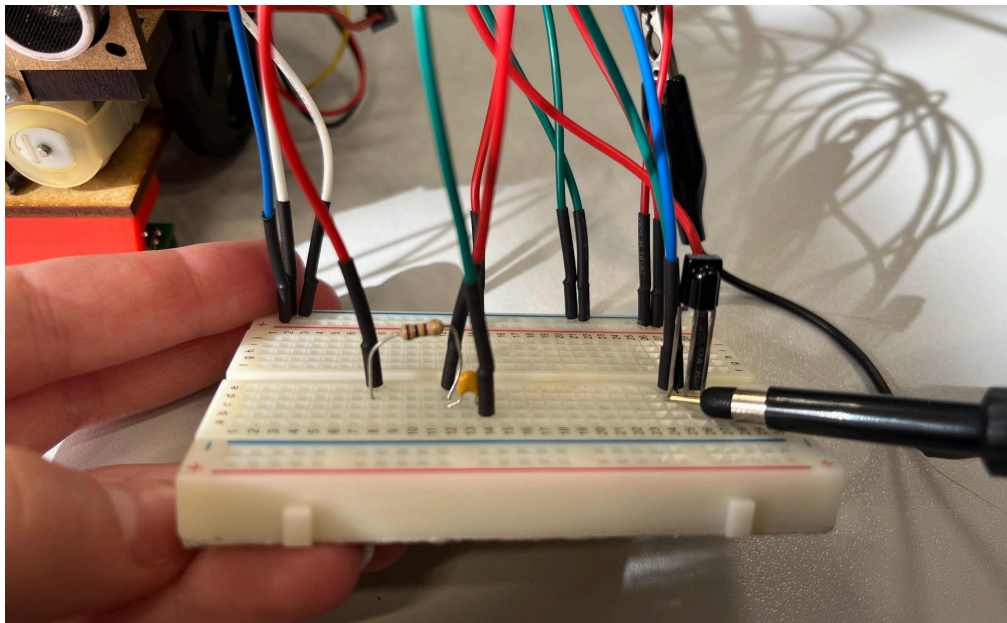


Figure 4.1.2: Câblage réel du capteur infrarouge

Une fois le montage réalisé, la carte Arduino a été connectée à l'ordinateur afin de téléverser le code de test. Ce programme permet de récupérer la trame NEC générée lors de l'appui sur un bouton de la télécommande infrarouge et de déterminer quelle bouton la trame correspond :

Robot Mini-Sumo (RMS)

```
#include <arduino.h>
#include <IRremote.h> // librairie de la télécommande infrarouge

#define Recepteur_Infrarouge 6 // broche du signal infrarouge
#define Adresse_NEC_1 0xFF30CF // Trame associée au bouton START
#define Adresse_NEC_2 0xFF18E7 // Trame associée au bouton ATTENTION
#define Adresse_NEC_3 0xFF7A85 // Trame associée au bouton STOP

IRrecv irrecv(Recepteur_Infrarouge); // associe la réception infrarouge à la broche 13 " Recepteur_Infrarouge"
decode_results Valeur; // décoder la trame NEC reçu

void setup() {
  pinMode(Recepteur_Infrarouge, INPUT); // Configure la broche infrarouge en entrée
  Serial.begin(115200); // Initialiser la communication série
  irrecv.enableIRIn(); // Active la réception IR
}

int Lire_Trame_NEC(){
  if(irrecv.decode(&Valeur)){
    Serial.println(Valeur.value);
    switch(Valeur.value){ // si le récepteur reçoit une trame alors 3 choix possibles

      case Adresse_NEC_1: //Dans le cas où le bouton 1 (START) est appuyé
        Serial.println("Le bouton START est appuyé"); // Afficher sur le moniteur série "Le bouton START est appuyé"
        break;

      case Adresse_NEC_2: //Dans le cas où le bouton 2 (ATTENTION) est appuyé
        Serial.println("Le bouton ATTENTION est appuyé"); // Afficher sur le moniteur série "Le bouton ATTENTION est appuyé"
        break;

      case Adresse_NEC_3: //Dans le cas où le bouton 3 (STOP) est appuyé
        Serial.println("Le bouton STOP est appuyé");// Afficher sur le moniteur série "Le bouton STOP est appuyé"
        break;

    }
    irrecv.resume(); // Préparer le récepteur pour le prochain signal
  }
}

void loop() {
  Lire_Trame_NEC();
}
```

Figure 4.1.3: Code pour le capteur IR afin d'observer une trame à l'oscilloscope

Nous avons ensuite testé le module avec un émetteur IR avec la télécommande. Nous avons connecté la broche OUT à un oscilloscope.

Suite à cela on observe un signal de sortie net et cohérent (passant de $VOL \approx 0$ mV à $VOH \approx VS$) sans pulsations parasites. Ceci signifie que tous les bruits ont filtré et que les valeurs de R_1 et C_1 sont bonnes.

Trame obtenu lors de l'appuie sur le bouton START :

Lors de l'appuie sur le Bouton 1 correspondant à la commande Start, la trame NEC envoyé est :
0X FF30CF en hexadécimal → 1111 1111 0011 0000 1100 1111 en binaire.

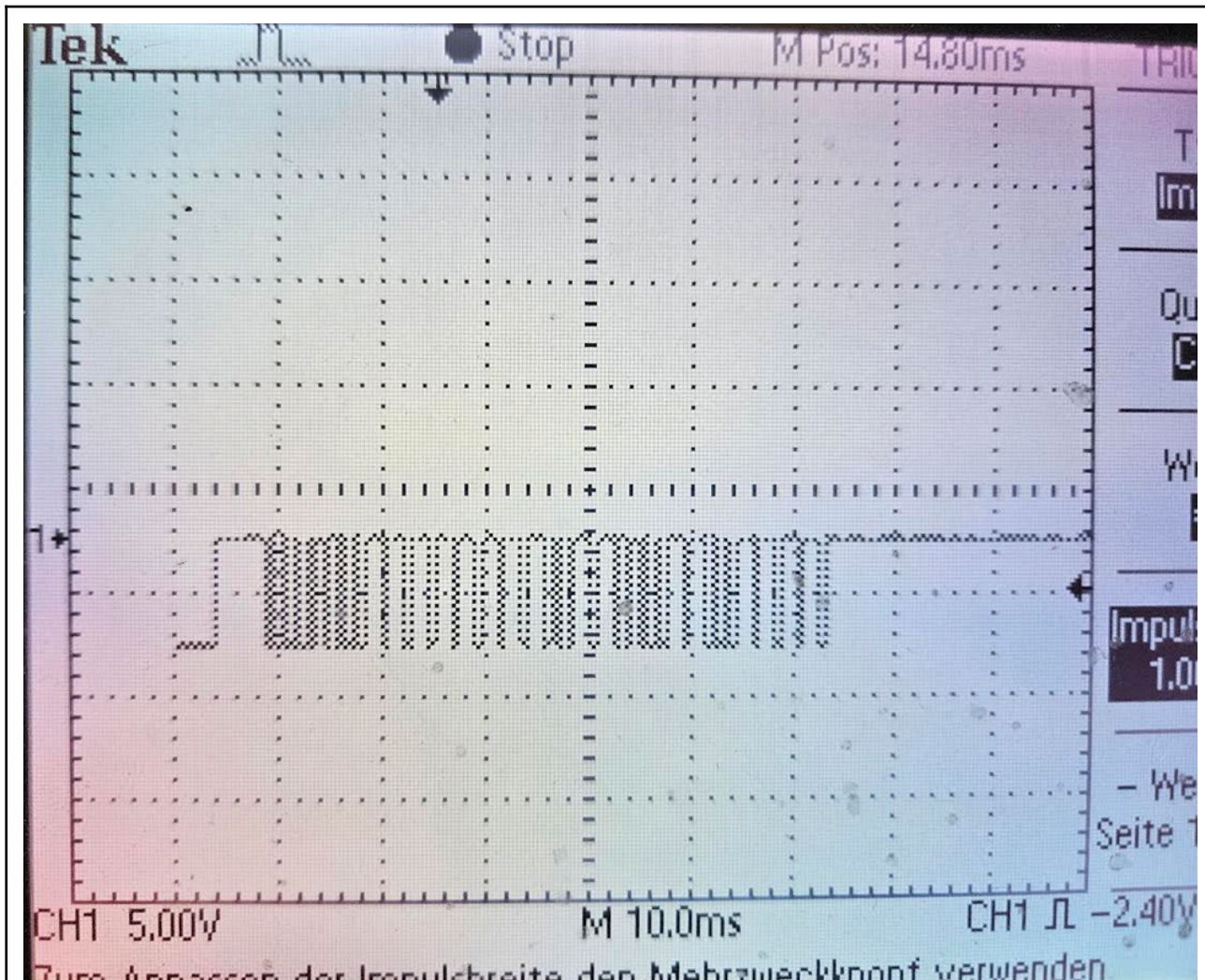


Figure 4.1.4: Trame observée lors de l'appuie d'un bouton Start sur la télécommande

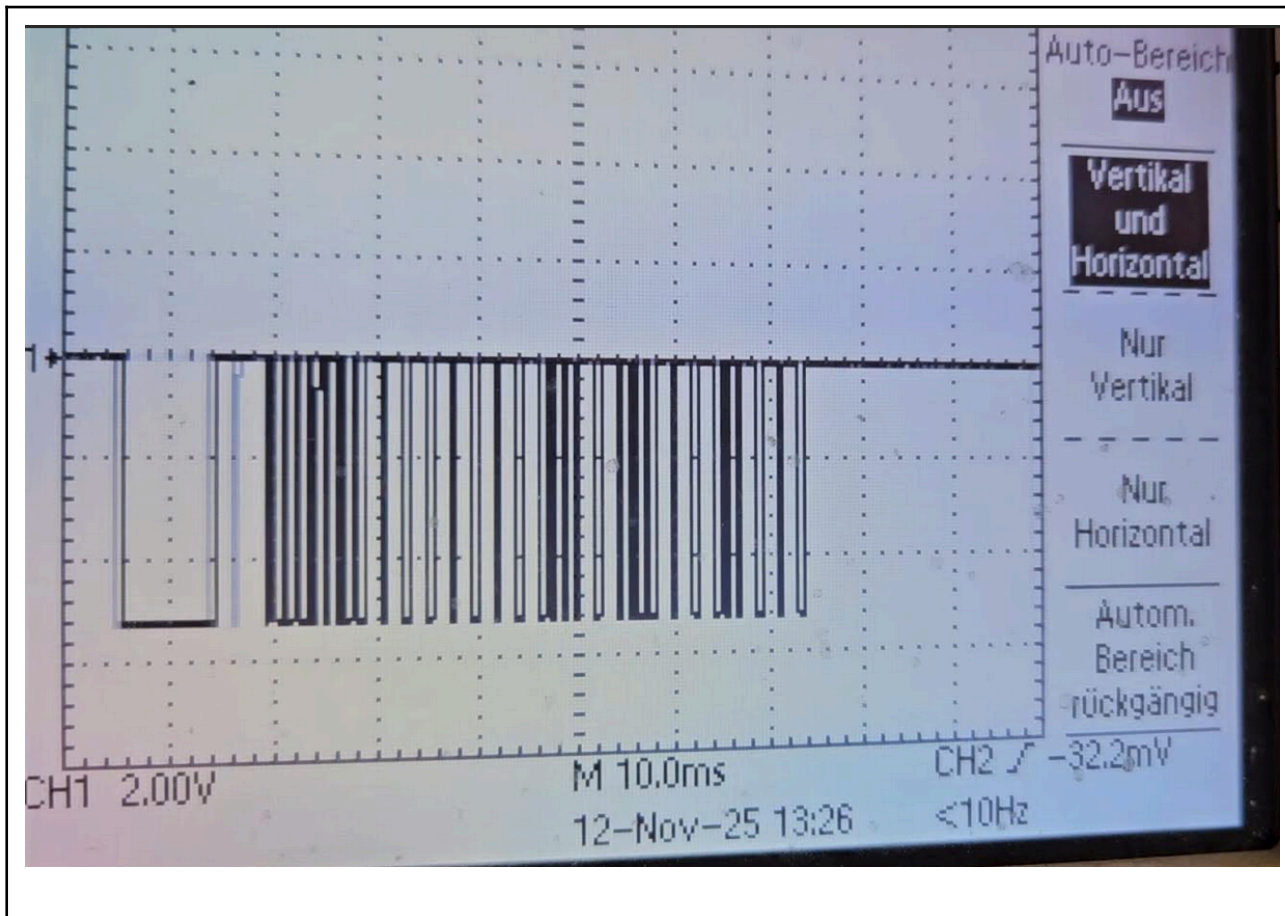


Figure 4.1.5: Trame observée lors de l'appuie d'un bouton Attention sur la télécommande

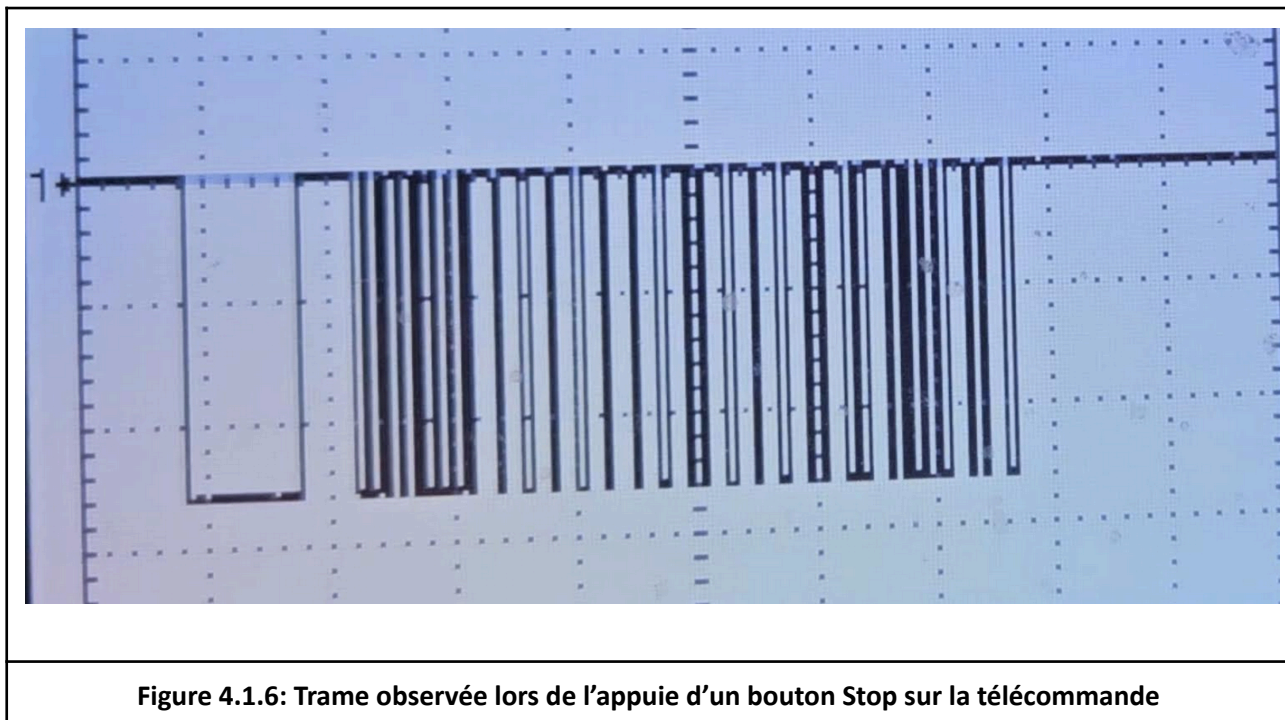


Figure 4.1.6: Trame observée lors de l'appuie d'un bouton Stop sur la télécommande

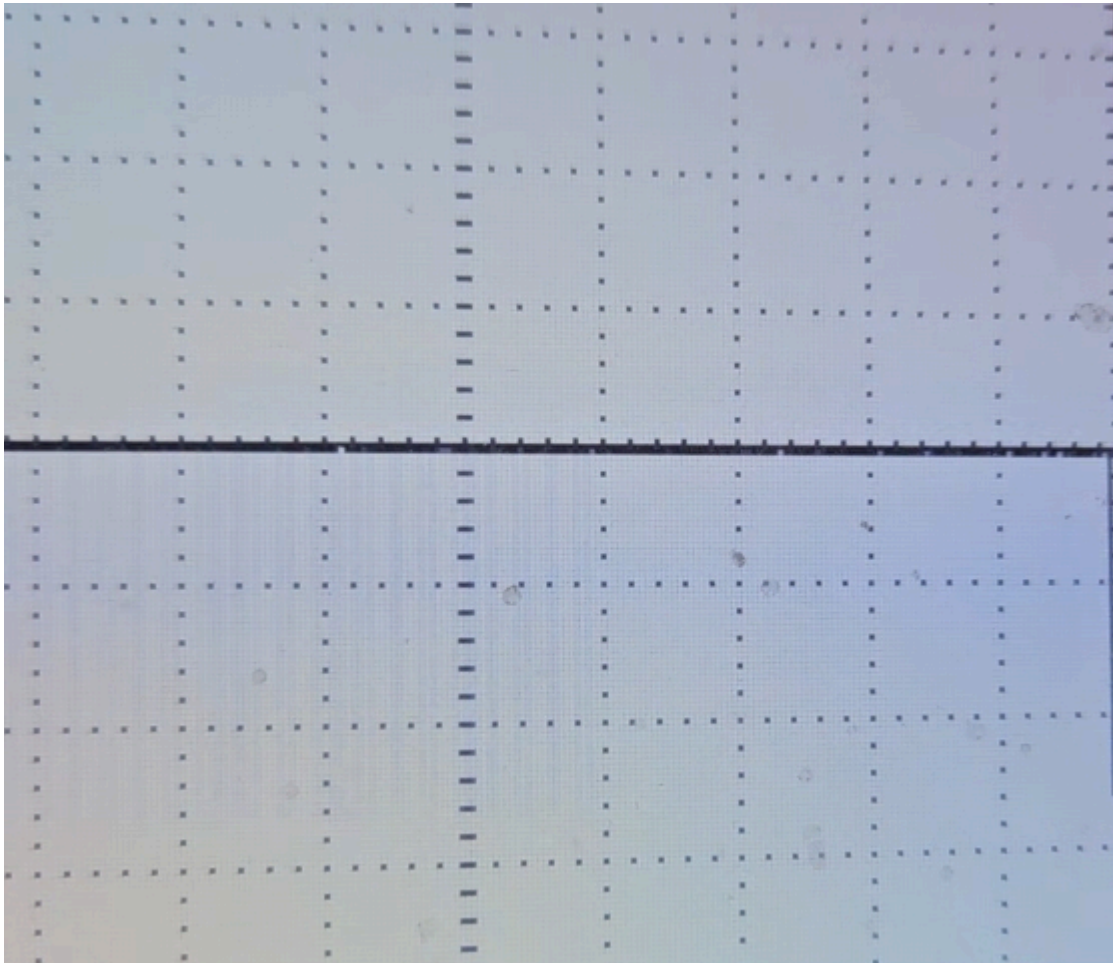


Figure 4.1.7: Trame observée lors de sans l'appuie sur la télécommande

Robot Mini-Sumo (RMS)

Sur l'oscilloscope on a un début de trame à 0000 0000, avant notre trame NEC. Les 0000 0000 initiaux correspondent au burst de démarrage de la trame NEC (9 ms de signal IR suivi de 4,5 ms de silence), qui est détecté comme une longue période de niveau bas par le capteur ; ce motif est interprété par le décodeur comme une synchronisation pour marquer le début de la trame, et ces zéros artificiels sont donc ignorés avant de lire les 32 bits de données réels qui suivent.

Nous décodons ainsi chacune des trames sans prendre en compte l'octet contenant les 0 :

Bouton	Trame en hexadécimal attendue	Trame en binaire attendu	Trame en binaire reçue
Start : bouton 1	0xFF30CF	1111 1111 0011 0000 1100 1111	1111 1111 0011 0000 1100 1111
Attention : bouton 2	0xFF18E7	1111 1111 0001 1000 1110 0111	1111 1111 0001 1000 1110 0111
Stop : bouton 3	0xFF7A85	1111 1111 0111 1010 1000 0101	1111 1111 0111 1010 1000 0101

Les tests s'étant révélés concluants, nous avons procédé à la conception du schéma électrique sous KiCad. Ce schéma intègre non seulement le capteur et son conditionnement (résistances de polarisation ainsi que condensateurs de découplage optimisés afin de minimiser le bruit), mais également les interfaces avec le microcontrôleur, notamment les brochages compatibles avec les GPIO.

Grandeur	Valeur mesurée	Conf/Non conf.
Trame NEC Lors de l'appui sur un bouton 1	0xFF30CF	Conforme
Trame NEC Lors de l'appui sur un bouton 2	0xFF18E7	Conforme
Trame NEC Lors de l'appui sur un bouton 3	0xFF7A85	Conforme
Aucune Trame NEC lorsqu'aucun bouton est appuyé	/	Conforme

Statut de l'essai : Conforme

Problèmes rencontrés : /

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	106/ 134
----------------------------------	---	-------------

4.1.2 Simulation des capteurs de sol

Référence de la simulation : SIM_Capteur_Contraste

Exigences client vérifiées : EXIG_LUMINOSITE

But de l'essai :

L'objectif de cet essai est de vérifier le bon fonctionnement du capteur de contraste. En effet, son bon fonctionnement est essentiel de la stratégie de combat. Lors des épreuves, l'éclairage ambiant ne doit pas perturber les capteurs de contraste ce qui pourrait fausser nos résultats.

Fichiers : [Schéma électrique Capteur sol](#) et [code informatique](#)

Procédure de simulation :

Pour confirmer le bon fonctionnement des capteurs dans notre système, nous avons réalisé des tests afin de valider leur capacité à détecter correctement par rapport à la distance au sol, la luminosité et les changements de couleur (noir/blanc).

À cet effet, nous avons développé un programme Arduino conçu pour évaluer la détection dans différents scénarios pour les trois capteurs.

```
#define capteur_avant_droit A1 // valeur renvoyée dans n_capteurs_L[i]: 15
#define capteur_avant_gauche A2 // valeur renvoyée dans n_capteurs_L[i]: 16
#define capteur_arriere A3 // valeur renvoyée dans n_capteurs_L[i]: 17

int valeurDeReference = 600; // seuil pour passer du blanc à noir trouvé par test
int n_capteurs_L[]={capteur_avant_droit,capteur_avant_gauche,capteur_arriere};

void setup() {
  Serial.begin(9600);
  pinMode(capteur_avant_droit, INPUT);
  pinMode(capteur_avant_gauche, INPUT);
  pinMode(capteur_arriere, INPUT);
}

char Capteur_Ligne(int i){
  int valeur = analogRead(n_capteurs_L[i]); // Lire la valeur analogique
  Serial.println(valeur); // affichage de la valeur sur le moniteur série
  if (valeur < valeurDeReference){ // comparaison de la valeur lue avec la valeur de référence
    return 'B'; } // B = surface blanche
  else{
    return 'N'; } // N = surface noire
}

void loop() {
  for (int i = 0; i < 3; i++) { // boucle qui permet de lire tout les capteurs
    Serial.print("Le Capteur ");
    Serial.print(n_capteurs_L[i]);
    Serial.print(" (A"); // broche analogique du capteur
    Serial.print(i+1); // numéro de la broche 1, 2 ou 3
    Serial.print(")");
    Serial.print(" est sur une surface : ");
    Serial.println(Capteur_Ligne(i)); // B ou N
  }
  delay(500); // Attendre 500 ms avant la prochaine lecture
}
```

Figure 4.1.2.1 : Code Arduino utilisé pour les capteurs sols

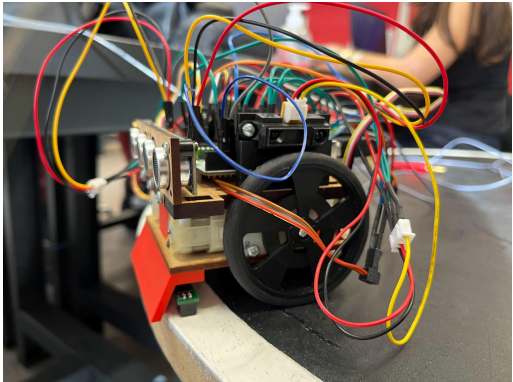
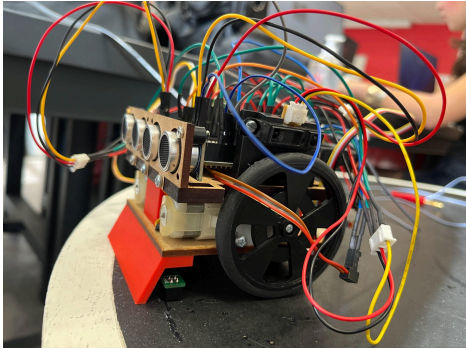
Robot Mini-Sumo (RMS)

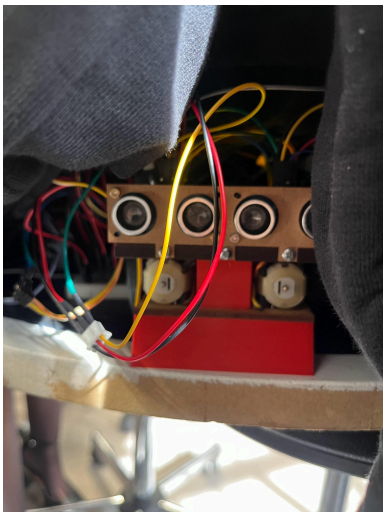
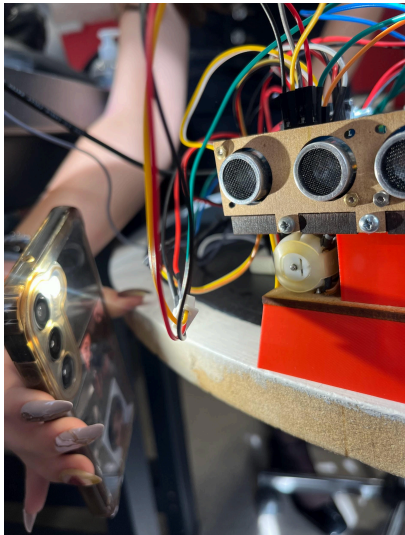
Résultats attendus :

A partir des exigences client issues du Cahier Des Charges, renseignez les valeurs attendues ci-dessous.

Grandeur	Valeur attendue	Tolérance
Ligne blanche détecté par 1 des 3 capteurs	blanc	/
Ligne noir détecté par les 3 capteurs	noir	/

Résultats obtenus :

Surface : Blanc	
Surface : Noir	

test dans le noir	
test avec un flash	
Figure 4.1.2.2 : Capteurs de contraste	

En testant différentes configurations (avec diverses luminosités, un capteur sur du blanc et les deux autres sur du noir, ou encore deux sur du noir et un sur du blanc, etc.), tous les tests se sont révélés concluants.

L'exigence luminosité est vérifiée car la lumière du jour est celle qui est la plus susceptible de perturber les capteurs de contraste. En effet, nos tests avec des flashes ainsi que la lumière de la salle ont été concluants néanmoins nous pouvons prouver cette conformité théoriquement. Les sources de lumière artificielle courantes (néons, LED) ont un spectre beaucoup plus étroit que le soleil et émettent une énergie significativement moins intense dans la plage spécifique du proche infrarouge (950 nm) utilisée par le capteur. Par conséquent, la quantité de rayonnement infrarouge parasite générée par l'éclairage de la pièce est beaucoup plus faible et plus facile à compenser ou à ignorer pour le circuit électronique du capteur. Un

flash d'appareil photo est une source de lumière très brève (impulsion de très courte durée) et est principalement optimisé pour le spectre visible (lumière blanche). Même s'il a une composante IR, sa courte durée et sa puissance IR relativement faible comparée au soleil limitent son impact sur la mesure du capteur.

Grandeur	Valeur mesurée	Conf/Non conf.
1 capteur sur la ligne blanche et 3 sur du noir	blanc	Conforme
2 capteur sur la ligne blanche et 1 sur du noir	blanc	Conforme
3 capteurs sur du noir	noir	Conforme

Statut de l'essai : Conforme

4.1.3 Simulation des capteurs télémétriques

Référence de la simulation : SIM_Capteurs_Télémétriques

Exigences client vérifiées : EXIG_ADVERSAIRE

But de l'essai :

L'objectif de cet essai est de vérifier le bon fonctionnement des capteurs télémétriques. En effet, leur bon fonctionnement est essentiel de la stratégie de combat, puisqu'il va en partie dicter l'action du robot sumo.

Fichiers : schéma électrique : [Schéma électrique - Google Drive](#) code : [capteurs_tele_fin.ino](#)

Procédure de simulation :

Pour vérifier le bon fonctionnement des capteurs télémétriques intégrés à notre système, nous avons mené une série de tests visant à confirmer leur capacité à détecter correctement un adversaire situé à droite ou à gauche du robot, sur une plage allant de 4 cm à 30 cm. Cette mesure de distance est également essentielle au début du match, lorsque les robots sont placés de part et d'autre de l'aire de combat. Ainsi, le capteur peut identifier un adversaire à très courte portée (4 cm) tout en offrant une distance suffisante pour suivre les déplacements latéraux durant l'affrontement.

Pour cela, nous avons développé un programme Arduino dédié à l'évaluation du fonctionnement des deux capteurs télémétriques.

Robot Mini-Sumo (RMS)

```
// Définition des broches des capteurs
#define tele_droite  A5
#define tele_gauche  A4

void setup() {
  Serial.begin(9600); // lancer la communication série à 9600 bauds
  pinMode(A5, INPUT);
  pinMode(A4, INPUT);
}

int Capteur_telemetrique_d(){
  // Lecture du capteur télémétrique droit
  int valeurBrute_d = analogRead(tele_droite);
  float tension_d = valeurBrute_d * (5.0 / 1023.0); // Calcul de la tension de la valeur récupérée par le capteur droit
  float distance_d = 13.0 * pow(tension_d, -1); // Estimation empirique de la distance entre le capteur télémétrique droit et l'adversaire
  // Affichage des résultats sur le moniteur série
  Serial.print("droite : ");
  Serial.print(distance_d);
  Serial.println(" cm");

  Serial.println();
}

int Capteur_telemetrique_g(){
  // Lecture du capteur télémétrique gauche
  int valeurBrute_g = analogRead(tele_gauche);
  float tension_g = valeurBrute_g * (5.0 / 1023.0); // Calcul de la tension de la valeur récupérée par le capteur gauche
  float distance_g = 13.0 * pow(tension_g, -1); // Estimation empirique de la distance entre le capteur télémétrique gauche et l'adversaire
  // Affichage des résultats sur le moniteur série
  Serial.print("gauche : ");
  Serial.print(distance_g);
  Serial.println(" cm");

  Serial.println();
}

int Capteur_telemetrique_g(){
  // Lecture du capteur télémétrique gauche
  int valeurBrute_g = analogRead(tele_gauche);
  float tension_g = valeurBrute_g * (5.0 / 1023.0); // Calcul de la tension de la valeur récupérée par le capteur gauche
  float distance_g = 13.0 * pow(tension_g, -1); // Estimation empirique de la distance entre le capteur télémétrique gauche et l'adversaire
  // Affichage des résultats sur le moniteur série
  Serial.print("gauche : ");
  Serial.print(distance_g);
  Serial.println(" cm");

  Serial.println();
}

void loop() {
  Capteur_telemetrique_g();
  Capteur_telemetrique_d();

  delay(300); // Pause entre deux mesures
}
```

Figure 4.1.3.1 : Code Arduino utilisé pour les capteurs télémétriques

La fonction lit la valeur du capteur télémétrique indiqué par `n_capteur` et renvoie sa position (droite ou gauche) ainsi que la distance qui le sépare de l'obstacle, c'est-à-dire l'adversaire.

Nous avons donc réalisé des tests pour chacun des capteurs (séparément et ensemble), en variant la distance de détection et en vérifiant leurs limites de fonctionnement. Les résultats ont confirmé que les capteurs répondent correctement et renvoient bien les valeurs attendues.

Dans notre code final, où tous les modules seront réunis, cette distance servira à déterminer si un adversaire est détecté ou non, et à quelle distance.

Grandeur	Valeur mesurée	Conf/Non conf.
Renvoie la distance quand un adversaire est à sa droite ou à sa gauche	distance en cm	Conforme
Ne renvoie rien quand il ne capte aucun adversaire	/	Conforme

Statut de l'essai : Conforme

4.1.4 Simulation des capteurs ultrasons

Référence de la simulation : SIM_Capteurs_avants

Exigences client vérifiées : EXIG_ADVERSAIRE

But de l'essai :

L'objectif de cet essai est de vérifier le bon fonctionnement des capteurs avant ultrasons. En effet, son bon fonctionnement est essentiel de la stratégie de combat.

Fichiers : [Schéma électrique Capteur Ultrasons](#) et [code informatique](#)

Procédure de simulation :

Pour valider le bon fonctionnement du capteur dans notre application, nous avons effectué des tests préliminaires visant à vérifier sa portée minimale et maximale, ainsi que l'angle maximal de détection de l'adversaire.

Nous avons ainsi rédigé un code Arduino simple, permettant de tester la détection dans plusieurs cas de figure.

```
// Définition des numéros de port
#define trigPin_d 12 // Trigger (émission) capteur droit
#define echoPin_d 13 // Echo (réception) capteur droit
#define trigPin_g 7 // Trigger (émission) capteur gauche
#define echoPin_g 8 // Echo (réception) capteur gauche

// Variables
long duree; // Durée de l'écho
int distance_d; // Distance mesurée pour le capteur droit
int distance_g; // Distance mesurée pour le capteur gauche
const int TOLERANCE = 2; // Tolérance en cm pour considérer les distances égales (précision des mesures selon les capteurs pouvant varier)
const int PERIMETRE = 57; // Périmètre de détection (77 cm(taille doyo) - 2*10 cm(taille robots))

void setup() {
  pinMode(trigPin_g, OUTPUT); // Configuration du port Trigger gauche comme SORTIE
  pinMode(echoPin_g, INPUT); // Configuration du port Echo gauche comme ENTREE
  pinMode(trigPin_d, OUTPUT); // Configuration du port Trigger droit comme SORTIE
  pinMode(echoPin_d, INPUT); // Configuration du port Echo droit comme ENTREE
  Serial.begin(9600); // Démarrage de la communication série
}
```

Robot Mini-Sumo (RMS)

```
// Fonction pour le capteur droit
int Capteur_Ultrason_d() {
    // Émission d'un signal de durée 10 microsecondes
    digitalWrite(trigPin_d, LOW);
    delayMicroseconds(5);
    digitalWrite(trigPin_d, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin_d, LOW);

    // Écoute de l'écho
    duree = pulseIn(echoPin_d, HIGH);

    // Calcul de la distance
    distance_d = (duree * 0.034) / 2;

    return distance_d; // Retourner la distance mesurée
}

// Fonction pour le capteur gauche
int Capteur_Ultrason_g() {
    // Émission d'un signal de durée 10 microsecondes
    digitalWrite(trigPin_g, LOW);
    delayMicroseconds(5);
    digitalWrite(trigPin_g, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin_g, LOW);

    // Écoute de l'écho
    duree = pulseIn(echoPin_g, HIGH);

    // Calcul de la distance
    distance_g = (duree * 0.034) / 2;

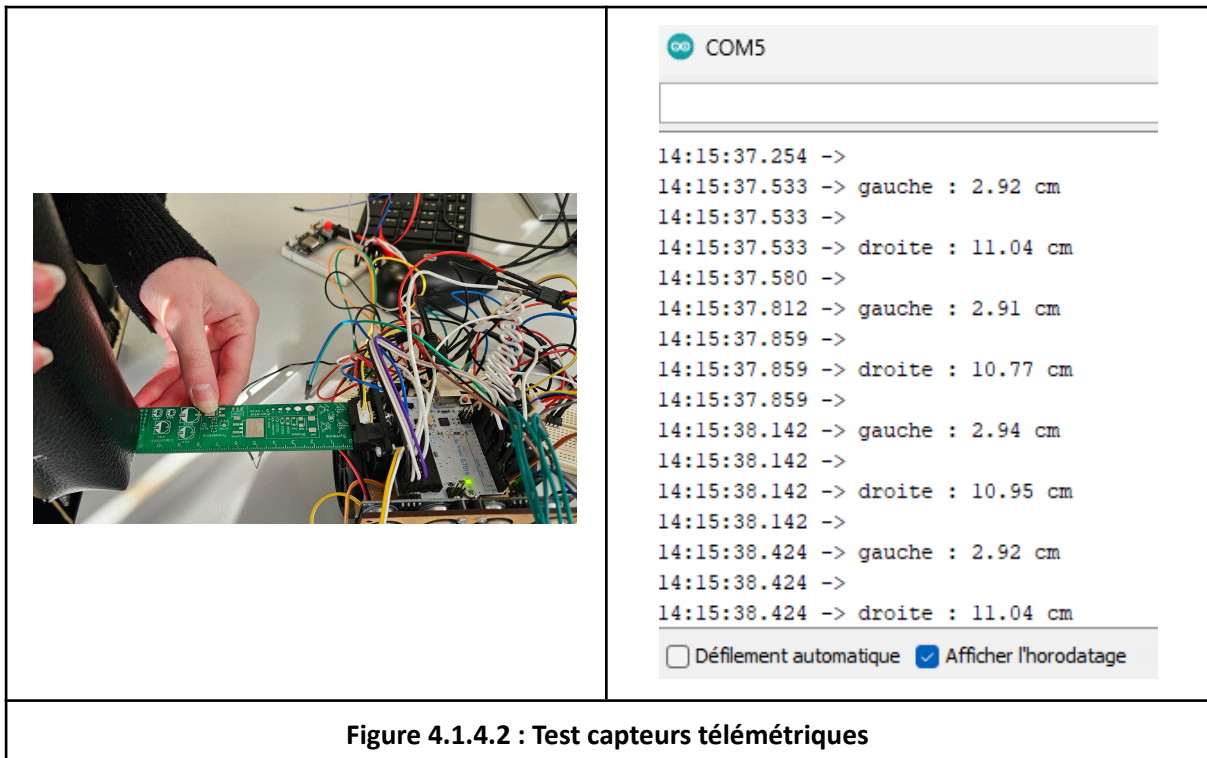
    return distance_g; // Retourner la distance mesurée
}

void loop() {
    dist_g = Capteur_Ultrason_g();
    dist_d = Capteur_Ultrason_d();
    if (dist_g <= PERIMETRE) {
        Serial.print("capteur gauche ");
        Serial.print(dist_g);
        Serial.println("cm");
    }
    if (dist_d <= PERIMETRE) {
        Serial.print("capteur droit ");
        Serial.print(dist_d);
        Serial.println("cm");
    }

    delay(1000); // Pause pour éviter des lectures trop rapides
}
```

Figure 4.1.4.1 : Code des capteurs ultrasons

Robot Mini-Sumo (RMS)



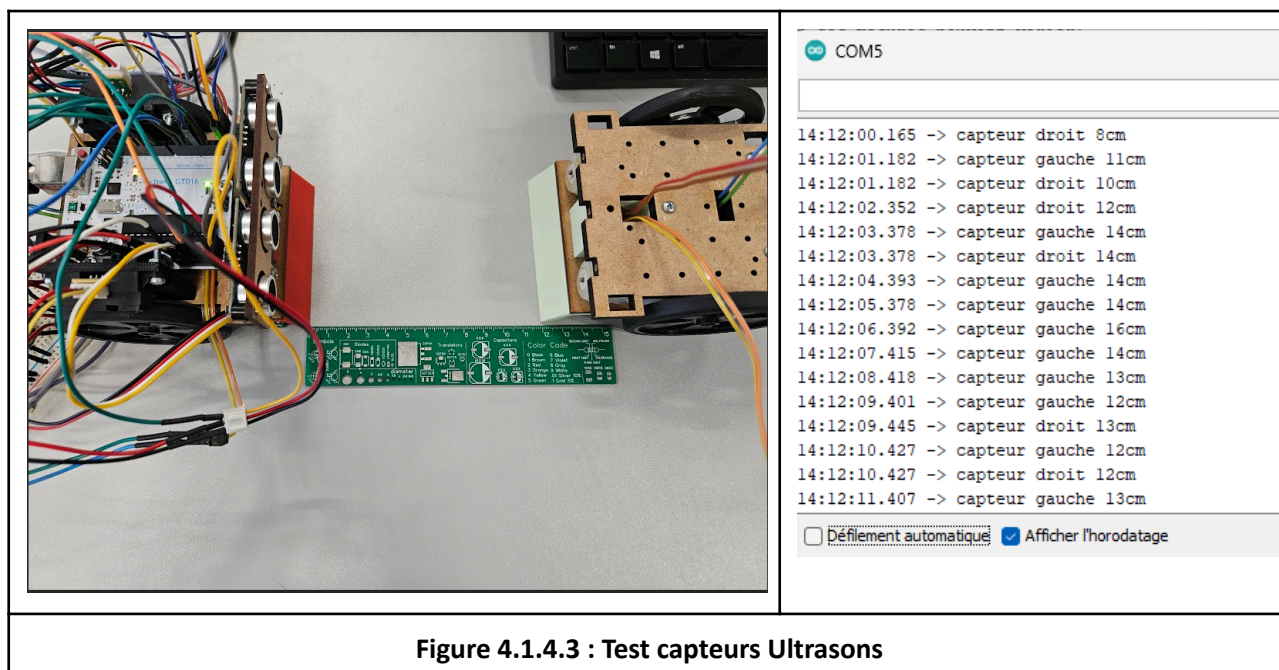
Résultats attendus :

Grandeur	Valeur attendue	Tolérance
Renvoie la distance quand un adversaire est devant lui	distance en cm	/
Ne renvoie rien quand il ne capte aucun adversaire	/	/

Résultats obtenus :

Ainsi on implémentant le code dans le robot sumo, placé à 14cm d'un robot sumo adverse nous obtenons les résultats suivants :

Robot Mini-Sumo (RMS)



Nous avons également effectué d'autres tests avec le robot à des distances variant de 2 cm à 60 cm. Les résultats se sont révélés concluants, sauf pour les distances inférieures à 3 cm. Toutefois, pour notre application, ce type de capteur est parfaitement adapté.

Grandeur	Valeur mesurée	Conf/Non conf.
Renvoie la distance quand un adversaire est devant lui	distance en cm	Conforme
Renvoie quand il ne capte aucun adversaire	0	Conforme

Statut de l'essai : Conforme

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	115/ 134
----------------------------------	---	-------------

Energie

Voici un schéma qui permet de dérisquer l'ensemble de la partie Énergie

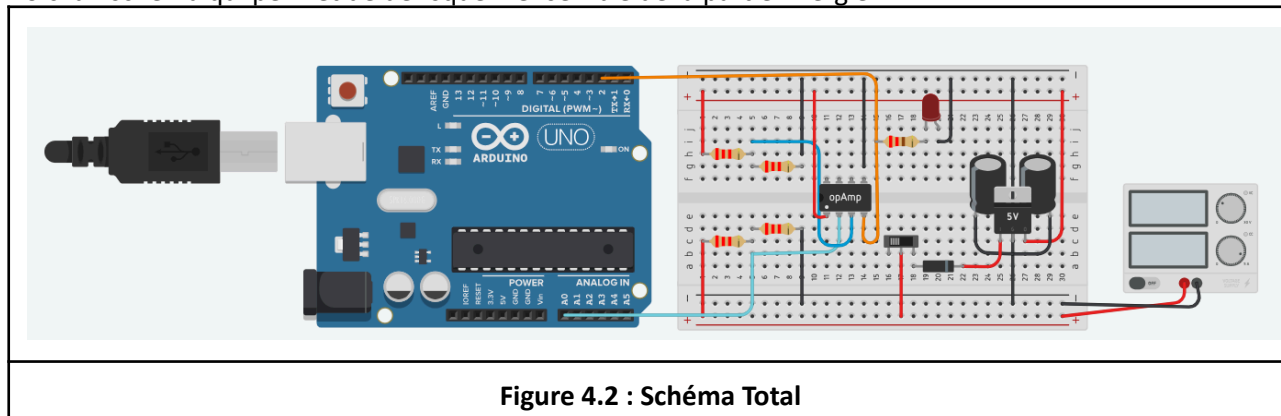


Figure 4.2 : Schéma Total

4.2.1 Prototypage rapide des ponts diviseur de tension de référence

Référence du test : SIM_Pont_Diviseur

Exigences client vérifiées : EXIG_SECUR_BATT

Commentaire de rédaction : Ce test a été réalisé avant le changement de la résistance R3 du pont diviseur. Un changement permettant de créer une tension de 3,4 V au lieu de 3,45 V.

But de l'essai : Vérifier par une mesure directe que la tension en sortie du pont diviseur de référence correspond bien à 3,45 V (+/- 2 %), pour une tension d'alimentation de 5 V.

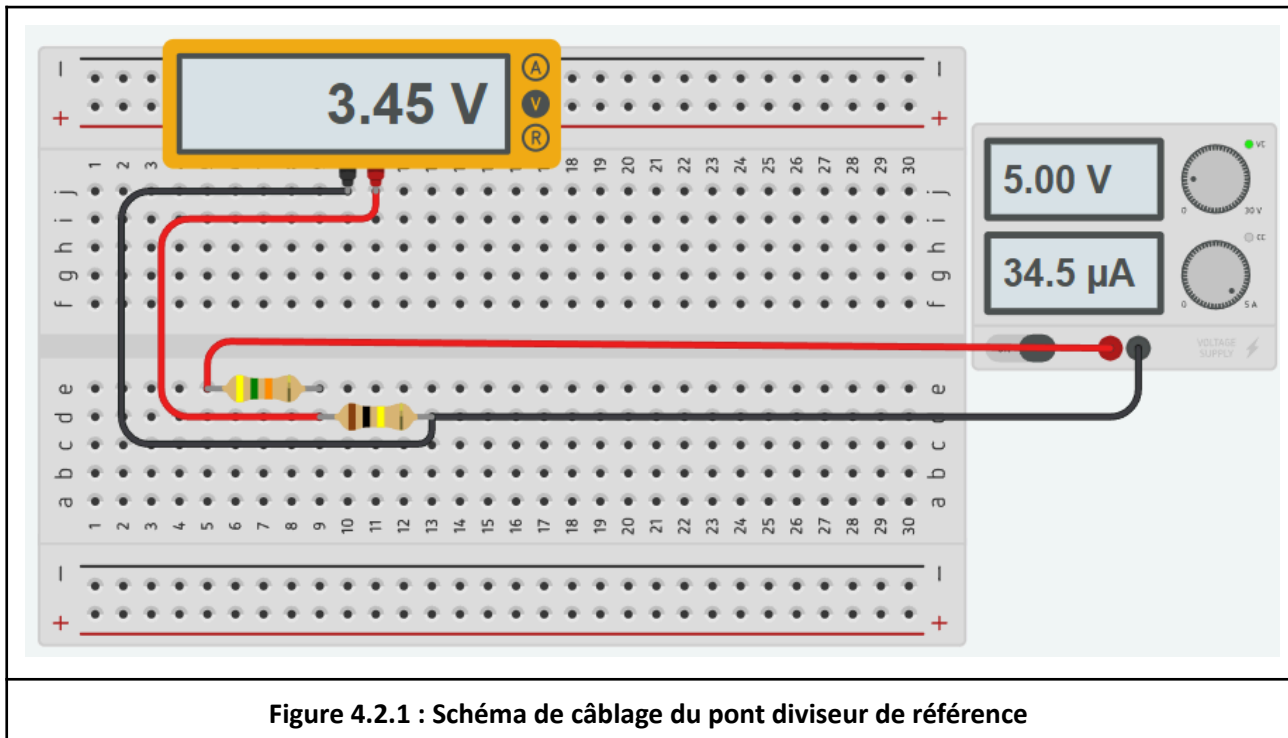
Fichiers : [Schéma de câblage](#)

Procédure de test :

- Réaliser le montage de la figure 4.2.1
- Régler l'alimentation de table à une tension de 5 V et à une limite de courant de 0,1 A
- Régler le voltmètre sur le calibre V DC.
- Allumer l'alimentation de table en appuyant sur OUTPUT
- Relever la valeur de la tension aux bornes de la résistance R4 comme le montre la figure 4.2.1
- Éteindre l'alimentation de table en appuyant sur OUTPUT
- Calculer à l'aide de la calculatrice l'erreur relative

$$Erreur\ relative = \frac{valeur\ obtenue - valeur\ attendue}{valeur\ attendue} \times 100 = \frac{valeur\ obtenue\ (en\ V) - 3,45}{3,45} \times 100$$

- Statuer de la conformité de l'essai

**Résultat attendu :**

La tension en sortie du pont diviseur de tension est de 3,45 V (+/- 2 %).

Grandeur	Valeur attendue	Tolérance
Tension en sortie du pont diviseur de tension (R4)	3,45 V	+/- 2 %

Résultat obtenu :

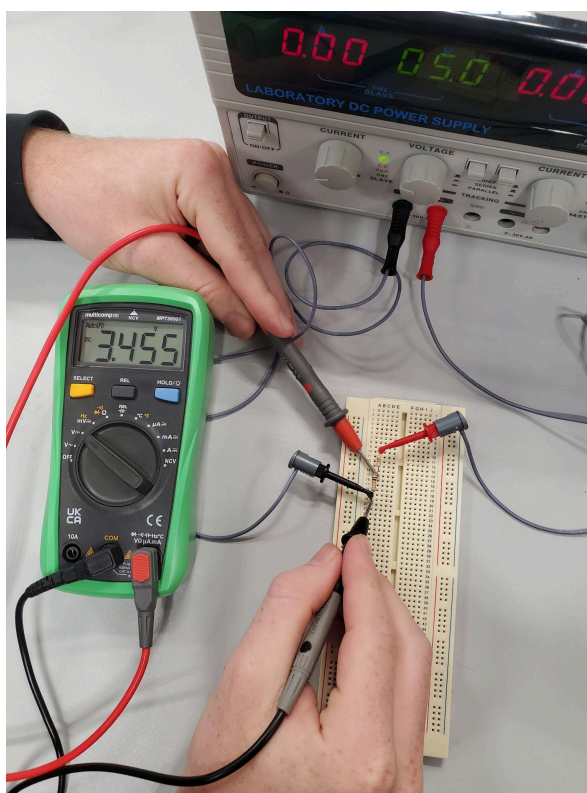


Figure 4.2.2 : Mesure de la tension en sortie du pont diviseur de tension de référence

Sur la figure 4.2.2, on relève une tension en sortie du pont diviseur de tension de 3,455 V.

$$\text{Erreur relative} = \frac{\text{valeur obtenue} - \text{valeur attendue}}{\text{valeur attendue}} \times 100 = \frac{3,455 - 3,45}{3,45} \times 100 = 0,14 \%$$

Grandeur	Valeur mesurée	Conf/Non conf.
Tension en sortie du pont diviseur (R4)	3,455 V	Conforme

Statut de l'essai : Le prototypage rapide confirme que le pont diviseur de tension est conforme aux exigences.

Problèmes rencontrés : Aucun

4.2.2 Prototypage rapide des ponts diviseur de tension Batterie

Référence du test : SIM_Pont_Diviseur

Exigences client vérifiées : EXIG_SECUR_BATT

But de l'essai : Vérifier par une mesure directe que la tension en sortie du pont diviseur de la batterie correspond bien à la tension d'alimentation divisée par deux.

Fichiers : [Schéma de câblage](#)

Procédure de test :

Robot Mini-Sumo (RMS)

- Réaliser le montage de la figure 4.3.1
- Régler l'alimentation de table à une tension de 8 V et à une limite de courant de 0,1 A
- Régler le voltmètre sur le calibre V DC.
- Allumer l'alimentation de table en appuyant sur OUTPUT
- Relever la valeur de la tension aux bornes de la résistance R2 comme le montre la figure 4.3.1
- Éteindre l'alimentation de table en appuyant sur OUTPUT
- Calculer à l'aide de la calculatrice l'erreur relative

$$Erreur\ relative = \frac{valeur\ obtenue - valeur\ attendue}{valeur\ attendue} \times 100 = \frac{valeur\ obtenue\ (en\ V) - 4}{4} \times 100$$

- Régler l'alimentation de table à une tension de 6,8 V et à une limite de courant de 0,1 A
- Allumer l'alimentation de table en appuyant sur OUTPUT
- Relever la valeur de la tension aux bornes de la résistance R2 comme le montre la figure 4.3.1
- Éteindre l'alimentation de table en appuyant sur OUTPUT
- Calculer à l'aide de la calculatrice l'erreur relative

$$Erreur\ relative = \frac{valeur\ obtenue - valeur\ attendue}{valeur\ attendue} \times 100 = \frac{valeur\ obtenue\ (en\ V) - 3,4}{3,4} \times 100$$

- Statuer de la conformité de l'essai

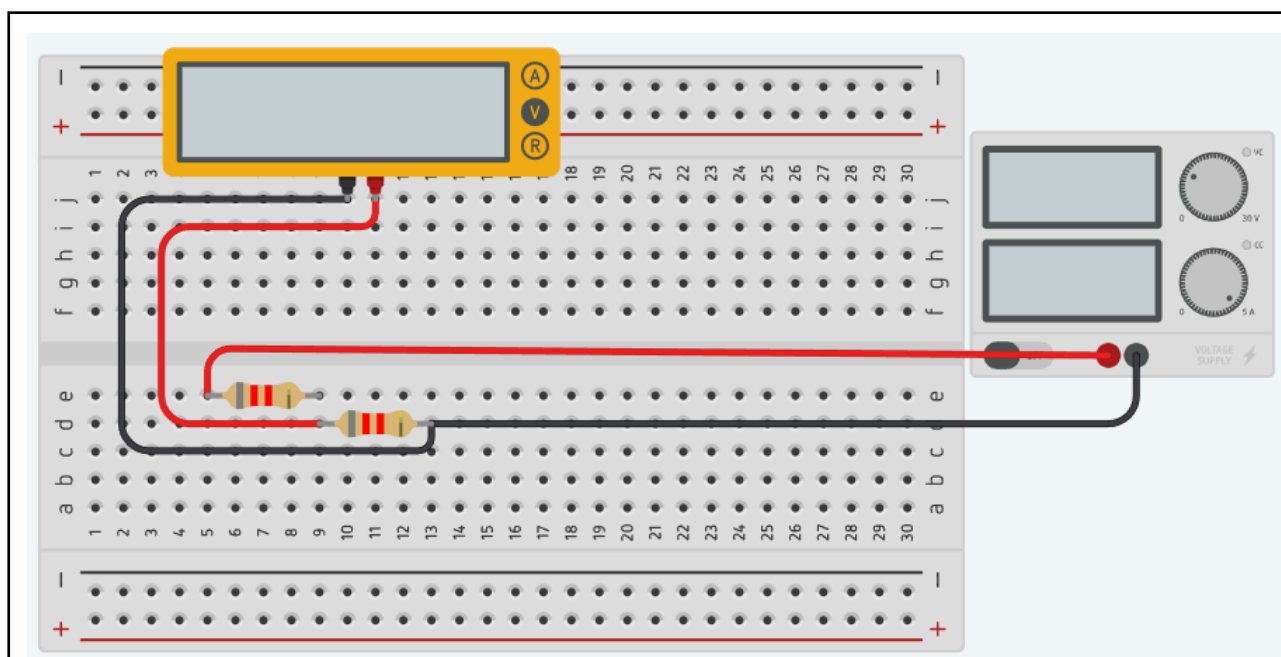


Figure 4.3.1 : Schéma de câblage du pont diviseur de la batterie

Résultat attendu :

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	119/ 134
----------------------------------	---	-------------

Grandeur	Valeur attendue	Tolérance
Tension en sortie du pont diviseur (R2) pour une tension d'alimentation de 8 V	4V	+/- 2%
Tension en sortie du pont diviseur (R2) pour une tension d'alimentation de 6,8 V	3,4	+/- 2%

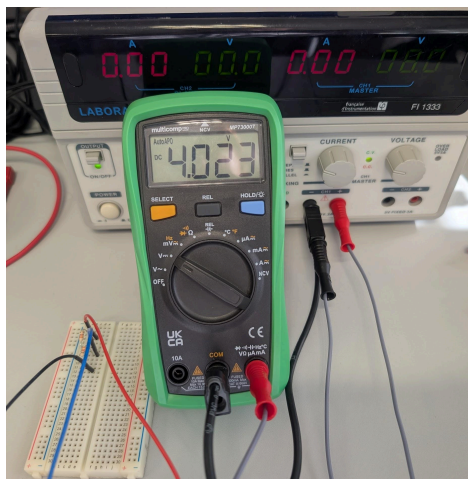
Résultat obtenu :

Figure 4.3.2 : Mesure de la tension en sortie du pont diviseur de tension de la batterie pour une tension d'alimentation de 8 V

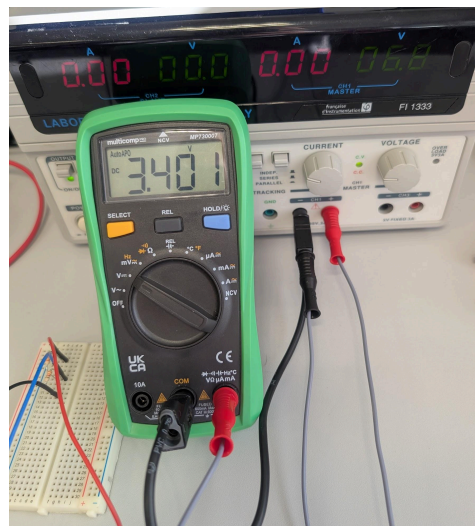


Figure 4.3.3 : Mesure de la tension en sortie du pont diviseur de tension de la batterie pour une tension d'alimentation de 6,8 V

Sur la figure 4.3.2, on relève une tension en sortie du pont diviseur de tension de 4,023 V.

$$\text{Erreur relative} = \frac{\text{valeur obtenue} - \text{valeur attendue}}{\text{valeur attendue}} \times 100 = \frac{4,023 - 4}{4} \times 100 = 0,58 \%$$

Sur la figure 4.3.3, on relève une tension en sortie du pont diviseur de tension de 3,401 V.

$$\text{Erreur relative} = \frac{\text{valeur obtenue} - \text{valeur attendue}}{\text{valeur attendue}} \times 100 = \frac{3,401 - 3,4}{3,4} \times 100 = 0,03 \%$$

Grandeur	Valeur mesurée	Conf/Non conf.
Tension en sortie du pont diviseur de tension (R2) pour une tension d'alimentation de 8 V	4,023 V	Conforme
Tension en sortie du pont diviseur de tension (R2) pour une tension d'alimentation de 6,8 V	3,401 V	Conforme

Statut de l'essai : Le prototypage rapide confirme que le pont diviseur de tension est conforme aux exigences.

Problèmes rencontrés : Aucun

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	120/ 134
----------------------------------	---	-------------

4.2.3 Simulation / prototypage Comparateur

Référence de la simulation : SIM_Comparateur

Exigences client vérifiées : EXIG_SECUR_BATT

But de l'essai : L'objectif de cette simulation est de vérifier que l'étage comparateur bascule correctement lorsque la tension de la batterie LiPo 2S descend sous 6,7 V, conformément à l'exigence EXIG_SECUR_BATT.

Plus précisément, il s'agit de valider la valeur de la tension de référence appliquée sur l'entrée du comparateur et de confirmer que la sortie du comparateur passe de l'état haut à l'état bas lorsque la tension batterie franchit ce seuil.

Fichiers : [Tinkercad](#)

Procédure de simulation :

Pour valider le bon fonctionnement du comparateur, nous avons d'abord réalisé séparément les deux ponts diviseurs :

- le pont diviseur "batterie", qui reproduit la tension issue de VBAT,
- le pont diviseur "seuil", qui génère la tension de référence appliquée sur l'autre entrée du comparateur.

Les sorties de ces deux ponts ont ensuite été connectées sur les entrées de l'amplificateur opérationnel configuré en mode comparateur (entrée non-inverseuse et entrée inverseuse).

L'amplificateur opérationnel est alimenté en +5 V / 0 V, fournis par une alimentation de laboratoire réglée à 5 V et limitée à 50 mA afin de protéger le montage en cas d'erreur de câblage.

Pour visualiser l'état logique de la sortie du comparateur, une LED en série avec sa résistance de limitation a été raccordée à la sortie. L'allumage ou l'extinction de la LED permet de vérifier le basculement du comparateur lorsque la tension batterie simulée franchit le seuil.

L'ensemble du montage correspond fidèlement au schéma présenté figure X, sur lequel tous les composants et connexions utilisés dans la simulation sont représentés.

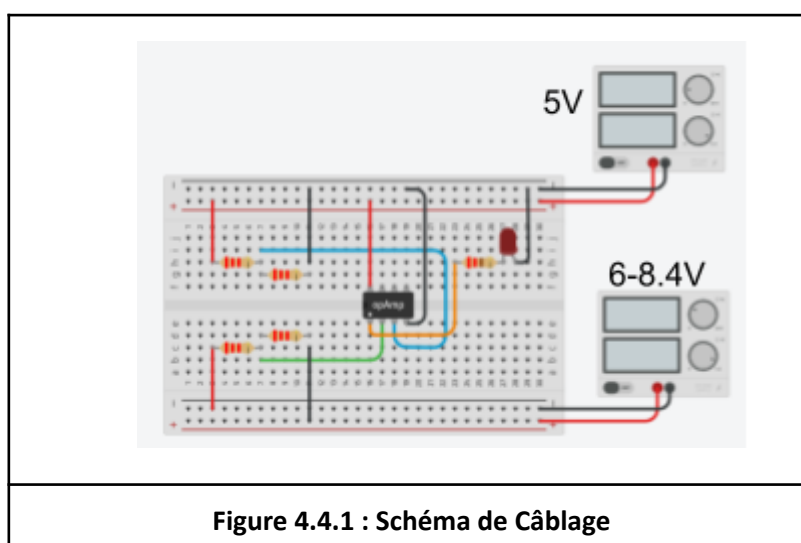


Figure 4.4.1 : Schéma de Câblage

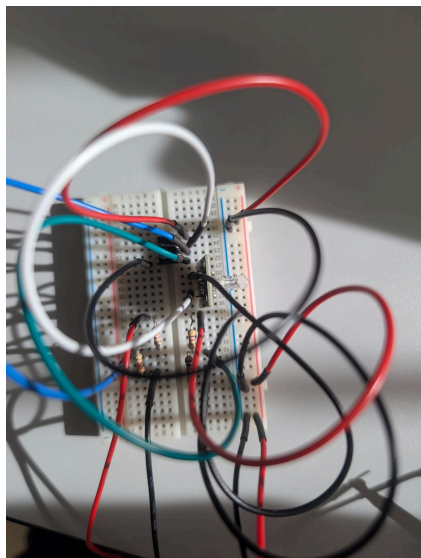


Figure 4.4.2 : Câblage sur Breadboard

Résultats attendus :

Lorsque la tension mesurée du côté “batterie” devient inférieure à la tension de seuil, le comparateur place sa sortie à 5 V, ce qui correspond à un état logique haut (1) pour l’Arduino sur l’entrée numérique (pin 4).

À l’inverse, lorsque la tension batterie est supérieure au seuil, la sortie du comparateur retombe à 0 V, donc en état logique bas (0).

Nous avons pris une tolérance de 0.7 V pour être sûr de valider l’exigence.

Tension Entrée	Tension Sortie attendue
Tension Batterie > 6.2V + Tolérance	5V état Haut
Tension Batterie < 6.2V	0V État Bas

Résultats obtenus :

Après avoir fait varier la tension simulant la batterie (à l’aide de l’alimentation de laboratoire), nous avons observé l’état de la LED. Ces observations nous ont permis de constater que :

Grandeur Entre	Tension sortie Comparateur	Conf/Non conf.
Tension batterie > 6.2 + Tolérance	(Led ON) 5V	Conforme
Tension batterie < 6.2	0V	Conforme

Statut de l’essai : Nous pouvons en conclure que l’étage de vérification de la tension batterie est capable de détecter correctement qu’une batterie est déchargée. Le fonctionnement observé est donc conforme à l’exigence de sécurité batterie.

Problèmes rencontrés : /NAN

4.2.4 Prototypage rapide Diode et Interrupteur

Référence de la simulation : SIM_Diode_Interrupteur

But de l'essai : Vérifier le bon fonctionnement de la diode de protection (protection contre inversion de polarité) et de l'interrupteur principal.

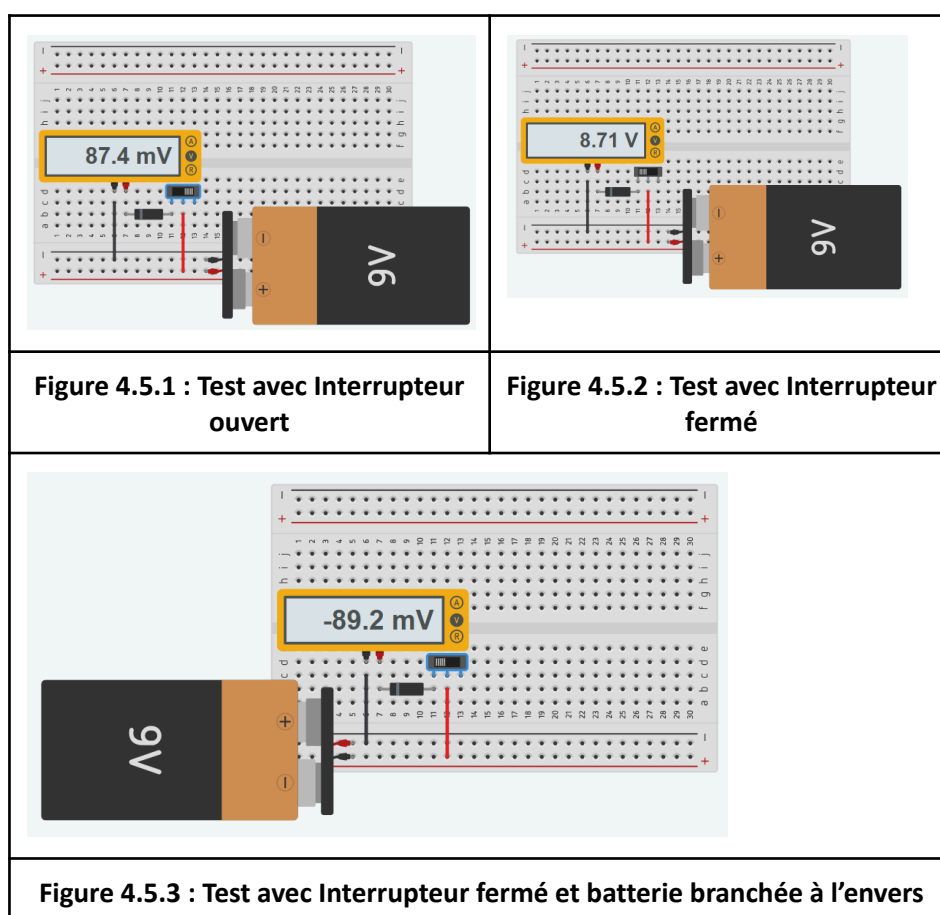
Fichiers : [Tinkercad](#)

Procédure de simulation :

Pour valider le comportement de l'interrupteur et de la diode de protection, deux montages ont été réalisés sur breadboard :

- Un montage avec multimètre

Ce montage permet de mesurer la tension disponible après la diode, en fonction de l'état de l'interrupteur et du sens de branchement de la batterie.



- Un montage avec LED témoin

Ce montage permet d'observer visuellement si le courant circule ou non.

La LED s'allume uniquement si l'alimentation passe correctement par la diode et l'interrupteur.

Robot Mini-Sumo (RMS)

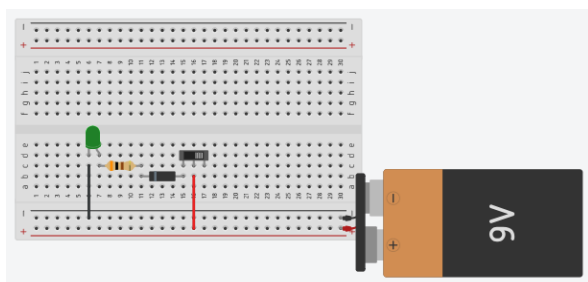


Figure 4.5.4 : Test avec Interrupteur ouvert

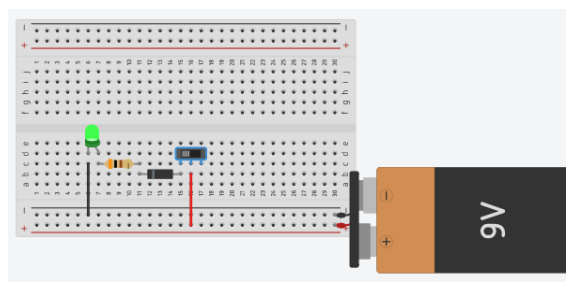


Figure 4.5.5 : Test avec Interrupteur fermé

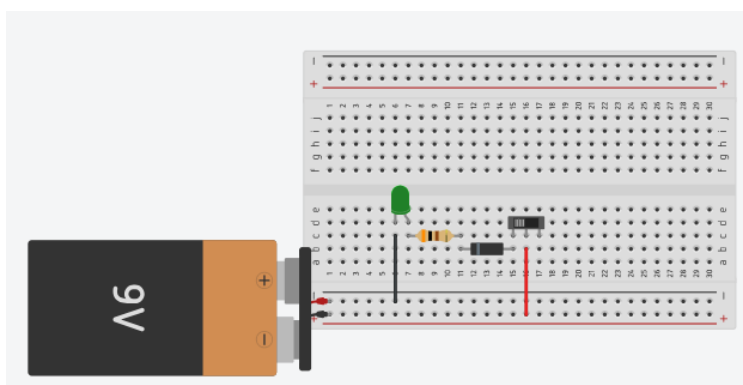


Figure 4.5.6 : Test avec Interrupteur fermé et batterie brancher sur a l'envers

Dans les deux cas, la diode est placée en série avec l'alimentation pour bloquer toute inversion de polarité, et l'interrupteur coupe ou autorise l'alimentation du reste du circuit.

Résultats attendus :

Etat Entrée	Etat attendue
Interrupteur ouvert → aucune tension disponible après la diode	LED éteinte / multimètre $\approx 0\text{ V}$
Interrupteur fermé → la tension passe normalement	LED allumée / multimètre $\approx$ tension batterie – chute de diode
Batterie inversée → diode bloque	aucune tension distribuée → LED éteinte / multimètre $\approx 0\text{ V}$

Résultats obtenus :

Les tests effectués confirment le comportement attendu :

- Lorsque l'interrupteur est **ouvert**, aucune tension n'est présente après la diode, la LED reste éteinte.
- Lorsque l'interrupteur est **fermé**, la tension mesurée est conforme au fonctionnement d'une diode en série (tension batterie moins la chute de diode), et la LED s'allume.
- Lorsque la batterie est **branchée à l'envers**, la diode bloque totalement le courant : la tension mesurée est de 0 V et la LED reste éteinte, ce qui montre que la protection contre inversion fonctionne correctement.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	124/ 134
----------------------------------	---	-------------

Grandeur	Conf/Non conf.
Fonctionnement diode + switch	Conforme

Statut de l'essai : La diode de protection et l'interrupteur assurent correctement leur rôle respectif :

- l'interrupteur coupe/autorise l'alimentation,
- la diode protège efficacement contre une inversion de polarité.

L'essai est donc conforme.

Problèmes rencontrés : /NAN

Action

4.3 Prototypage rapide pont en H

Chaque bloc fonctionnel ainsi que l'ensemble du montage doit faire l'objet d'une fiche de simulation / prototypage rapide, présentée comme suit.

Référence de la simulation : SIM_Polulu

Exigences client vérifiées : EXIG_DEPLACEMENT

But de l'essai : Le but de la simulation est de pouvoir vérifier si nous pouvons faire avancer, reculer et tourner le robot grâce au pont en H polulu DRV8835

Fichiers :

Procédure de simulation :

- Réaliser le montage de la figure X
- Régler les alimentations de table à une tension de 9V pour la partie moteur et 5V pour la partie logique
- Allumer l'alimentation de table en appuyant sur OUTPUT
- Ouvrir IDE Arduino sur l'ordinateur
- Téléverser le code de la figure 4.6.1
- Réinitialiser la carte arduino
- Vérifier que le robot sumo avance, recule et tourne
- Statuer de la conformité de l'essai

Robot Mini-Sumo (RMS)

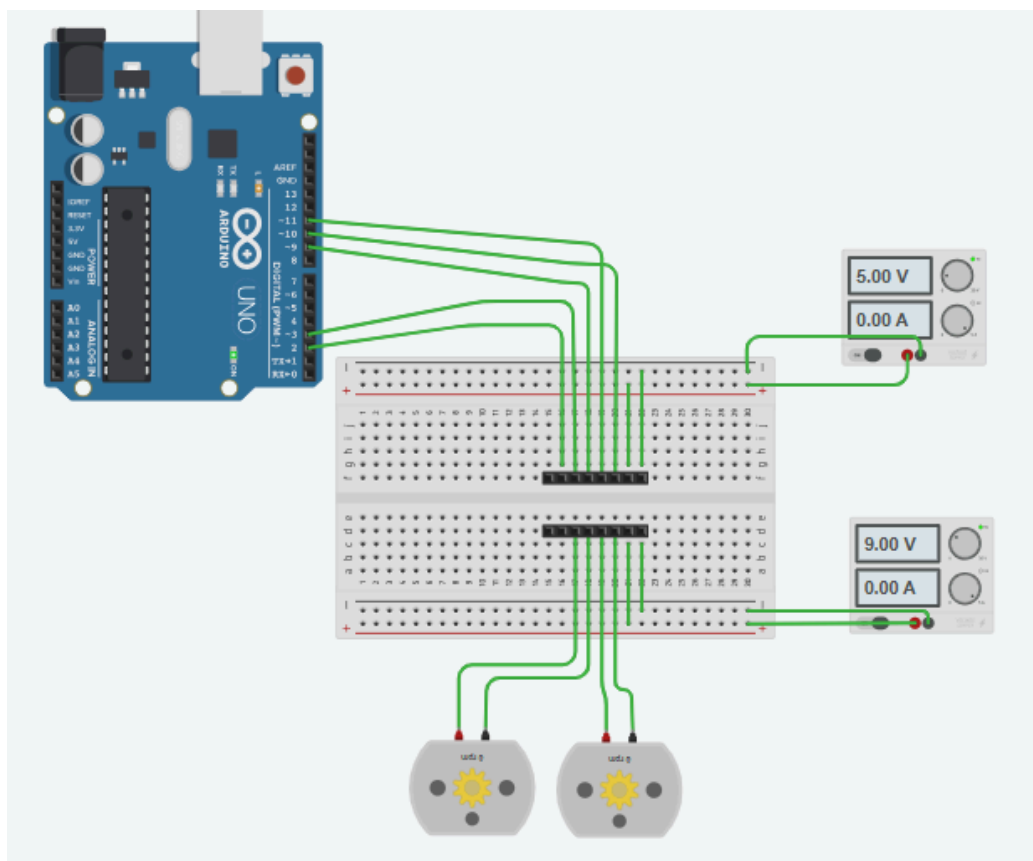


Figure 4.6.1 : Schéma électrique du prototypage rapide

```

int MOTA = 9;
int MOTB = 10;
int PWMA = 11;
int PWMB = 6;
int HIGH_MODE = 12;

void setup() {
  pinMode(MOTA, OUTPUT);
  pinMode(MOTB, OUTPUT);
  pinMode(PWMA, OUTPUT);
  pinMode(PWMB, OUTPUT);
  pinMode(HIGH_MODE, OUTPUT);
  digitalWrite(HIGH_MODE, 1);

  //Moteur dans le sens horraire
  digitalWrite(MOTA, 1);
  digitalWrite(MOTB, 1);

  //On avance avec une vitesse à 100%
  analogWrite(PWMA, 255);
  analogWrite(PWMB, 255);
  delay(1000);
  //On tourne en avançant
  analogWrite(PWMA, 255);
  analogWrite(PWMB, 0);
  delay(500);
  //On tourne en avançant mais dans l'autre sens
  analogWrite(PWMA, 0);
  analogWrite(PWMB, 255);
  delay(1000);
  //On change le sens des moteurs
  digitalWrite(MOTA, 0);
  digitalWrite(MOTB, 0);
  //On recule avec une vitesse à 100%
  analogWrite(PWMA, 255);
  analogWrite(PWMB, 255);

}

```

Figure 4.6.2 : Code de test du prototypage rapide

Résultats attendus :

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ21 Révision : 2 – 12/11/2025	127/ 134
----------------------------------	---	-------------

Grandeur	Valeur attendue	Tolérance
Avance	Action effectuée	Sans objet
Reculé	Action effectuée	Sans objet
Tourne	Action effectuée	Sans objet

Résultats obtenus :

Après avoir réaliser le schéma sur la breadboard comme sur la figure 4.6.2 :

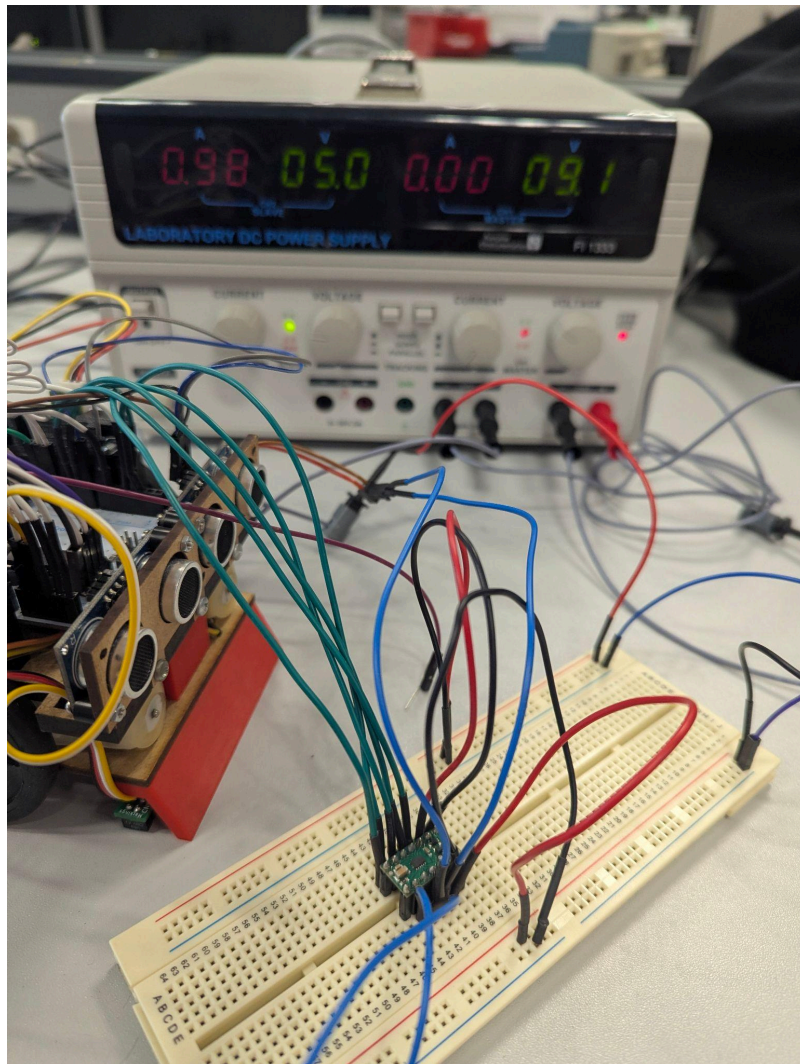


Figure 4.6.3 : Prototypage rapide sur breadboard

Nous avons donc obtenues les résultats suivant

Robot Mini-Sumo (RMS)

Grandeur	Valeur mesurée	Conf/Non conf.
Avance	Action effectuée	Sans objet
Reculé	Action effectuée	Sans objet
Tourne	Action effectuée	Sans objet

Statut de l'essai :

L'essai est conforme, le robot a pu avancer, tourner ainsi que reculer sans problème.

Problèmes rencontrés :

Aucun problème n'as été rencontrés lors de ce prototypage rapide

4.3 Conclusion de la simulation / prototypage rapide du produit

Après avoir simulé et prototyper les différents étages du produit (Acquisition, Energie, Action, traitement), nous pouvons confirmer que la carte se comporte conformément aux attentes de la phase de conception détaillée.

Les essais réalisés ont validé le dimensionnement des composants ainsi que le fonctionnement général de chaque bloc.

Aucune non-conformité significative n'a été observée.

Ces validations successives montrent que la carte est prête à passer en phase de fabrication, les différents essais ayant démontré que les choix réalisés en conception sont pertinents et robustes.

5. Conclusion de la conception du produit

Les exigences clients spécifiées dans le cahier des charges ont bien été traitées et le dimensionnement des composants qui en ressort nous permet théoriquement de fournir un prototype entièrement conforme à ces exigences.

6. Matrice de conformité du produit

Ce chapitre synthétise par l'intermédiaire d'un tableau la conformité du produit développé par rapport aux exigences issues du Cahier des Charges.

Exigence	Méthodes Vérification	Paramètre en lien avec l'exigence	Statut
EXIG_COMPORTEMENT	CONCEPTION DÉRISQUAGE VÉRIFICATION	CPR01 CPR09 CPR13 CDT01 CDT17	Conf.
EXIG_AUTONOMIE	VÉRIFICATION FABRICATION	CPR01 CPR07 CPR08 CPR09 CPR12 CDT01 CDT13	Conf.
EXIG_SECUR_BATT	CONCEPTION DÉRISQUAGE VÉRIFICATION FABRICATION	CPR01 CPR07 CPR08 CPR09 CPR12 CDT01 CDT09 CDT10	Conf.

Robot Mini-Sumo (RMS)

		CDT11 CDT12 SIM_Pont_Diviseur SIM_Comparateur FAB07	
EXIG_DEPLACEMENT	CONCEPTION DÉRISQUAGE VÉRIFICATION	CPR01 CPR06 CPR09 CPR11 CDT01 CDT06 SIM_Polulu	Conf.
EXIG_ADVERSAIRE	CONCEPTION DÉRISQUAGE VÉRIFICATION	CPR01 CPR02 CPR03 CPR09 CPR10 CDT01 CDT02 CDT05 SIM_Capteurs_Téléométriques SIM_Capteurs_avants	Conf.
EXIG_LUMINOSITE	CONCEPTION DÉRISQUAGE VÉRIFICATION	CPR01 CPR04 CPR09 CDT01 CDT03 SIM_Capteur_Contraste	Conf.
EXIG_DEPART	CONCEPTION DÉRISQUAGE VÉRIFICATION	CPR01 CPR05 CPR09	Conf.

Robot Mini-Sumo (RMS)

		CPR10 CDT01 CDT04 SIM_Capteur_IR	
EXIG_CARTE	CONCEPTION DÉRISQUAGE VÉRIFICATION FABRICATION	CPR01 CPR09 CDT01	Conf.
EXIG_COUT	Etude et calcul	CDT18	Conf.
EXIG_CHASSIS_DIMENSIONS	VÉRIFICATION : par inspection documentaire & par mesure	CPR01 CPR09 CDT01	Conf.
EXIG_MASSE	VÉRIFICATION : par mesure	CPR01 CPR09 CDT01	Conf.
EXIG_INTEGRITE_MECA	VÉRIFICATION	CPR01 CPR09 CDT01	Conf.
EXIG_FIXATION_CARTE	VÉRIFICATION FABRICATION	CPR01 CPR09 CDT01 FAB01 FAB02 FAB03 FAB04 FAB05 FAB06	Conf.
EXIG_DELAI	Réalisation du projet	CDT19	Conf.
EXIG_DRIVE	Réalisation du projet	Tout	Conf.

Robot Mini-Sumo (RMS)

EXIG_FORMAT_DOC	Réalisation du projet	Tout	Conf.
EXIG_NOM_DOC	Réalisation du projet	Tout	Conf.